Timeline Compiler

Technical Specification & Architecture

Timeline Compiler Project

June 2026

Contents

0.1 Problem Definition . . . 10
0.1.1 The Communication Gap . . . 10
0.1.2 Why Existing Tools Are Insufficient . . . 10
0.1.2.1 Mermaid and Documentation-Grade Diagramming . . . 10
0.1.2.2 Project Management Tools . . . 11
0.1.2.3 Design and Presentation Tools . . . 11
0.1.2.4 AI Agent Blindspot . . . 11
0.1.3 Who This Is For . . . 12
0.1.3.1 Human Authors . . . 12
0.1.3.2 AI Agents . . . 12
0.1.4 What Success Looks Like . . . 12
0.1.4.1 Scenario: Product Roadmap . . . 12
0.1.4.2 Scenario: Transformation Plan . . . 13
0.1.4.3 Success Criteria . . . 14
0.1.5 What This Is Not . . . 14
0.2 Design Principles . . . 15
0.2.1 Presentation-First . . . 15
0.2.2 Deterministic Output . . . 15
0.2.3 Human-Readable Source . . . 15
0.2.4 Agent-Friendly Authoring . . . 16
0.2.5 Theme-Driven Rendering . . . 16
0.2.6 Separation of Planning and Rendering . . . 17
0.2.7 Minimal Visual Clutter . . . 17
0.2.8 Source-Control Friendly . . . 17
0.2.9 Composability . . . 18
0.2.10 Graceful Degradation . . . 18

0.2.11 Extensibility Without Complexity . . . 19
0.2.12 Principle Summary . . . 19
0.3 Scope Boundaries . . . 19
0.3.1 Governing Principle . . . 19
0.3.2 In Scope . . . 20
0.3.2.1 Visual Artifact Types . . . 20
0.3.2.2 IR Elements . . . 20
0.3.2.3 Presentation Features . . . 21
0.3.2.4 Output Formats . . . 21
0.3.2.5 Tooling . . . 21
0.3.3 Out of Scope . . . 22
0.3.3.1 Project Management Semantics . . . 22
0.3.3.2 Data Management . . . 23
0.3.3.3 Interactive Features . . . 23
0.3.3.4 Data Ingestion . . . 23
0.3.4 Boundary Cases . . . 23
0.3.4.1 Dependency Arrows (Visual Only) . . . 23
0.3.4.2 Today Line . . . 24
0.3.4.3 Auto-Layout . . . 24
0.3.4.4 External Image Embedding . . . 24
0.3.5 Scope Summary . . . 24
0.4 Timeline IR . . . 24
0.4.1 Design Principles . . . 24
0.4.2 Type Vocabulary . . . 25
0.4.3 Document Structure . . . 25
0.4.3.1 Root Fields . . . 26
0.4.4 Metadata . . . 26
0.4.4.1 TimeRange . . . 27
0.4.4.2 AxisUnit . . . 27
0.4.4.3 Date Anchor and Fiscal Calendar . . . 27

0.4.5 Date Model . . . 28
0.4.5.1 Date Representations . . . 28
0.4.5.2 Uncertain and Open Dates . . . 29
0.4.5.3 DateRange . . . 29
0.4.6 Tracks (Swimlanes) . . . 30
0.4.7 Groups . . . 30
0.4.8 Activities . . . 31
0.4.8.1 Status Enum . . . 32
0.4.8.2 Progress . . . 33
0.4.9 Milestones . . . 33
0.4.10 Annotations . . . 34
0.4.10.1 Annotation Types . . . 34
0.4.11 Sections . . . 34
0.4.12 Legend . . . 35
0.4.13 ID and Reference System . . . 35
0.4.13.1 Identifier Format . . . 35
0.4.13.2 References . . . 36
0.4.13.3 Reference Namespacing . . . 37
0.4.14 Well-Formedness Rules . . . 37
0.4.14.1 Structural Invariants . . . 37
0.4.14.2 Referential Invariants . . . 37
0.4.14.3 Temporal Invariants . . . 37
0.4.14.4 Value Invariants . . . 38
0.4.14.5 Soft Warnings (Valid but Suspicious) . . . 38
0.4.15 Complete Examples . . . 38
0.4.15.1 Example 1: Product Roadmap . . . 38
0.4.15.2 Example 2: Executive Program Timeline . . . 40
0.4.15.3 Example 3: Architecture Evolution Timeline . . . 42
0.4.16 Extensibility . . . 45
0.4.16.1 Metadata Extension . . . 45

0.4.16.2 Custom Categories . . . 45
0.4.16.3 Version Compatibility . . . 45
0.4.17 Serialization and Syntax . . . 46
0.4.17.1 Canonical Format . . . 46
0.4.17.2 Alternative Formats . . . 46
0.4.17.3 Git Friendliness . . . 46
0.4.18 Validation . . . 47
0.4.18.1 Error Reporting . . . 47
0.4.18.2 Strict vs. Lenient Mode . . . 47
0.4.19 Relationship to Other Components . . . 47
0.5 Rendering Model . . . 48
0.5.1 Determinism Contract . . . 48
0.5.2 Coordinate System . . . 49
0.5.3 Timeline Axis . . . 49
0.5.3.1 Axis Unit and Inference . . . 49
0.5.3.2 Date-to-Coordinate Function . . . 50
0.5.3.3 Date Coercion for Bar Edges . . . 50
0.5.3.4 Tick and Gridline Placement . . . 51
0.5.4 The Six-Phase Layout Algorithm . . . 51
0.5.4.1 Phase 1: Axis Computation . . . 52
0.5.4.2 Phase 2: Track Placement . . . 52
0.5.4.3 Phase 3: Activity Geometry . . . 53
0.5.4.4 Phase 4: Milestone Geometry . . . 54
0.5.4.5 Phase 5: Sections and Annotations . . . 55
0.5.4.6 Phase 6: Label Collision Resolution . . . 55
0.5.5 Edge Cases . . . 56
0.5.6 Known IR Gaps . . . 58
0.6 Theme Architecture . . . 58
0.6.1 Theme Design Philosophy . . . 59
0.6.2 Theme Schema . . . 59

0.6.2.1 Canvas and Margins . . . 59
0.6.2.2 Typography . . . 60
0.6.2.3 Axis Styling . . . 60
0.6.2.4 Track Styling . . . 60
0.6.2.5 Activity Styling . . . 61
0.6.2.6 Milestone Styling . . . 61
0.6.2.7 Annotation and Legend Styling . . . 62
0.6.2.8 Status-to-Style Map . . . 62
0.6.2.9 Category-to-Style Map . . . 63
0.6.3 Built-in Themes . . . 63
0.6.3.1 Consulting Theme . . . 63
0.6.3.2 Executive Theme . . . 64
0.6.3.3 Product Theme . . . 65
0.6.3.4 Release Theme . . . 65
0.6.3.5 Minimal Theme . . . 66
0.6.4 Cross-Theme Visual Comparison . . . 66
0.6.5 Theme Inheritance and Extension . . . 66
0.6.6 Theme-Engine Contract (Formal) . . . 68
0.7 Output Targets . . . 68
0.7.1 SVG — Primary Geometry Format . . . 68
0.7.2 PNG — Raster Output . . . 70
0.7.3 PDF — Archival and Print . . . 71
0.7.4 PPTX — Editable Presentation . . . 72
0.7.5 HTML — Interactive Preview . . . 73
0.7.6 Output Priority Recommendation . . . 74
0.7.7 Format Comparison Summary . . . 75
0.8 Distribution Strategy . . . 75
0.8.1 Distribution Requirements . . . 75
0.8.2 Packaging Options . . . 76
0.8.2.1 Core Library . . . 76

0.8.2.2 Command-Line Interface . . . 77
0.8.2.3 VS Code Extension . . . 77
0.8.2.4 MCP Server (Model Context Protocol) . . . 78
0.8.2.5 NPM Package for Embedding . . . 78
0.8.2.6 Standalone Go Binary . . . 79
0.8.3 Implementation Language Trade-offs . . . 79
0.8.4 Recommended Architecture . . . 80
0.8.5 Versioning and Compatibility . . . 81
0.8.6 Distribution Priorities . . . 81
0.9 Agent Integration . . . 82
0.9.1 The Ingestion Contract . . . 82
0.9.1.1 The Prompt as First-Class Input . . . 82
0.9.1.2 What Ingestion May Not Do . . . 82
0.9.2 Ingestion Workflows . . . 83
0.9.2.1 Azure DevOps Work Items . . . 83
0.9.2.2 Natural Language Prose . . . 85
0.9.2.3 GitHub Projects . . . 87
0.9.2.4 Mermaid Timeline Syntax . . . 87
0.9.3 Reliable IR Generation by Agents . . . 89
0.9.3.1 JSON Schema Publication . . . 89
0.9.3.2 Structured Output Constraints . . . 91
0.9.3.3 IR Design Choices That Help Agents . . . 91
0.9.4 Validation Pipeline . . . 92
0.9.4.1 Errors vs. Warnings . . . 93
0.9.5 Error Handling and Agent Repair Cycle . . . 93
0.9.5.1 Error Message Format . . . 93
0.9.5.2 Agent Repair Cycle . . . 94
0.9.6 MCP Server . . . 95
0.9.6.1 Tool Contracts . . . 95
0.9.6.2 MCP Server Deployment . . . 97

0.9.7 Round-Trip and Iterative Editing . . . 98
0.9.7.1 Source Provenance via Metadata . . . 98
0.9.7.2 Re-Sync Strategy . . . 99
0.9.7.3 Stable IDs and Git-Friendly YAML . . . 99
0.9.7.4 Human Edits and Incremental Refinement . . . 100
0.9.8 IR Gaps and Open Questions . . . 100
0.10 Comparison Analysis . . . 101
0.10.1 Framing the Comparison . . . 101
0.10.2 Tool-by-Tool Analysis . . . 101
0.10.2.1 Mermaid (timeline and gantt) . . . 101
0.10.2.2 PlantUML . . . 102
0.10.2.3 vis-timeline . . . 102
0.10.2.4 Frappe Gantt . . . 103
0.10.2.5 Microsoft Project . . . 103
0.10.2.6 think-cell . . . 103
0.10.2.7 PowerPoint Timeline (native SmartArt) . . . 103
0.10.3 Comparison Table . . . 104
0.10.4 Where Timeline Compiler Intentionally Differs . . . 104
0.10.5 Real-Data Crawl: Practical Patterns Observed . . . 105
0.11 MVP Design . . . 106
0.11.1 MVP Philosophy . . . 106
0.11.2 Feature Prioritization . . . 106
0.11.2.1 Must Have (P0) . . . 106
0.11.2.2 Should Have (P1) . . . 107
0.11.2.3 Nice to Have (P2) . . . 107
0.11.2.4 Future (P3) . . . 108
0.11.3 MVP Scope Summary . . . 108
0.11.4 Phased Roadmap . . . 108
0.11.4.1 Phase 1: Core Engine (MVP Launch) . . . 108
0.11.4.2 Phase 2: Polish and Usability . . . 109

0.11.4.3 Phase 3: Ecosystem . . . 109
0.11.4.4 Phase 4: Scale and Community . . . 110
0.11.5 Risks and Mitigations . . . 110
0.11.5.1 Technical Risks . . . 110
0.11.5.2 Adoption Risks . . . 110
0.11.5.3 Complexity Hotspots . . . 111
0.11.6 Smallest Viable Version . . . 111
0.11.7 Success Metrics . . . 111
0.12 Open-Source Strategy . . . 112
0.12.1 Is This a Viable Open-Source Project? . . . 112
0.12.2 Closest Existing Projects and the Gap They Leave . . . 112
0.12.3 Potential Adopters . . . 112
0.12.4 Ecosystem Fit . . . 113
0.12.4.1 Mermaid / Markdown / CI Ecosystem . . . 113
0.12.4.2 MCP / Agent Ecosystem . . . 113
0.12.4.3 PPTX Ecosystem . . . 114
0.12.5 Extension Opportunities . . . 114
0.12.6 Strongest Differentiators . . . 114
0.12.7 Community Contribution Model . . . 115
0.12.8 Adoption Risk Assessment . . . 115
0.13 Timeline Grammar vs. Task Scheduling Grammar . . . 116
0.13.1 The Thesis . . . 116
0.13.2 Two Grammars Compared . . . 117
0.13.2.1 Task Scheduling Grammar . . . 117
0.13.2.2 Timeline Grammar . . . 117
0.13.3 Why Gantt-Thinking Is Wrong Here . . . 118
0.13.3.1 Computational Complexity . . . 118
0.13.3.2 Semantic Overload . . . 118
0.13.3.3 Schema Bloat . . . 119
0.13.3.4 Agent Hostility . . . 119

0.13.3.5 The Gantt Trap 119
0.13.4 Timeline Grammar: Design Implications 120
0.13.4.1 IR Simplicity 120
0.13.4.2 No Computation 120
0.13.4.3 Visual-First Extensions 120
0.13.4.4 Determinism by Default 121
0.13.5 Addressing the Counter-Arguments 121
0.13.5.1 “But I need dependency arrows” 121
0.13.5.2 “What if my dates are inconsistent?” 121
0.13.5.3 “Project management tools already have timeline views” 121
0.13.5.4 “Won’t users expect scheduling features?” 122
0.13.6 Optimization Goals Alignment 122
0.13.7 Conclusion 122

0.1 Problem Definition

0.1.1 The Communication Gap

Executives, product managers, consultants, and program leads communicate plans through visual timelines. These artifacts appear in board presentations, investor decks, quarterly reviews, transformation roadmaps, and architecture evolution documents. The quality of these visuals directly affects stakeholder comprehension and decision-making.

Yet producing presentation-quality timelines remains surprisingly difficult. The gap exists because available tools fall into two categories, each failing half the requirement:

Documentation Tools
(Mermaid, PlantUML, text-based DSLs) — machine-readable, version-controllable, agent-friendly, but produce diagrams suited for technical documentation rather than executive communication.

Design Tools
(PowerPoint, Visio, Lucidchart, think-cell, Figma) — produce polished visuals, but require significant manual effort, resist automation, and cannot be reliably generated by AI agents.

Timeline Compiler bridges this gap: a compiler target for structured plan data that produces presentation-quality visual output.

0.1.2 Why Existing Tools Are Insufficient

0.1.2.1 Mermaid and Documentation-Grade Diagramming

Mermaid [11] has become the de facto standard for diagrams-as-code in technical documentation. Its Gantt syntax illustrates the limitation:

```
gantt
    title Product Roadmap Q3
    dateFormat YYYY-MM-DD
    section Platform
    Core API :a1, 2026-07-01, 30d
    Mobile SDK :a2, after a1, 45d
    section Integrations
    Partner A :b1, 2026-08-01, 60d
```

Listing 1: Mermaid Gantt — functional but not presentation-grade

Mermaid excels at documentation — syntax is simple, output is deterministic, rendering integrates with GitHub, GitLab, and Notion. But the output is not presentation-grade:

- Fixed visual style optimized for legibility in documentation contexts
- No theming beyond basic color overrides
- Limited typographic control

- Dependency-centric (task scheduling) rather than communication-centric (timeline narrative)
- No built-in support for swimlanes as first-class visual groupings
- No annotations, callouts, or emphasis mechanisms for executive communication

The result: Mermaid diagrams are useful in READMEs and wikis but inappropriate for board decks and investor presentations.

0.1.2.2 Project Management Tools

Tools like Microsoft Project, Jira, Asana, Monday.com, and Linear are optimized for *managing work*, not *communicating plans*. Their timeline views:

- Export poorly or not at all to static visual formats
- Embed project-management semantics (resources, dependencies, critical paths) that clutter executive communication
- Assume ongoing interaction rather than snapshot communication
- Produce visuals tied to the tool's aesthetic, not the organization's brand

A quarterly roadmap exported from Jira looks like a Jira export, not like a McKinsey slide.

0.1.2.3 Design and Presentation Tools

PowerPoint, Keynote, Visio, Figma, and Lucidchart produce beautiful results — but at the cost of manual effort. Each timeline is a bespoke artifact:

- No structured data input; changes require manual redrawing
- Not version-controllable in meaningful ways (binary or JSON blobs)
- Not agent-accessible; an AI cannot reliably produce a Visio file
- Theming requires manual application
- Determinism is impossible; two designers will produce different artifacts from the same brief

This creates a maintenance burden. When scope shifts, the timeline must be manually updated. When quarterly plans roll forward, someone rebuilds the slide.

0.1.2.4 AI Agent Blindspot

Modern AI agents (LLMs with tool use, planning agents, copilots) can generate structured plans — sprints, phases, milestones, dependencies. But they have no widely-adopted open-source target for rendering those plans into visual artifacts.

An agent can output JSON or YAML describing a roadmap. But then what? Without a compiler that accepts structured input and produces presentation-grade output, the agent's plan stops at structured text.

0.1.3 Who This Is For

Timeline Compiler serves two complementary audiences:

0.1.3.1 Human Authors

Executives and Chiefs of Staff
— Need polished quarterly plans, transformation roadmaps, and board-ready timelines without designer dependency.

Product Managers
— Need roadmaps that communicate priority and sequencing to stakeholders, not dependency graphs for engineers.

Consultants
— Need branded deliverables (BCG-style roadmaps, McKinsey program timelines) that are data-driven yet visually refined.

DevRel and Technical Writers
— Need version-controlled timelines that embed in documentation but also export to slides.

Program Managers
— Need milestone views and phase summaries that roll up operational detail into executive narrative.

0.1.3.2 AI Agents

AI agents require:

- A well-defined intermediate representation (IR) they can generate
- Deterministic rendering — the same IR always produces the same visual
- Theming separation — agents produce structure, not style
- Validation — agents can verify their output conforms to the schema
- Embeddability — agents can invoke rendering as a tool

Timeline Compiler's IR is designed as a *compiler target*: a stable, documented, validatable representation that agents can emit and humans can inspect, version, and override.

0.1.4 What Success Looks Like

0.1.4.1 Scenario: Product Roadmap

A product manager writes (or an agent generates):

```
timeline:
  title: "Platform Roadmap 2026"
  range: { start: 2026-Q1, end: 2026-Q4 }
  tracks:
    - id: platform
```

```
      label: "Core Platform"
      activities:
        - label: "API v2"
          range: { start: 2026-01, end: 2026-03 }
          status: complete
        - label: "SDK Release"
          range: { start: 2026-04, end: 2026-06 }
          status: in-progress
          progress: 0.6
    - id: mobile
      label: "Mobile"
      activities:
        - label: "iOS App"
          range: { start: 2026-05, end: 2026-08 }
          status: planned
  milestones:
    - label: "Public Beta"
      date: 2026-06-15
      tracks: [platform, mobile]
```

Listing 2: Product Roadmap IR Example

The compiler, given a theme (e.g., `corporate-blue`), produces a PDF suitable for a board presentation: clean typography, proper spacing, consistent color palette, no visual noise.

0.1.4.2 Scenario: Transformation Plan

A management consultant defines a multi-year digital transformation:

```
timeline:
  title: "Digital Transformation — 3 Year Plan"
  range: { start: 2026-01, end: 2028-12 }
  sections:
    - id: foundation
      label: "Foundation"
      range: { start: 2026-01, end: 2026-12 }
    - id: scaling
      label: "Scaling"
      range: { start: 2027-01, end: 2027-12 }
    - id: optimization
      label: "Optimization"
      range: { start: 2028-01, end: 2028-12 }
  tracks:
    - id: technology
      label: "Technology"
      activities:
        - label: "Cloud Migration"
          range: { start: 2026-03, end: 2026-09 }
          section: foundation
        - label: "Platform Modernization"
          range: { start: 2027-01, end: 2027-09 }
          section: scaling
    - id: organization
      label: "Organization"
      activities:
        - label: "Team Restructuring"
          range: { start: 2026-06, end: 2026-12 }
```

```
          section: foundation
  annotations:
    - label: "Board Checkpoint"
      date: 2026-12-15
      style: callout
```

Listing 3: Transformation Timeline IR Example

The output: a three-year view with clear phase demarcation, swimlanes for workstreams, and a visual hierarchy that communicates strategic sequencing rather than tactical dependencies.

0.1.4.3 Success Criteria

Timeline Compiler succeeds when:

1. A human can write the IR by hand in under 10 minutes for a typical roadmap
2. An AI agent can generate valid IR without special prompting beyond the schema
3. The same IR, rendered with the same theme, produces byte-identical output across runs (determinism)
4. A non-designer can produce board-ready visuals by selecting a theme
5. The IR is diff-friendly — changes to the plan produce readable diffs
6. The output is accepted in contexts where PowerPoint slides are currently required

0.1.5 What This Is Not

To prevent scope creep and misunderstanding:

- **Not a project management tool** — does not track work, assign resources, or compute schedules
- **Not a Gantt chart replacement** — though it can render timeline views, it is not optimized for dependency visualization
- **Not a Jira/Asana competitor** — does not replace work-tracking systems; may consume data from them
- **Not a design tool** — does not provide a canvas, drag-and-drop, or WYSIWYG editing
- **Not interactive** — output is static (SVG, PNG, PDF, PPTX); interactivity is out of scope

Timeline Compiler is a *compiler*: structured input in, presentation-quality visual output out. The value is in the deterministic, themeable, agent-accessible transformation — not in the tooling around authoring or managing plans.

0.2 Design Principles

The following principles govern Timeline Compiler's architecture. They are listed in priority order; when principles conflict, earlier principles take precedence.

0.2.1 Presentation-First

Principle: Every design decision optimizes for presentation-quality visual output.

Rationale: The distinguishing feature of Timeline Compiler is producing visuals suitable for board presentations, investor decks, and executive communication. This is not a documentation tool (Mermaid serves that purpose) nor a project management tool (many exist). If a feature does not improve presentation quality, it should not exist.

Implications:

- Typography, spacing, and color receive first-class treatment in the theme engine
- The IR includes presentation hints (emphasis, annotations, callouts) that have no project-management semantics
- Output formats prioritize fidelity (PDF, SVG) over convenience

0.2.2 Deterministic Output

Principle: The same IR plus the same theme produces byte-identical output across all runs.

Rationale: Determinism enables:

- Version control — changes to the IR produce predictable visual diffs
- CI/CD integration — automated rendering in pipelines
- Agent reliability — agents can trust their output
- Reproducibility — artifacts can be regenerated exactly

Implications:

- No random layout algorithms; all positioning is computed deterministically
- Timestamps, UUIDs, and other entropy sources are excluded from output
- Font metrics must be stable (system fonts are problematic; embedded or specified fonts are required)
- Floating-point operations must be reproducible (or avoided)

0.2.3 Human-Readable Source

Principle: The IR must be readable and writable by humans without tooling.

Rationale: The IR serves dual audiences (humans and agents). Human readability ensures:

- Onboarding friction is low — a PM can write a roadmap without learning a complex DSL
- Debugging is possible — when output is wrong, the IR is inspectable
- Manual overrides are feasible — humans can adjust agent-generated IR
- Version control diffs are meaningful

Implications:

- YAML or YAML-like syntax preferred over JSON (less visual noise)
- Field names are descriptive English, not abbreviations
- Nesting depth is minimized
- Dates use ISO 8601; durations use human-readable formats where possible

0.2.4 Agent-Friendly Authoring

Principle: AI agents must be able to generate valid IR without special prompting or examples.

Rationale: A primary use case is agent-generated timelines. If agents struggle to produce valid IR, the system fails its purpose.

Implications:

- The IR has a formal schema (JSON Schema or equivalent) that agents can reference
- Field names and structure align with how LLMs naturally express plans
- The IR avoids footguns — common mistakes should be invalid or auto-corrected
- Validation provides clear, actionable error messages
- The schema is small enough to fit in a context window

0.2.5 Theme-Driven Rendering

Principle: Visual styling is entirely separated from content; all presentation decisions flow from the theme.

Rationale: Separation of content and presentation enables:

- Rebranding without changing content — swap themes, regenerate
- Consistent organizational aesthetics across timelines
- Agents authoring content without making design decisions
- Themes as a distribution mechanism (theme marketplaces, branded themes)

Implications:

- The IR contains no colors, fonts, or sizes — only semantic hints
- Themes define all visual properties
- The rendering model maps semantic IR elements to themed visual primitives
- Inline style overrides, if permitted, are explicit escapes from theme control

0.2.6 Separation of Planning and Rendering

Principle: The IR represents *what to communicate*, not *how to compute it*.

Rationale: Timeline Compiler is a rendering compiler, not a scheduling engine. The IR is a communication artifact, not a project plan. This separation:

- Keeps the IR minimal and focused
- Avoids reimplementing project-management semantics
- Allows integration with any upstream planning tool
- Clarifies scope boundaries

Implications:

- No dependency resolution, critical path calculation, or resource leveling
- Dates are provided, not computed
- Progress is asserted, not measured
- The IR describes the *output* of planning, not the *inputs* to planning

0.2.7 Minimal Visual Clutter

Principle: Default output favors clarity over information density.

Rationale: Presentation-quality visuals communicate one message clearly rather than many messages densely. Executive audiences prefer clean visuals over comprehensive ones.

Implications:

- Default themes use generous whitespace
- Labels are concise; detail lives in supporting materials
- Gridlines, axes, and chrome are minimized by default
- Information hierarchy is clear — not all elements receive equal visual weight

0.2.8 Source-Control Friendly

Principle: The IR is designed for version control workflows.

Rationale: Modern teams version everything. If the IR produces noisy diffs or merge conflicts, adoption suffers.

Implications:

- Text-based format (YAML preferred)
- Stable field ordering (or sorted keys)
- No generated IDs unless explicitly authored
- Changes to one element do not cascade spuriously to others
- Line-oriented structure where possible (each item on its own line)

0.2.9 Composability

Principle: Timeline definitions can reference, include, and compose other definitions.

Rationale: Real-world use involves:

- Shared track definitions across timelines
- Theme inheritance and overrides
- Milestone libraries
- Multi-file organization for large timelines

Implications:

- Support for file includes or references
- Namespacing to avoid collisions
- Clear resolution rules for overrides
- Themes can extend other themes

0.2.10 Graceful Degradation

Principle: Invalid or incomplete IR produces useful output or clear errors, not silent failures.

Rationale: Authors (especially agents) make mistakes. The system should:

- Validate early and report clearly
- Render partial output when possible (with warnings)
- Never produce corrupt or misleading visuals silently

Implications:

- Schema validation runs before rendering
- Semantic validation catches contradictions (e.g., end before start)
- Warnings are visible but do not block output when the issue is cosmetic
- Errors specify exactly what is wrong and where

0.2.11 Extensibility Without Complexity

Principle: The IR and theme systems support extension without requiring core changes.

Rationale: Long-term success requires community contribution — new themes, new semantic elements, integration with new upstream tools. But extension points must not compromise core simplicity.

Implications:

- Unknown IR fields are ignored (forward compatibility)
- Themes support custom properties via namespacing
- Plugin or extension architecture for renderers (future)
- Versioned schemas with clear migration paths

0.2.12 Principle Summary

Table 1: Design Principles Summary

Principle	Core Commitment
Presentation-First	Optimize for board-ready visuals
Deterministic Output	Same input, same output, always
Human-Readable Source	Writable without tooling
Agent-Friendly Authoring	Agents generate valid IR naturally
Theme-Driven Rendering	Content and style fully separated
Separation of Planning and Rendering	Describe output, not compute schedule
Minimal Visual Clutter	Clarity over density
Source-Control Friendly	Clean diffs, easy merges
Composability	Reference, include, extend
Graceful Degradation	Clear errors, useful partial output
Extensibility Without Complexity	Extend without core changes

0.3 Scope Boundaries

This section defines explicit boundaries for Timeline Compiler. Clarity here prevents scope creep and ensures the system remains focused on its core value proposition: transforming structured plan descriptions into presentation-quality visual timelines.

0.3.1 Governing Principle

The scope boundary is defined by the *rendering abstraction*: Timeline Compiler transforms a communication-oriented intermediate representation into visual output. It does not generate, compute, or manage the plan data itself.

> *If a feature requires understanding the semantics of work — dependencies, resources, costs, velocity — it is out of scope. If a feature improves the visual communication of a plan already decided, it is potentially in scope.*

0.3.2 In Scope

The following capabilities are within Timeline Compiler's scope:

0.3.2.1 Visual Artifact Types

Timelines
Horizontal time-based visualizations showing activities, milestones, and phases across a date range.

Roadmaps
Product or program roadmaps with tracks (swimlanes), grouped activities, and strategic milestones.

Milestone Views
Focused views highlighting key dates and deliverables without detailed activity bars.

Phase Summaries
High-level views showing phases or stages of a program without granular activities.

Swimlane Diagrams
Multi-track views where parallel workstreams are visually separated.

0.3.2.2 IR Elements

The IR supports the following semantic elements (detailed in Section 0.4):

`tracks`
Named swimlanes or rows that group related activities. Tracks provide visual and semantic organization.

`groups`
Logical groupings within or across tracks for nested organization (e.g., workstreams within a track).

`activities`
Time-bounded items (bars) representing work, initiatives, or phases. Activities have start/end dates, labels, and optional metadata.

`milestones`
Point-in-time markers representing key dates, deliverables, or decisions.

`sections`
Temporal divisions of the timeline (e.g., quarters, phases) that span all tracks.

`range`
The overall time window for the visualization.

`date ranges`
Start and end dates for activities and sections. Supports dates, months, quarters, and years.

`progress`
Numeric (0.0–1.0) indication of completion for activities. Purely visual — not computed.

`status`
Semantic status indicators (e.g., `planned`, `in-progress`, `complete`, `at-risk`, `blocked`).

`labels`
Text labels for all elements. Brief, presentation-appropriate text.

`annotations`
Callouts, notes, or emphasis markers attached to dates or elements.

`legends`
Explanatory legends for status colors, track semantics, or custom indicators.

0.3.2.3 Presentation Features

Theming
Complete visual customization via theme files: colors, typography, spacing, shapes.

Emphasis
Visual mechanisms to highlight specific activities or milestones (e.g., bold, glow, callout).

Today Markers
Visual indicator for the current date (provided as input, not computed).

Progress Visualization
Visual representation of progress within activity bars.

Status Indicators
Color or icon mapping for semantic statuses.

Annotations
Callouts, arrows, or labels that highlight specific points.

0.3.2.4 Output Formats

SVG
Primary vector output for web embedding and further processing.

PNG
Rasterized output for contexts requiring bitmap images.

PDF
Print-quality vector output for documents and decks.

PPTX
PowerPoint-compatible output for direct use in presentations.

0.3.2.5 Tooling

CLI
Command-line interface for rendering (`timeline render input.yaml -o output.pdf`).

Library API
Programmatic access for embedding in other tools.

Schema Validation
Validation of IR against formal schema.

Theme Validation
Validation of theme files.

0.3.3 Out of Scope

The following capabilities are explicitly excluded:

0.3.3.1 Project Management Semantics

Dependency Scheduling
Timeline Compiler does not compute dates from dependencies. If Activity B depends on Activity A, the author must provide both dates explicitly. The IR may optionally *record* dependencies for visual rendering (e.g., arrows), but it does not *resolve* them.
Rationale: Dependency resolution is core project-management logic. It requires understanding task semantics, durations, constraints, and often iterative solving. This is the domain of MS Project, Primavera, and similar tools. Timeline Compiler consumes the *output* of scheduling, not the inputs.

Resource Management
No support for assigning people, teams, or resource pools to activities. No resource leveling or capacity planning.
Rationale: Resource management requires organizational context (who is available, at what capacity, with what skills) that is outside the scope of a rendering compiler.

Critical Path Analysis
No calculation of critical paths or slack time.
Rationale: Critical path analysis is a scheduling concern, not a rendering concern. Upstream tools compute this; Timeline Compiler may visualize the result if provided.

Sprint/Iteration Tracking
No support for agile sprints, velocity tracking, burndowns, or iteration planning.
Rationale: Sprint tracking is operational work management. Timeline Compiler visualizes strategic communication, not operational tracking.

Cost Management
No budgets, cost tracking, earned value, or financial projections.
Rationale: Financial data requires integration with accounting and project-control systems. This is far outside the rendering scope.

Portfolio Optimization
No multi-project prioritization, resource allocation across projects, or strategic portfolio balancing.
Rationale: Portfolio management is a strategic planning function, not a visualization function.

Baseline Comparison
No tracking of baseline vs. actual schedules over time.

Rationale: Baseline tracking requires temporal versioning and comparison logic. This is project-control functionality.

Risk Registers
No formal risk tracking, probability/impact matrices, or mitigation planning.
Rationale: Risk management is a project-management discipline with its own tooling requirements.

0.3.3.2 Data Management

Persistent Storage
Timeline Compiler does not store timelines. It transforms input files to output files. Storage is the user's responsibility.

Collaboration Features
No real-time collaboration, comments, change tracking, or multi-user editing.

Version History
No built-in versioning. Users should use Git or similar tools.

0.3.3.3 Interactive Features

Clickable Outputs
Output formats (SVG, PNG, PDF, PPTX) are static. Hyperlinking or interactive drill-down is out of scope for MVP, though SVG hyperlinks may be a future extension.

WYSIWYG Editing
No drag-and-drop authoring. Timeline Compiler is a compiler, not an editor.

Live Preview During Authoring
Out of scope for core; a VS Code extension may provide this separately.

0.3.3.4 Data Ingestion

Native Integrations
Timeline Compiler does not natively read from Jira, Azure DevOps, Asana, GitHub Projects, or other systems. It consumes only its IR format.
Note: Separate adapter tools or scripts may transform external data to the IR, but these are outside Timeline Compiler's core scope.

0.3.4 Boundary Cases

Some features lie at the boundary and require explicit decisions:

0.3.4.1 Dependency Arrows (Visual Only)

Decision: IN SCOPE — visual only.

The IR may include dependency declarations for the purpose of rendering arrows or lines between activities. However, dependencies are purely visual annotations; they do not affect date computation or validation.

```
activities:
  - id: design
    label: "Design Phase"
    range: { start: 2026-01, end: 2026-03 }
  - id: build
    label: "Build Phase"
    range: { start: 2026-04, end: 2026-08 }
    depends_on: [design]  # Visual arrow only
```

Listing 4: Dependencies as visual annotations

0.3.4.2 Today Line

Decision: IN SCOPE — provided as input.

The "today" date may be provided in the IR (e.g., `today: 2026-06-09`) and rendered as a vertical marker. The compiler does not infer today's date from the system clock; determinism requires all inputs to be explicit.

0.3.4.3 Auto-Layout

Decision: IN SCOPE — deterministic algorithms only.

The compiler automatically lays out activities within tracks (e.g., stacking overlapping items). Layout algorithms must be deterministic. No randomized or heuristic placement that could vary between runs.

0.3.4.4 External Image Embedding

Decision: DEFERRED to future.

Embedding external images (logos, icons) in output is not in MVP scope. Theme-provided icons (e.g., milestone diamonds) are in scope.

0.3.5 Scope Summary

0.4 Timeline IR

The Timeline Intermediate Representation (IR) is the central contract of this system. It defines *what* a timeline communicates—entities, relationships, temporal structure, and semantic status—while remaining agnostic to *how* that communication is rendered. This separation enables deterministic rendering, source-control-friendly editing, and reliable generation by both humans and AI agents.

0.4.1 Design Principles

1. **Make illegal states unrepresentable.** The IR's type system should prevent malformed timelines at the structural level, not merely validate them after the fact.

Table 2: Scope Boundaries Summary

Category	Scope Status
Timelines, roadmaps, milestone views	In Scope
Tracks, activities, milestones, sections, annotations	In Scope
Progress and status visualization	In Scope
Theming and visual customization	In Scope
SVG, PNG, PDF, PPTX output	In Scope
CLI and library API	In Scope
Dependency arrows (visual only)	In Scope
Today marker (explicit input)	In Scope
Dependency scheduling/resolution	Out of Scope
Resource management	Out of Scope
Critical path analysis	Out of Scope
Sprint/iteration tracking	Out of Scope
Cost management	Out of Scope
Portfolio optimization	Out of Scope
Baseline comparison	Out of Scope
Risk registers	Out of Scope
Data storage and collaboration	Out of Scope
Interactive/clickable output	Out of Scope
WYSIWYG editing	Out of Scope
Native data source integrations	Out of Scope

2. **Rendering-agnostic.** The IR describes meaning, not pixels. Visual decisions—colors, fonts, positioning—belong to the theme engine.
3. **Git-friendly.** Stable field ordering, line-oriented YAML syntax, deterministic serialization, minimal diff churn.
4. **Agent-generatable.** Clear required-vs-optional distinctions, explicit defaults, forgiving structure that remains validatable.
5. **Orthogonal core.** A small set of primitives; convenience features are optional layers, not core complexity.

0.4.2 Type Vocabulary

The IR uses the following type vocabulary consistently throughout this specification:

0.4.3 Document Structure

A Timeline IR document is a single YAML file with the following root structure:

```
version: "1.0"
metadata: { ... }
tracks: [ ... ]
groups: [ ... ]       # optional
activities: [ ... ]
milestones: [ ... ]   # optional
annotations: [ ... ]  # optional
```

Type	Description
`string`	UTF-8 text, non-empty unless explicitly noted
`string?`	Optional string (may be omitted or null)
`int`	Integer
`number`	Decimal number
`number[0..1]`	Decimal in closed interval $[0, 1]$
`bool`	Boolean (`true`/`false`)
`date`	A temporal point (see §0.4.5)
`daterange`	A temporal interval with start and optional end
`id`	Document-unique identifier (kebab-case slug, e.g., `alpha-release`)
`ref<T>`	Reference to an entity of type `T` by its `id`
`list<T>`	Ordered list of elements of type `T`
`optional<T>`	Value of type `T` that may be omitted
`enum{...}`	One of the listed literal values
`map<K,V>`	Key-value mapping

Table 3: IR type vocabulary

```
sections: [ ... ]      # optional
legend: { ... }        # optional
```

Listing 5: Root document structure

0.4.3.1 Root Fields

Field	Type	Req	Default	Description
`version`	`string`	Yes	—	Specification version (semver, e.g., `"1.0"`)
`metadata`	`Metadata`	Yes	—	Document-level metadata
`tracks`	`list<Track>`	Yes	—	Swimlanes/rows; at least one required
`groups`	`list<Group>`	No	`[]`	Logical groupings of activities
`activities`	`list<Activity>`	Yes	—	Time-spanning items; may be empty
`milestones`	`list<Milestone>`	No	`[]`	Point-in-time markers
`annotations`	`list<Annotation>`	No	`[]`	Callouts, notes, highlights
`sections`	`list<Section>`	No	`[]`	Visual/logical document sections
`legend`	`Legend`	No	auto	Legend mapping; auto-generated if omitted

Table 4: Root document fields

Invariant: The document must contain at least one track. Activities and milestones may both be empty, but a timeline with neither is semantically vacuous (valid but degenerate).

0.4.4 Metadata

The `metadata` object contains document-level information and temporal framing.

Field	Type	Req	Default	Description
`title`	`string`	Yes	—	Primary document title
`subtitle`	`string?`	No	`null`	Secondary title or tagline
`author`	`string?`	No	`null`	Author or team name
`created`	`date`	No	now	Document creation date
`updated`	`date`	No	now	Last modification date
`time_range`	`TimeRange`	Yes	—	Temporal bounds of the timeline
`axis_unit`	`AxisUnit`	No	inferred	Granularity of the time axis
`theme`	`string?`	No	`null`	Theme identifier (resolved by renderer)
`locale`	`string?`	No	`"en"`	BCP-47 locale for date formatting
`today`	`date?`	No	eval time	Reference date for `now`, relative dates, and today-marker; see §0.4.4.3
`fiscal_year_start`	`int [1..12]`	No	`1`	Calendar month on which the fiscal year begins (1 = January); see §0.4.4.3
`description`	`string?`	No	`null`	Extended description or notes

Table 5: Metadata fields

0.4.4.1 TimeRange

The `time_range` defines the temporal viewport of the timeline—the span that will be rendered. Activities or milestones outside this range are valid but may be clipped.

```
time_range:
  start: 2026-Q1
  end: 2026-Q4
```

Field	Type	Req	Default	Description
`start`	`date`	Yes	—	Inclusive start of the viewport
`end`	`date`	No	open	Inclusive end; omit for open-ended

Table 6: TimeRange fields

0.4.4.2 AxisUnit

```
enum AxisUnit { day, week, month, quarter, half, year }
```

If omitted, the renderer infers axis unit from the time range span. The axis unit affects visual tick marks and labels, not the precision of individual dates.

0.4.4.3 Date Anchor and Fiscal Calendar

Date anchor (`today`). The symbolic date `now` and all relative date offsets (`+3m`, `-2w`, etc.) resolve against the *date anchor*, evaluated in this order:

1. `metadata.today` — if present, this value is used exclusively.
2. `metadata.created` — fallback when `today` is absent.
3. **Hard error** — if neither is present and any `now` or relative date appears in the document, the document is not deterministically evaluable and validation must reject it.

Renderers **must not** consult the system clock to resolve `now` or relative dates. The today-marker annotation (`type: today-marker`) also uses this anchor.

Determinism caveat: When `today` is omitted and `created` serves as the anchor, output is deterministic (the `created` date is fixed in the document). However, the document's visual "current position" does not advance with real time. Authors and agents **should** set `today` explicitly whenever `now` or relative dates appear, so that the document's meaning is unambiguous and byte-stable across environments and rendering runs.

Fiscal calendar (`fiscal_year_start`). `fiscal_year_start` specifies the calendar month (1–12) on which the fiscal year begins. The default is 1 (January), which makes fiscal and calendar years identical.

Fiscal quarter dates (`FYnn-Qk`) map to calendar dates as follows. `FY`nn denotes the fiscal year beginning on month `fiscal_year_start` of calendar year $20nn$. Fiscal quarter k ($k \in \{1, 2, 3, 4\}$) spans three consecutive calendar months starting at calendar month m_k, where:

$$m_k = ((\texttt{fiscal\_year\_start} - 1) + (k - 1) \times 3) \bmod 12 + 1$$

and the calendar year is $20nn$ if $m_k \geq$ `fiscal_year_start`, otherwise $20nn + 1$.

Examples with `fiscal_year_start: 1` (default): FY26-Q1 = Jan–Mar 2026; FY26-Q2 = Apr–Jun 2026 (identical to calendar quarters).

Examples with `fiscal_year_start: 4` (April fiscal year): FY26-Q1 = Apr–Jun 2026; FY26-Q2 = Jul–Sep 2026; FY26-Q3 = Oct–Dec 2026; FY26-Q4 = Jan–Mar 2027.

If any `FY...` date appears in the document and `fiscal_year_start` $\neq 1$, authors **should** set `fiscal_year_start` explicitly to ensure all renderers produce consistent x-coordinates for fiscal dates.

0.4.5 Date Model

Timelines present temporal information at varying granularities. A product roadmap may show quarters; a conference schedule shows hours. The IR must represent this heterogeneity without forcing false precision or losing meaningful vagueness.

0.4.5.1 Date Representations

A `date` in the IR is one of the following forms:

Design rationale: We deliberately support coarse granularity (quarters, halves, years) as first-class concepts rather than forcing conversion to day-level dates. This preserves authorial intent: "Q3 2026" means the entire quarter, not "2026-07-01".

Form	Example	Semantics
ISO date	`2026-06-09`	Specific calendar day
ISO datetime	`2026-06-09T14:00`	Specific moment (minute precision)
Year-month	`2026-06`	Entire month (June 2026)
Year only	`2026`	Entire year
Quarter	`2026-Q2`	Calendar quarter (Apr–Jun)
Half	`2026-H1`	Calendar half (Jan–Jun)
Fiscal quarter	`FY26-Q2`	Fiscal quarter (calendar mapping per `fiscal_year_start`; see §0.4.4.3)
Relative	`+3m`	3 months from the date anchor (`metadata.today` → `metadata.created`; see §0.4.4.3)
Symbolic	`now`	Resolves to the date anchor (`metadata.today` → `metadata.created`; see §0.4.4.3)

Table 7: Date representation forms

0.4.5.2 Uncertain and Open Dates

For dates that are genuinely unknown, tentative, or open-ended, the IR provides explicit encodings rather than using null-as-magic:

Value	Semantics
`tbd`	Date is not yet determined; renders as "TBD"
`ongoing`	No planned end; activity continues indefinitely
`unknown`	Date exists but is not known to the author
`~2026-Q3`	Approximate/tentative (prefix tilde)

Table 8: Uncertain and open date markers

Invariant: `tbd`, `ongoing`, and `unknown` are valid only in positions where the schema allows uncertainty (e.g., `end` of a daterange, not `start` of the document's `time_range`).

0.4.5.3 DateRange

A `daterange` represents a temporal interval:

```
# Explicit start and end
start: 2026-04-01
end: 2026-06-30

# Open-ended (no planned end)
start: 2026-04-01
end: ongoing

# Shorthand for single-granularity span
span: 2026-Q2    # equivalent to start: 2026-04-01, end: 2026-06-30
```

Invariant: When both `start` and `end` are concrete dates, `end >= start`. The `span` shorthand expands to the natural bounds of its granularity (e.g., `span: 2026-Q2` expands to Q2's first and last days).

Field	Type	Req	Default	Description
`start`	`date`	Yes*	—	Interval start (inclusive)
`end`	`date|ongoing|tbd`	No	`ongoing`	Interval end (inclusive); omitting is equivalent to `end: ongoing`
`span`	`date`	Yes*	—	Single-unit shorthand; mutually exclusive with `start`/`end`

Table 9: DateRange fields (*exactly one of `span` or `start` (+ optional `end`) is required; co-presence of `span` with `start` or `end` is a well-formedness error)

Invariant (exclusivity): `span` is *mutually exclusive* with `start` and `end`. A daterange must use exactly one of:

- `span` alone (shorthand form), or
- `start` alone or `start` + `end` (explicit form).

Co-presence of `span` with `start` or `end` on the same entity is a well-formedness error (`SPAN_START_CONFLICT`).

Open/ongoing semantics: An Activity or daterange with `start` specified but `end` omitted is an *open/ongoing interval*. This is semantically identical to writing `end: ongoing` and is an explicit, valid state (not an authoring error). Renderers must extend the bar to the canvas right edge and apply an open-end indicator (e.g., right-pointing chevron).

0.4.6 Tracks (Swimlanes)

Tracks are horizontal lanes that organize activities visually. Every activity belongs to exactly one track. Tracks appear in document order (first track at top).

```
tracks:
  - id: platform
    label: Platform Team
    description: Core infrastructure work

  - id: mobile
    label: Mobile Apps
    color: blue          # optional hint
```

Listing 6: Track examples

Invariant: Track `id` values must be unique within the document.

0.4.7 Groups

Groups provide logical clustering of activities, tracks, or other groups. They enable hierarchical organization (e.g., “Engineering” containing “Platform” and “Mobile” tracks) without conflating logical grouping with visual swimlanes.

Field	Type	Req	Default	Description
`id`	`id`	Yes	—	Unique track identifier
`label`	`string`	Yes	—	Display label
`description`	`string?`	No	`null`	Extended description
`color`	`string?`	No	theme	Color hint (theme may override)
`group`	`ref<Group>?`	No	`null`	Parent group (for nested tracks)
`index`	`int?`	No	position	Explicit sort index; unique non-negative integer; tracks render top-to-bottom in ascending `index` order
`collapsed`	`bool`	No	`false`	Initial collapsed state (if renderer supports)

Table 10: Track fields

```
groups:
  - id: engineering
    label: Engineering
    children:
      - platform    # ref to track
      - mobile      # ref to track

  - id: releases
    label: Release Milestones
    members:
      - alpha-release   # ref to milestone
      - beta-release
```

Listing 7: Group example

Field	Type	Req	Default	Description
`id`	`id`	Yes	—	Unique group identifier
`label`	`string`	Yes	—	Display label
`description`	`string?`	No	`null`	Extended description
`parent`	`ref<Group>?`	No	`null`	Parent group (for nesting)
`children`	`list<ref<Track|Group»`	No	`[]`	Ordered child tracks/groups
`members`	`list<ref<Activity|Milestone»`	No	`[]`	Member entities
`color`	`string?`	No	theme	Color hint

Table 11: Group fields

Invariant: Group hierarchies must be acyclic. A group cannot be its own ancestor.

0.4.8 Activities

Activities are time-spanning items—phases, projects, tasks, or efforts. They are the primary content of most timelines.

```
activities:
  - id: api-redesign
    label: API Redesign
```

```
    track: platform
    start: 2026-Q1
    end: 2026-Q2
    status: in-progress
    progress: 0.4

  - id: mobile-launch
    label: Mobile App Launch
    track: mobile
    span: 2026-Q3
    status: planned
    category: launch
```

Listing 8: Activity examples

Field	Type	Req	Default	Description
id	id	Yes	—	Unique activity identifier
label	string	Yes	—	Display label (shown on bar)
track	ref<Track>	Yes	—	Containing track
start	date	Yes*	—	Start date
end	date|ongoing|tbd	No	ongoing	End date; omitting is equivalent to end: ongoing (open interval)
span	date	Yes*	—	Single-unit shorthand; mutually exclusive with start/end
status	Status	No	planned	Visual communication status
progress	number[0..1]?	No	null	Completion fraction
category	string?	No	null	Semantic category (for styling)
description	string?	No	null	Extended description
group	ref<Group>?	No	null	Logical group membership
url	string?	No	null	External link
tags	list<string>	No	[]	Freeform tags
metadata	map<string,any>	No	{}	Extension metadata

Table 12: Activity fields (*exactly one of span or start (+ optional end) is required; co-presence of span with start or end is a well-formedness error)

0.4.8.1 Status Enum

The status field communicates *visual* state for presentation purposes. This is explicitly **not** a project-management workflow state; it describes what the audience should understand about the item's current condition.

```
enum Status {
  planned,      # Scheduled but not yet started
  in-progress,  # Currently active
  done,         # Completed
  at-risk,      # May miss target (visual warning)
  blocked,      # Cannot proceed (visual alert)
  cancelled,    # No longer planned
  tentative     # Not yet confirmed
}
```

Design rationale: We include `at-risk` and `blocked` because executive timelines frequently need to communicate risk visually. We omit workflow states like "in review" or "approved" which belong to project-management tools, not visual communication artifacts.

0.4.8.2 Progress

The `progress` field is a number in $[0, 1]$ representing completion fraction. It has no inherent visual meaning—the theme decides whether to render it as a progress bar, percentage label, or ignore it entirely.

Invariant: If `progress` is provided, it must be in $[0, 1]$. `progress: 1.0` does not imply `status: done`; these are orthogonal.

0.4.9 Milestones

Milestones are point-in-time markers—releases, deadlines, decision points, or events. Unlike activities, they have no duration.

```
milestones:
  - id: alpha-release
    label: Alpha Release
    date: 2026-03-15
    track: platform
    status: done
    icon: flag

  - id: board-review
    label: Board Review
    date: 2026-Q2        # first day of Q2
    category: governance
```

Listing 9: Milestone examples

Field	Type	Req	Default	Description
`id`	`id`	Yes	—	Unique milestone identifier
`label`	`string`	Yes	—	Display label
`date`	`date`	Yes	—	Point in time
`track`	`ref<Track>?`	No	global	Associated track (or global)
`status`	`Status`	No	`planned`	Visual status
`category`	`string?`	No	`null`	Semantic category
`icon`	`string?`	No	theme	Icon hint (diamond, flag, star, etc.)
`description`	`string?`	No	`null`	Extended description
`group`	`ref<Group>?`	No	`null`	Logical group membership
`url`	`string?`	No	`null`	External link
`tags`	`list<string>`	No	`[]`	Freeform tags

Table 13: Milestone fields

Note: When `track` is omitted, the milestone is "global" and may be rendered spanning all tracks (e.g., a vertical line with label at top).

0.4.10 Annotations

Annotations add contextual information—callouts, notes, highlights, or today-markers—without being first-class timeline entities.

```
annotations:
  - type: today-marker
    date: now

  - type: callout
    target: api-redesign
    text: "Critical path item"
    position: above

  - type: bracket
    targets: [alpha-release, beta-release]
    label: "Testing Phase"

  - type: period
    start: 2026-06-01
    end: 2026-06-15
    label: "Code Freeze"
    style: highlight
```

Listing 10: Annotation examples

Field	Type	Req	Default	Description
type	AnnotationType	Yes	—	Annotation kind
id	id?	No	auto	Optional identifier
target	ref<Activity|Milestone>?	Cond	—	Single target entity
targets	list<ref<...»?	Cond	—	Multiple targets
date	date?	Cond	—	Point annotation
start/end	date?	Cond	—	Range annotation
text/label	string?	No	null	Display text
position	enum{above,below,left,right}?	No	auto	Hint only
style	string?	No	theme	Style hint

Table 14: Annotation fields (conditional fields depend on type)

0.4.10.1 Annotation Types

```
enum AnnotationType {
  today-marker,  # Vertical line at current date
  callout,       # Note attached to an entity
  bracket,       # Visual bracket spanning entities
  period,        # Highlighted time period
  note,          # Freestanding note
  connector      # Line connecting two entities
}
```

0.4.11 Sections

Sections divide the timeline into logical or visual chapters. They enable multi-page timelines or distinct phases that should be visually separated.

```
sections:
  - id: current-state
    label: "Current State (Q1-Q2)"
    time_range:
      start: 2026-Q1
      end: 2026-Q2

  - id: future-state
    label: "Future Roadmap (Q3-Q4)"
    time_range:
      start: 2026-Q3
      end: 2026-Q4
```

Listing 11: Section example

Field	Type	Req	Default	Description
`id`	`id`	Yes	—	Unique section identifier
`label`	`string`	Yes	—	Section title
`time_range`	`TimeRange?`	No	inherit	Override document time range
`tracks`	`list<ref<Track»?`	No	all	Subset of tracks to show
`description`	`string?`	No	`null`	Section description
`page_break`	`bool`	No	`false`	Hint for paginated output

Table 15: Section fields

0.4.12 Legend

The legend documents the meaning of statuses, categories, and visual conventions used in the timeline. If omitted, the renderer may auto-generate a legend from used values.

```
legend:
  show: true
  position: bottom-right
  entries:
    - key: in-progress
      label: In Progress
      description: Currently being worked on
    - key: at-risk
      label: At Risk
      description: May miss planned date
    - key: launch
      label: Launch Event
      category: true
```

Listing 12: Legend example

0.4.13 ID and Reference System

0.4.13.1 Identifier Format

All `id` fields must be:

Field	Type	Req	Default	Description
show	bool	No	true	Whether to render legend
position	string?	No	theme	Position hint
title	string?	No	"Legend"	Legend title
entries	list<LegendEntry>	No	auto	Legend entries

Table 16: Legend fields

Field	Type	Req	Default	Description
key	string	Yes	—	Status or category key
label	string	Yes	—	Human-readable label
description	string?	No	null	Extended description
category	bool	No	false	True if this is a category (not status)

Table 17: LegendEntry fields

- Non-empty strings
- Composed of lowercase letters, digits, and hyphens
- Starting with a letter
- Unique within their entity type AND globally within the document

Regex: `^[a-z][a-z0-9-]*$`

Rationale for global uniqueness: References may appear in contexts where the entity type is ambiguous (e.g., annotation targets). Global uniqueness eliminates the need for type prefixes in references.

```
# Valid IDs
id: api-v2
id: q3-release
id: mobile-team-ios

# Invalid IDs
id: API-v2          # uppercase
id: 2026-release    # starts with digit
id: api_v2          # underscore
```

Listing 13: ID examples

0.4.13.2 References

A reference is a string containing the `id` of another entity. The IR uses bare strings for references (not structured objects) for YAML terseness:

```
activities:
  - id: feature-a
    track: platform      # ref<Track>
    group: engineering   # ref<Group>

annotations:
  - type: callout
    target: feature-a    # ref<Activity>
```

Invariant: Every reference must resolve to exactly one entity in the document. A validator must reject documents with dangling references.

0.4.13.3 Reference Namespacing

Because IDs are globally unique, references are unambiguous without type prefixes. The schema context determines what types are valid:

- `activity.track`: must reference a Track
- `annotation.target`: may reference Activity or Milestone
- `group.children`: may reference Track or Group

A validator checks that the referenced entity has the correct type for its context.

0.4.14 Well-Formedness Rules

A Timeline IR document is **well-formed** if and only if all the following invariants hold:

0.4.14.1 Structural Invariants

1. **Version present:** `version` field exists and matches a known specification version.
2. **Required fields:** All fields marked "Req: Yes" are present and non-null.
3. **At least one track:** The `tracks` list is non-empty.
4. **Unique IDs:** All `id` values are globally unique within the document.
5. **Valid ID format:** All `id` values match `^[a-z][a-z0-9-]*$`.

0.4.14.2 Referential Invariants

6. **References resolve:** Every reference points to an existing entity.
7. **Reference types match:** References point to entities of the expected type (e.g., `activity.track` references a Track, not a Group).
8. **No circular groups:** The group parent-child graph is acyclic.

0.4.14.3 Temporal Invariants

9. **Valid time range:** If `metadata.time_range` has both start and end, then `end >= start`.
10. **Activity duration non-negative:** For activities with concrete start and end, `end >= start`.
11. **Date format valid:** All date values conform to the date model (§0.4.5).
12. **Activity date-source exclusive:** Every activity must satisfy exactly one of: (a) `span` is present and neither `start` nor `end` is present; or (b) `start` is present and `span` is absent (`end` optional). Co-presence of `span` with `start` or `end` is a hard well-formedness error: `SPAN_START_CONFLICT`.

13. **Date anchor present when required:** If any date field in the document contains `now` or a relative offset (e.g., `+3m`, `-2w`), then at least one of `metadata.today` or `metadata.created` must be present. Absence of both is a hard error: `DATE_ANCHOR_MISSING`.
14. **Track index values unique:** When `track.index` is present, all supplied values must be unique non-negative integers within the document.

0.4.14.4 Value Invariants

15. **Progress in range:** If `progress` is present, $0 \leq \text{progress} \leq 1$.
16. **Status valid:** All `status` values are members of the Status enum.
17. **Axis unit valid:** If present, `axis_unit` is a member of AxisUnit enum.

0.4.14.5 Soft Warnings (Valid but Suspicious)

The following are not errors but may indicate authorial mistakes:

- Activity or milestone outside `metadata.time_range` (will be clipped)
- `progress: 1.0` with `status: in-progress` (likely stale)
- Empty `activities` and `milestones` lists (vacuous timeline)
- Unused tracks (no activities reference them)

0.4.15 Complete Examples

The following examples demonstrate realistic Timeline IR documents. Each is a complete, valid document showcasing different timeline use cases.

0.4.15.1 Example 1: Product Roadmap

A quarterly product roadmap with multiple teams, showing how the IR represents a typical software planning artifact.

```
version: "1.0"

metadata:
  title: "Product Roadmap 2026"
  subtitle: "Engineering & Design"
  author: "Product Team"
  created: 2026-06-09
  time_range:
    start: 2026-Q1
    end: 2026-Q4
  axis_unit: quarter
  theme: corporate-blue

tracks:
  - id: platform
    label: Platform
    description: Core infrastructure and APIs

```

```
  - id: mobile
    label: Mobile Apps
    description: iOS and Android applications

  - id: design
    label: Design System
    description: Component library and guidelines

groups:
  - id: foundation
    label: Foundation Work
    members: [api-v2, component-lib]

activities:
  - id: api-v2
    label: API v2 Migration
    track: platform
    start: 2026-Q1
    end: 2026-Q2
    status: in-progress
    progress: 0.6
    category: infrastructure

  - id: mobile-redesign
    label: Mobile App Redesign
    track: mobile
    start: 2026-Q2
    end: 2026-Q3
    status: planned
    description: Complete visual refresh

  - id: component-lib
    label: Component Library
    track: design
    start: 2026-Q1
    end: 2026-Q4
    status: in-progress
    progress: 0.3

  - id: analytics
    label: Analytics Integration
    track: platform
    span: 2026-Q3
    status: planned

milestones:
  - id: api-ga
    label: API v2 GA
    date: 2026-06-30
    track: platform
    status: planned
    icon: flag

  - id: app-launch
    label: App Store Launch
    date: 2026-Q4
    track: mobile
    status: planned
```

```
    category: launch

annotations:
  - type: today-marker
    date: now

legend:
  show: true
  entries:
    - key: infrastructure
      label: Infrastructure
      category: true
    - key: launch
      label: Launch Event
      category: true
```

Listing 14: Product roadmap example

0.4.15.2 Example 2: Executive Program Timeline

A high-level transformation program timeline suitable for board presentations, demonstrating status communication and risk visibility.

```
version: "1.0"

metadata:
  title: "Digital Transformation Program"
  subtitle: "FY26 Executive View"
  author: "PMO"
  created: 2026-06-09
  time_range:
    start: 2026-01-01
    end: 2026-12-31
  axis_unit: month
  theme: executive

tracks:
  - id: strategy
    label: Strategy & Governance

  - id: technology
    label: Technology Modernization

  - id: people
    label: People & Change

activities:
  - id: strategy-define
    label: Strategy Definition
    track: strategy
    start: 2026-01-01
    end: 2026-03-31
    status: done
    progress: 1.0

  - id: vendor-selection
    label: Vendor Selection
```

```
    track: technology
    start: 2026-02-01
    end: 2026-04-30
    status: done

  - id: platform-build
    label: Platform Build
    track: technology
    start: 2026-04-01
    end: 2026-09-30
    status: in-progress
    progress: 0.35

  - id: data-migration
    label: Data Migration
    track: technology
    start: 2026-07-01
    end: 2026-11-30
    status: at-risk
    description: Dependency on legacy system documentation

  - id: training
    label: Training Program
    track: people
    start: 2026-06-01
    end: 2026-12-31
    status: planned

  - id: change-mgmt
    label: Change Management
    track: people
    start: 2026-03-01
    end: ongoing
    status: in-progress

milestones:
  - id: board-approval
    label: Board Approval
    date: 2026-03-15
    status: done
    icon: star

  - id: go-live
    label: Go Live
    date: 2026-10-01
    track: technology
    status: planned
    icon: flag

  - id: program-close
    label: Program Close
    date: 2026-12-15
    status: planned

annotations:
  - type: today-marker
    date: now

```

```
  - type: callout
    target: data-migration
    text: "Risk: Legacy documentation gaps"
    position: above

  - type: bracket
    targets: [platform-build, data-migration]
    label: "Critical Path"

sections:
  - id: h1
    label: "H1 2026: Foundation"
    time_range:
      start: 2026-01-01
      end: 2026-06-30

  - id: h2
    label: "H2 2026: Execution"
    time_range:
      start: 2026-07-01
      end: 2026-12-31
```

Listing 15: Executive program timeline

0.4.15.3 Example 3: Architecture Evolution Timeline

A technical architecture timeline showing system evolution over multiple years, demonstrating coarse granularity and technical categorization.

```
version: "1.0"

metadata:
  title: "Platform Architecture Evolution"
  subtitle: "2026-2028 Technical Roadmap"
  author: "Architecture Team"
  created: 2026-06-09
  time_range:
    start: 2026
    end: 2028
  axis_unit: half
  theme: technical

tracks:
  - id: frontend
    label: Frontend

  - id: backend
    label: Backend Services

  - id: data
    label: Data Platform

  - id: infra
    label: Infrastructure

activities:
  - id: react-migration
```

```
    label: React 19 Migration
    track: frontend
    start: 2026-H1
    end: 2026-H2
    status: in-progress
    category: modernization

  - id: micro-frontend
    label: Micro-Frontend Architecture
    track: frontend
    start: 2027-H1
    end: 2027-H2
    status: planned
    category: architecture

  - id: api-gateway
    label: API Gateway
    track: backend
    span: 2026-H1
    status: done
    category: infrastructure

  - id: service-mesh
    label: Service Mesh Adoption
    track: backend
    start: 2026-H2
    end: 2027-H1
    status: planned
    category: infrastructure

  - id: event-driven
    label: Event-Driven Architecture
    track: backend
    start: 2027
    end: 2028
    status: tentative
    category: architecture

  - id: lakehouse
    label: Data Lakehouse
    track: data
    start: 2026-H1
    end: 2027-H1
    status: in-progress
    progress: 0.4
    category: data

  - id: ml-platform
    label: ML Platform
    track: data
    start: 2027
    end: ongoing
    status: tentative
    category: ai

  - id: k8s-migration
    label: Kubernetes Migration
    track: infra
```

```
    span: 2026
    status: in-progress
    progress: 0.7
    category: infrastructure

  - id: multi-cloud
    label: Multi-Cloud Strategy
    track: infra
    start: 2027
    end: 2028
    status: planned
    category: infrastructure

milestones:
  - id: legacy-sunset
    label: Legacy System Sunset
    date: 2027-H1
    track: backend
    status: planned
    icon: flag

  - id: cloud-native
    label: Cloud Native Milestone
    date: 2026
    track: infra
    status: planned

annotations:
  - type: period
    start: 2026-H2
    end: 2027-H1
    label: "Transition Period"
    style: highlight

legend:
  entries:
    - key: modernization
      label: Modernization
      category: true
    - key: architecture
      label: Architecture Change
      category: true
    - key: infrastructure
      label: Infrastructure
      category: true
    - key: data
      label: Data Platform
      category: true
    - key: ai
      label: AI/ML
      category: true
```

Listing 16: Architecture evolution timeline

0.4.16 Extensibility

The IR is designed to be extended without breaking existing consumers.

0.4.16.1 Metadata Extension

Every entity type includes a `metadata` field (type `map<string,any>`) for application-specific data:

```
activities:
  - id: feature-x
    label: Feature X
    track: platform
    span: 2026-Q2
    metadata:
      jira_key: PROJ-1234
      owner: alice@example.com
      priority: high
```

Renderers should ignore unrecognized metadata keys. Source integrations may use metadata to preserve round-trip fidelity with external systems.

0.4.16.2 Custom Categories

The `category` field accepts arbitrary strings. The theme engine maps categories to visual styles. New categories can be introduced without IR changes:

```
activities:
  - id: security-audit
    category: security   # custom category
    # ...

legend:
  entries:
    - key: security
      label: Security Work
      category: true
```

0.4.16.3 Version Compatibility

The `version` field enables forward compatibility:

- Consumers should accept documents with higher minor versions (1.1, 1.2, etc.) and ignore unknown fields.
- Major version changes (2.0) may introduce breaking changes; consumers should reject documents with unsupported major versions.

0.4.17 Serialization and Syntax

0.4.17.1 Canonical Format

YAML 1.2 is the canonical surface syntax for Timeline IR documents. Design decisions prioritizing YAML:

- **Minimal punctuation:** No braces for simple mappings, no quotes for most strings.
- **Line-oriented:** Each field on its own line for clean diffs.
- **Stable ordering:** Fields should appear in specification order for consistency; validators may enforce this.
- **Comments preserved:** YAML allows comments; tools should preserve them.

0.4.17.2 Alternative Formats

Tooling may support additional formats with lossless conversion:

- **JSON:** Direct mapping; verbose but universally parseable.
- **TOML:** Alternative for users who prefer it.

The IR is defined abstractly; the YAML syntax is one serialization, not the definition.

0.4.17.3 Git Friendliness

For minimal diff churn:

1. Lists use block style (one item per line), not flow style.
2. Fields appear in consistent order.
3. Auto-generated fields (like `updated` timestamps) are optional and may be omitted in version-controlled documents.
4. IDs are stable slugs, not generated UUIDs.

```
# Good: block style, stable order
activities:
  - id: feature-a
    label: Feature A
    track: platform
    span: 2026-Q2

# Avoid: flow style, hard to diff
activities: [{id: feature-a, label: Feature A, track: platform,
   span: 2026-Q2}]
```

Listing 17: Git-friendly style

0.4.18 Validation

A conforming validator must check all invariants in §0.4.14. Validation is a function:

$$\text{validate} : \text{Document} \to \text{Valid} \mid \text{Errors}$$

0.4.18.1 Error Reporting

Validation errors should include:

- Path to the offending field (e.g., `activities[2].track`)
- The invariant violated
- The actual value
- Suggested fix (when determinable)

0.4.18.2 Strict vs. Lenient Mode

- **Strict mode:** All invariants enforced; required for rendering.
- **Lenient mode:** Warnings instead of errors for soft issues; useful for work-in-progress documents.

0.4.19 Relationship to Other Components

The IR is the stable contract between pipeline stages:

Ingestion (upstream)
Source adapters (ADO, GitHub, Markdown, etc.) produce IR documents. The IR defines the target shape; adapters handle mapping.

Theme Engine (downstream)
Themes consume IR and produce styled visual specifications. Themes interpret `status`, `category`, and hint fields (`color`, `icon`) but cannot require additional IR fields.

Renderer (downstream)
The renderer consumes themed IR (or IR + theme) and produces output (SVG, PNG, PDF, PPTX). The IR remains rendering-agnostic; pixel-level decisions are not IR concerns.

Agent Integration
AI agents generate IR directly. The IR's explicit typing, clear required/optional fields, and forgiving structure enable reliable generation. Agents need not understand rendering.

Contract stability: Downstream components may not require IR fields beyond this specification. Extension fields (`metadata`) are pass-through.

0.5 Rendering Model

The rendering model defines a *deterministic mapping* from a Timeline IR document and a theme to a concrete visual geometry: positions, dimensions, label placements, and visual treatments. "Deterministic" is not aspirational; it is a hard contract. Two conforming renderers, given identical IR and theme, must produce geometrically identical output. If they diverge, a defect exists in one renderer—not an interpretation ambiguity in this specification [21].

The model is organized as an ordered six-phase pipeline. Each phase consumes only outputs of preceding phases and produces geometry for later phases. The pipeline is *pure*: no phase reads from the IR after its designated turn, and no phase performs I/O, reads a system clock, or invokes a random number generator.

0.5.1 Determinism Contract

Two conforming renderers R_1 and R_2, given the same IR document D and theme θ, must produce identical coordinate values, label positions, and visual-treatment assignments. This contract requires:

1. **Pure function.** Output is a pure function of (D, θ). No external state—timestamps, random values, environment variables, system locale, or screen resolution—influences any computed value.
2. **Stable sort keys.** Every ordered collection is sorted by an explicit, unambiguous key sequence. Collections without semantic ordering sort by `id` alphabetically:
   - Tracks: by `index` ascending (integer, unique by well-formedness invariant).
   - Activities within a track: by `(start_ordinal, id)` ascending.
   - Milestones: by `(date_ordinal, id)` ascending.
   - Annotations, sections, groups: by `id` ascending.

   The ID values break all ties; global uniqueness is guaranteed by the IR invariant.
3. **Fixed rounding.** All intermediate floating-point values are rounded using *round-half-up* ($\lfloor v+0.5 \rfloor$). SVG coordinate values are expressed to two decimal places using the same convention. No other rounding rule is used anywhere in the pipeline.
4. **Deterministic symbolic date resolution.** The symbolic date `now` and relative dates (`+3m`, `-2w`) resolve to a fixed anchor. The anchor is: `metadata.today` if present; else `metadata.created`; else a validation error (see Gap 1 in §0.5.6). The system clock is never consulted.
5. **Embedded font metrics.** Label-width and line-height computations use metrics from font files bundled with the theme. System fonts are not consulted for any layout-critical measurement. This guarantees that label placement is identical on macOS, Linux, and Windows.
6. **Specification-version governance.** The renderer reads `version` from the IR document and verifies it against a supported specification version before beginning layout. A version mismatch is a hard error, not a warning.

Consequence. If R_1 and R_2 both satisfy this contract and their outputs diverge, the divergent renderer has a defect. The specification is the arbiter.

0.5.2 Coordinate System

The renderer works in a *logical-pixel* coordinate system. The top-left corner of the canvas is the origin (0, 0); x increases rightward, y downward. All layout computations use integer arithmetic in logical pixels; SVG output converts to decimal coordinates at the end.

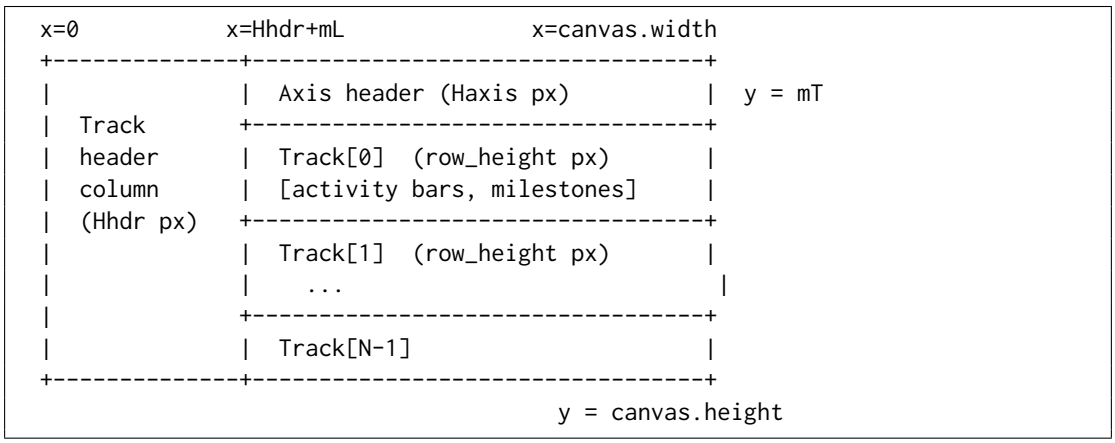


Figure 1: Canvas coordinate system and layout zones

Symbol	Definition
W	`theme.canvas.width` — total canvas width in logical pixels
m_L, m_R, m_T, m_B	Left, right, top, bottom margins
H_{hdr}	`theme.track.header_width` — left column for track labels
H_{axis}	`theme.axis.height` — horizontal time-axis strip
W_{draw}	$W - m_L - m_R - H_{\text{hdr}}$ — drawable timeline width
H_{draw}	$\sum_i h_i + \sum_{i>0} g$ — total height of all track rows plus gaps
H	Computed canvas height: $m_T + H_{\text{axis}} + H_{\text{draw}} + m_B$

Table 18: Coordinate system symbols

The canvas *width* is fixed by the theme. The canvas *height* is always computed from track count and sub-lane expansion; it must never be truncated.

0.5.3 Timeline Axis

0.5.3.1 Axis Unit and Inference

`metadata.axis_unit` controls tick granularity. If absent, the renderer infers a suitable unit from the `time_range` span, applying the first matching threshold:

These thresholds yield 4–26 visible ticks at a 1200-pixel canvas width—the range that avoids crowding while remaining informative for the executive-roadmap use cases described in Section 0.2.

time_range span (days, inclusive)	Inferred axis_unit
≤ 14	`day`
≤ 91	`week`
≤ 548	`month`
≤ 1461	`quarter`
≤ 2922	`half`
> 2922	`year`

Table 19: Axis unit inference thresholds (first match wins)

0.5.3.2 Date-to-Coordinate Function

All horizontal positions derive from a single deterministic function $x(T)$. Let T_{ord} denote the *day ordinal* of date T: an integer count of days since 2000-01-01 (epoch day 0). Coarser-precision dates are resolved to their period-start day before ordinal conversion (see §0.5.3.3).

$$x(T) \;=\; H_{\text{hdr}} + m_L \;+\; \left\lfloor \frac{(T_{\text{ord}} - T_{s,\text{ord}}) \;\cdot\; W_{\text{draw}}}{T_{e,\text{ord}} - T_{s,\text{ord}}} \;+\; 0.5 \right\rfloor$$

where $T_{s,\text{ord}}$ and $T_{e,\text{ord}}$ are the day ordinals of the axis domain start and end after coercion. The numerator product $(T_{\text{ord}} - T_{s,\text{ord}}) \cdot W_{\text{draw}}$ is computed before division to prevent precision loss. For a 1200-pixel canvas spanning ten years (3 650 days), the intermediate product is at most $1200 \times 3650 = 4\,380\,000$, well within 32-bit integer range.

Boundary invariants. $x(T_s) = H_{\text{hdr}} + m_L$ exactly; $x(T_e) = H_{\text{hdr}} + m_L + W_{\text{draw}}$ exactly. Any date at or before T_s maps to the left canvas boundary; any date at or after T_e maps to the right canvas boundary, before clipping.

0.5.3.3 Date Coercion for Bar Edges

Dates coarser than `day` must be coerced to a specific calendar day before ordinal conversion. The rule depends on whether the date is a *left edge* (start of a bar) or a *right edge* (end of a bar):

Date form	As left edge (start)	As right edge (end)
`2026-06-09` (day)	2026-06-09	2026-06-09
`2026-06` (month)	2026-06-01	2026-06-30
`2026` (year)	2026-01-01	2026-12-31
`2026-Q2` (quarter)	2026-04-01	2026-06-30
`2026-H1` (half)	2026-01-01	2026-06-30
`FY26-Q2` (fiscal)	First day of fiscal Q2	Last day of fiscal Q2

Table 20: Date coercion to day boundaries by edge role

Fiscal dates require a fiscal-year start month that is currently absent from the IR contract (see Gap 2 in §0.5.6).

The same left-edge/right-edge coercion applies to `time_range.start` (left) and `time_range.end` (right) when establishing the axis domain.

0.5.3.4 Tick and Gridline Placement

1. Enumerate all `axis_unit` period-start dates within $[T_s, T_e]$ inclusive.
2. For each period boundary T_k: compute $x_k = x(T_k)$ per §0.5.3.2.
3. Place a tick mark of height `theme.axis.tick_height` px at x_k, at the bottom of the axis header band.
4. Place a vertical gridline at x_k spanning from the top of Track[0] to the bottom of Track$[N-1]$, styled per `theme.axis.gridline_*` properties.
5. Place a tick label centered at x_k, `theme.axis.tick_label_offset` px below the tick. Format using `metadata.locale` and the label rules in Table 21. If a label would overlap its neighbor (right edge > neighbor's left edge), suppress the label for this tick only.

axis_unit	Primary label content	Secondary label (every N ticks)
`day`	Day-of-month number	Month name (every 7)
`week`	Week-start day	Month name (every 4)
`month`	Month abbreviation	Year number (every 12)
`quarter`	Q1 / Q2 / Q3 / Q4	Year number (every 4)
`half`	H1 / H2	Year number (every 2)
`year`	Year number	None

Table 21: Tick label content rules per axis unit

Week ticks. For `axis_unit = week`, week boundaries are ISO 8601 Monday starts regardless of locale. The locale affects label string formatting only (language, order), never tick positions.

Section bands. If `sections[]` is non-empty, the theme may render alternating column shading between section boundaries. Band edges coincide with section start/end dates coerced per Table 20. Section bands are painted at the lowest z-order, below gridlines and all content elements.

0.5.4 The Six-Phase Layout Algorithm

The renderer executes geometry computation in exactly six phases in the order given below. Each phase's output is immutable input to subsequent phases. No phase may invoke logic from a later phase.

```
Phase 1 | AXIS COMPUTATION
        |   Input:  IR metadata (time_range, axis_unit, today-anchor)
        |   Output: axis domain [Ts, Te], axis_unit, tick_positions[]

Phase 2 | TRACK PLACEMENT
        |   Input:  tracks[] sorted by index
        |   Output: track.y_top[], track.height[] (provisional)
```

```
8
9  Phase 3 | ACTIVITY GEOMETRY
10         |   Input:  activities[], Phase-1 axis, Phase-2 track
               positions
11         |   Output: activity.{x_left, x_right, y, lane, width}
12         |           track.height[] (final, expanded for sub-lanes)
13
14 Phase 4 | MILESTONE GEOMETRY
15         |   Input:  milestones[], Phase-1 axis, Phase-3 final track
               heights
16         |   Output: milestone.{x_center, y_center, stack_index}
17
18 Phase 5 | SECTIONS AND ANNOTATIONS
19         |   Input:  sections[], annotations[], all Phase 1-4 geometry
20         |   Output: section.{x_left, x_right}, annotation.{x, y}
21
22 Phase 6 | LABEL COLLISION RESOLUTION
23         |   Input:  all geometry from Phases 3-5
24         |   Output: final label positions for milestones
```

Listing 18: Layout pipeline overview

0.5.4.1 Phase 1: Axis Computation

```
1. Resolve today-anchor:
2.      if metadata.today is set : anchor = metadata.today
3.      elif metadata.created is set : anchor = metadata.created
4.      else : ERROR("Cannot resolve symbolic/relative dates: no
            today/created")
5.
6. Resolve time_range.start:
7.      a. Expand 'now' -> anchor; expand '+3m' -> anchor + 3 months;
            etc.
8.      b. Coerce to period start (left-edge rule, Table 5.3).  -> T_s
9.
10 Resolve time_range.end (same expansion; coerce to period end ->
    T_e).
11      ASSERT T_e >= T_s; else ERROR.
12
13 Infer axis_unit if absent:
14      span_days = T_e_ord - T_s_ord
15      Apply Table 5.2 thresholds in order; assign axis_unit.
16
17 Compute W_draw = canvas.width - mL - mR -
    theme.track.header_width.
18
19 Enumerate tick positions:
20      T_k = all period-start dates of axis_unit in [T_s, T_e]
            inclusive.
21      For each T_k: x_k = x(T_k) per Section 5.3.2.
22      Compute tick label strings using metadata.locale.
```

Listing 19: Phase 1: Axis computation

0.5.4.2 Phase 2: Track Placement

```
1. Sort tracks by index ascending.
   ASSERT all index values are unique non-negative integers.

2. y_cursor = mT + Haxis

3. For i in 0 .. N_tracks-1:
     track[i].y_top  = y_cursor
     track[i].height = theme.track.row_height     -- provisional
     y_cursor       += theme.track.row_height + theme.track.row_gap

4. Record group bracket extents after Phase 3 finalises track
   heights.
```

Listing 20: Phase 2: Track placement

Track heights set here are provisional. Phase 3 may expand them when overlapping activities require sub-lanes. Downstream phases must use the Phase-3-finalised heights.

0.5.4.3 Phase 3: Activity Geometry

Sub-lane assignment.

```
For each track T (in index order):
  1. A = activities where A.track == T.id.
  2. Resolve each A.start_ord = ordinal(coerce_left(A.start)).
     Resolve each A.end_x per special-end-date rules below.
  3. Sort A by (start_ord, id) ascending (stable sort).
  4. Greedy sub-lane assignment:
       lane_end_x = [-infinity]
       For each A_i in sort order:
         k = smallest index where lane_end_x[k] <= x(A_i.start_ord)
         if no such k exists:
           if len(lane_end_x) < theme.track.max_sub_lanes:
             append -infinity to lane_end_x; k = new last index
           else:
             k = theme.track.max_sub_lanes - 1   -- cap; emit WARNING
         A_i.lane    = k
         lane_end_x[k] = A_i.end_x
  5. n_lanes = max(A.lane) + 1
  6. if n_lanes > 1:
       track[T].height = theme.track.row_height
                       + (n_lanes - 1) * theme.track.sub_lane_height
  7. For each A_i:
       A_i.y = track[T].y_top
             + theme.activity.bar_top_margin
             + A_i.lane * theme.track.sub_lane_height
       A_i.x_left  = x(A_i.start_ord)
       A_i.x_right = A_i.end_x
       logical_w   = A_i.x_right - A_i.x_left
       if logical_w < theme.activity.min_width:
         mid         = (A_i.x_left + A_i.x_right) / 2   --
            round-half-up
         A_i.x_left  = mid - theme.activity.min_width / 2
         A_i.x_right = A_i.x_left + theme.activity.min_width
       A_i.width = A_i.x_right - A_i.x_left
```

Listing 21: Phase 3: Activity geometry and greedy sub-lane assignment

Special end-date resolution.

- `end` absent (omitted): treated identically to `ongoing`.
- `ongoing`: `end_x` $= H_{\text{hdr}} + m_L + W_{\text{draw}}$ (right canvas boundary); append right-chevron ($\rightarrow$) decoration.
- `tbd`: `end_x` = `x_left` + `theme.activity.tbd_extension_px` (default: $2 \times \overline{\Delta x_{\text{tick}}}$, the mean tick spacing in pixels); render rightward extension with dashed border at 50% opacity; place "TBD" label inside extension zone.
- `unknown`: same geometry as `tbd`; use `unknown` visual treatment from theme status map.
- `~date`: coerce date normally to ordinal; `end_x` $= x$(date_ord); flag right edge as approximate for gradient-fade rendering.

Label placement (deterministic three-way rule). The rule is applied exactly once per activity; no iteration:

1. **Inside**: if `A.width` $\geq$ `theme.activity.label_inside_min_width`, render label horizontally centered in bar, vertically centered on bar height, in `theme.activity.label_color_inside`.
2. **Right**: render label starting at `A.x_right`$+4$ px in `theme.activity.label_color_outside`. If label right edge $> H_{\text{hdr}} + m_L + W_{\text{draw}} - 4$, proceed to step 3.
3. **Left**: render label ending at `A.x_left`-4 px. If label left edge $< H_{\text{hdr}} + m_L$, truncate label to `theme.activity.label_truncate_chars` characters + "…" (U+2026) and re-attempt step 2.

Progress indicator. If `activity.progress` is set: render a filled strip of height `theme.activity.progress_bar_height` px at the bottom of the bar, width $= \lfloor$`A.width` $\cdot$ `A.progress` $+ 0.5 \rfloor$ px, inset inside the bar border.

0.5.4.4 Phase 4: Milestone Geometry

Placement and stacking.

```
For each milestone M (sorted by date_ord, id):
  1. x_center = x(M.date_ordinal).
  2. Determine base_y:
       if M.track is set:
         base_y = track[M.track].y_top + track[M.track].height / 2
       else (cross-track):
         base_y = mT + Haxis + H_draw / 2

  3. Stacking within (effective_track_id, x_center) group:
       Sort group members by id ascending.
       Assign stack_index k = 0, 1, 2, ...
       M.y_center = base_y + k * theme.milestone.stack_offset_y

  4. Diamond vertices (integer arithmetic):
       d = theme.milestone.diamond_size
       top    = (x_center,     M.y_center - d)
       right  = (x_center + d, M.y_center    )
       bottom = (x_center,     M.y_center + d)
       left   = (x_center - d, M.y_center    )
```

Listing 22: Phase 4: Milestone placement and stacking

The diamond is a four-vertex polygon with vertices computed directly. No `transform="rotate(...)"` attribute is used; rotation-transform-dependent rendering differences across SVG viewers are avoided.

Cross-track milestones. When `track` is null, the milestone is drawn at the vertical centre of the full rendered area. Its diamond size is `theme.milestone.diamond_size` (not scaled); visual emphasis comes from the spanning vertical extent of the label, which may be increased by a theme multiplier `theme.milestone.cross_track_label_scale` (default: 1.2).

0.5.4.5 Phase 5: Sections and Annotations

Section bands. For each section S (sorted by start date, then id):

- $x_{\text{left}}(S) = x(\text{coerce\_left}(S.\texttt{start}))$.
- $x_{\text{right}}(S) = x(\text{coerce\_right}(S.\texttt{end}))$.
- Band spans vertically from $m_T + H_{\text{axis}}$ to $m_T + H_{\text{axis}} + H_{\text{draw}}$.
- Z-order: sections paint first (below gridlines, activities, milestones, labels).

Overlapping sections are valid; a later section in sort order paints over an earlier one.

Annotation placement.

1. Sort annotations by `id` ascending.
2. For each annotation: compute anchor as centre of target entity's bounding box.
3. Place annotation box in the first quadrant that keeps it entirely within the canvas. Preference order: top-right, top-left, bottom-right, bottom-left. If no quadrant fits, use top-right and clip to canvas boundary.
4. Draw connector (line/elbow) from annotation box edge to anchor, styled per `theme.annotation.connec`

0.5.4.6 Phase 6: Label Collision Resolution

Activity labels are placed definitively in Phase 3 (no iteration). Milestone labels require a collision-resolution pass because multiple milestones at similar x positions can produce overlapping labels.

```
1. Collect all milestone label bounding boxes (x_label, y_label,
   width, height, id).
2. Sort by (x_label, y_label, id) ascending (stable).
3. Scan pass (bounded by N iterations, N = label count):
     For each consecutive pair (L_i, L_{i+1}):
       if L_i.right_edge > L_{i+1}.left_edge
          AND |L_i.y_label - L_{i+1}.y_label| < label_height:
         L_{i+1}.y_label += theme.milestone.label_stack_offset
```

```
4. Repeat step 3 until no overlapping pairs remain OR N iterations
   reached.
   If overlaps persist after N iterations: truncate labels to
   theme.milestone.label_max_width px, accepting visual overlap.
5. Clamp all final y_label values to [mT, mT + Haxis + H_draw + mB].
```

Listing 23: Phase 6: Milestone label collision resolution

Because the input sort and every shift decision are deterministic, two renderers executing this algorithm on identical inputs produce identical label placements.

0.5.5 Edge Cases

Every case below is normative. A renderer that produces different behaviour for any case has a defect. The summary table gives the complete catalogue; extended descriptions follow for the most structurally complex cases.

Table 22: Edge-case definitions (normative)

Case	Condition	Rendering Rule
Zero-duration activity	`start == end` or `span` that resolves to a single point	Render bar of `min_width` px centred at x(start). Never render 0-width. Label placed right (outside).
Overlapping activities (same track)	Two activities in same track whose date ranges overlap	Greedy sub-lane assignment (Phase 3). Track height expands. Cap at `max_sub_lanes`.
Open interval (`ongoing`)	`end: ongoing` or `end` absent	Bar extends to right canvas boundary. Right-chevron decoration. No overflow beyond canvas edge.
TBD end date	`end: tbd`	Bar extends `tbd_extension_px` beyond `x_left`. Dashed, 50% opacity. "TBD" label inside extension.
Both dates TBD	`start: tbd` or equivalent	Full-track-width hatched block at 40% opacity. Activity label + "(TBD)".
Unknown start	`start: unknown`	Left edge placed at $x(T_s)$. Left-fade gradient over `approx_fade_px`.
Unknown end	`end: unknown`	Same geometry as `tbd`; rendered with `unknown` visual treatment from theme.
Approximate start	`start: ~date`	Geometry at nominal date; left edge rendered with gradient fade of `approx_fade_px`.
Approximate end	`end: ~date`	Geometry at nominal date; right edge rendered with gradient fade.
Activity fully before range	end $< T_s$	Not rendered. Renderer warning. Activity still appears in legend.

continued . . .

Table 22 continued

Case	Condition	Rendering Rule
Activity fully after range	start $> T_e$	Not rendered. Renderer warning. Activity still appears in legend.
Activity partially before range	start $< T_s$, end $\geq T_s$	Bar clipped to $[x(T_s), x(\text{end})]$. Left-clip indicator on clipped edge.
Activity partially after range	start $\leq T_e$, end $> T_e$	Bar clipped to $[x(\text{start}), x(T_e)]$. Right-clip indicator on clipped edge.
Very short span	$x(\text{end}) - x(\text{start}) <$ min_width after rounding	Render at `min_width`; centre on logical midpoint (round-half-up). Label right (outside).
Very long span	Bar occupies $> 80\%$ of W_{draw}	Render normally. Prefer inside label placement if label fits within bar.
Simultaneous milestones (same track)	Two milestones with equal `date` on the same `track`	Stack vertically by `stack_offset_y`, sorted by `id` ascending (Phase 4).
Simultaneous milestones (cross-track)	Two milestones with `track: null` and equal `date`	Stack at full-timeline vertical centre, sorted by `id` ascending.
Milestone outside range	Milestone date $< T_s$ or $> T_e$	Not rendered. Renderer warning. Appears in legend.
Empty track	Track has no activities and no milestones	Row rendered at `row_height`. Label visible. Body empty. Never collapsed.
Sub-lane cap exceeded	Overlapping activities require $>$ `max_sub_lanes` lanes	Excess activities placed in the last lane (visible overlap). Renderer warning.

Overlapping activities: sub-lane assignment in detail. The greedy algorithm assigns each activity (in stable `(start_ord, id)` order) to the lowest-indexed lane where it does not overlap any previously placed activity. Overlap is defined as $x_{\text{left}}(A_j) < x_{\text{right}}(A_i)$ for activities A_i, A_j in the same lane. Because sort order is deterministic and each assignment is a pure function of the current `lane_end_x` state, all renderers produce identical lane assignments for identical inputs.

Simultaneous milestones in detail. When milestones share an (`track`, x_{center}) pair, they are stacked downward from the nominal track centre. The first milestone in `id`-ascending order is at the base position (stack index 0). Subsequent milestones are offset by $k \times$ `theme.milestone.stack_offset_y`. Stacking is permitted to overflow into the inter-track gap; milestones must *not* be silently suppressed if they cannot fit within track height.

Clip indicators. An activity bar clipped at the axis boundary receives a *clip indicator*: a small "//" mark (two angled lines, 4×8 px) drawn in the activity's fill colour at full

opacity, positioned at the clipped edge centre. The indicator signals to the reader that the activity continues beyond the visible axis range.

Minimum-width semantics. `theme.activity.min_width` (default: 4 px) prevents activities from being rendered invisibly narrow at coarse zoom levels. A zero-duration activity and a very-short-span activity both produce a bar of exactly `min_width` px. The two cases are *visually indistinguishable*; distinguishing them would require a separate icon or tooltip, which is a theme-level option outside the core rendering contract.

0.5.6 Known IR Gaps

The following gaps in Mark's IR contract (§0.4) affect rendering determinism. They are flagged here for Mark and Leslie to resolve; this spec does *not* unilaterally extend the IR. Renderers encountering these gaps should apply the documented fallback and emit a validation warning.

Gap 1 — `metadata.today` field missing.
Leslie's decision record mandates an explicit `today` field in the IR for deterministic resolution of the `now` symbolic date and the today-marker feature. Mark's current `metadata` object does not include this field. **Proposed resolution:** add `today` (type `date`, optional) to `metadata`. Renderer fallback chain: `metadata.today` → `metadata.created` → validation error.

Gap 2 — `metadata.fiscal_year_start` missing.
The `FY26-Q2` fiscal-date format requires knowing the fiscal-year start month. Two renderers assuming different organisations' fiscal calendars (US federal: October; UK: April; common corporate: January) will produce different x-coordinates for identical fiscal dates. **Proposed resolution:** add `fiscal_year_start` (type `int`, range 1–12, default 1 = January) to `metadata`.

Gap 3 — Relative date anchor undefined.
Relative dates (`+3m`, `-2w`) are described in Mark's contract as resolving from "context", which is insufficient. A renderer must know the anchor date to compute an ordinal. **Proposed resolution:** define that relative dates use the same anchor as Gap 1 (`today` → `created` → error).

Gap 4 — Omitted `end` semantics ambiguous.
Mark's Activity contract marks `end` as optional but does not state whether an absent `end` is equivalent to `ongoing` or a validation error. This rendering model treats omitted `end` as `ongoing` (see the open-interval row in Table 22). The IR spec should make this explicit.

0.6 Theme Architecture

A theme is a *pure styling layer*: it accepts the geometry produced by the rendering model (Section 0.5) and assigns colors, typography, shapes, and decorations, producing a final visual specification. A theme operates exclusively on signals the renderer has already computed—coordinates, status values, categories, progress fractions. It may not alter geometry, and it may not require IR fields beyond those defined in Section 0.4 [21, 24].

> **Theme-engine contract (binding):** A theme MUST accept all valid IR documents. A theme MUST NOT require IR fields beyond the specification. Two valid themes applied to the same IR may produce visually different outputs, but their underlying geometries—coordinates, stacking order, label positions—must be identical.

This section defines the complete theme schema, describes the five built-in themes, and illustrates how the same IR renders visually differently under each.

0.6.1 Theme Design Philosophy

Themes are built on four principles:

- **Data-independent.** A theme knows nothing about the content of the IR. It maps semantic signals (status, category) to visual treatments through fixed lookup tables. No IR field other than those defined in Section 0.4 is read.
- **Composable.** Themes support single inheritance. A child theme specifies only the properties that differ from its parent; all others inherit. This enables branded variants (`acme-consulting` extending `consulting`) with minimal duplication.
- **Separable.** Swapping themes requires zero IR changes. The same YAML document renders correctly under any theme. Content and style are fully orthogonal.
- **Deterministic.** Themes introduce no layout variability. Given the same IR, theme, and rendering algorithm, output is fully determined (see Section 0.5.1).

0.6.2 Theme Schema

The theme schema defines all visual properties a theme may configure. Properties not set by a theme are inherited from its `extends` parent, defaulting to the built-in `default` theme if no parent is specified. The schema is organized into blocks; all property names, types, and defaults apply uniformly across themes.

0.6.2.1 Canvas and Margins

```
canvas:
  width:            1200  # logical pixels; height is always computed
  background_color: "#FFFFFF"
  margin:
    top:    48            # px above the axis header
    right:  40            # px right of the last tick
    bottom: 48            # px below the last track row
    left:    0            # px left of the track header column
```

Listing 24: Theme schema: canvas block

The canvas height is computed from track heights and inter-track gaps (§0.5.2) and is never specified in a theme.

0.6.2.2 Typography

```
typography:
  font_family:         "Helvetica Neue, Arial, sans-serif"
  # Embedded font files required for deterministic metric
      computation:
  font_files:
    regular: "fonts/HelveticaNeue-Regular.woff2"
    bold:    "fonts/HelveticaNeue-Bold.woff2"
  font_size_base:    11  # pt; body text and activity labels
  font_size_axis:    10  # pt; axis tick labels
  font_size_title:   18  # pt; document title in canvas header
  font_size_subtitle: 13 # pt
  font_size_track:   11  # pt; track header labels
  font_weight_label:  600
  font_weight_axis:   400
  font_weight_header: 700
  locale_aware:      true # use metadata.locale for date label
      formatting
```

Listing 25: Theme schema: typography block

Font embedding requirement. Themes must specify embedded font files (WOFF2 or equivalent) for all families used in layout-critical computations (label width, line height, character advance widths). System fonts are permitted only for decorative text—e.g., HTML tooltip overlays—where exact cross-platform metric agreement is not required. The renderer ships the embedded metrics tables at build time.

0.6.2.3 Axis Styling

```
axis:
  height:                40  # px; height of the time-axis header
      band
  tick_height:            6  # px; length of tick marks
  tick_label_offset:      4  # px; gap between tick mark base and
      label top
  gridline_color:    "#E0E0E0"
  gridline_width:       0.5  # px
  gridline_opacity:     0.8
  gridline_style:    "solid" # solid | dashed | none
  section_band_alpha:   0.04 # alternating column shading opacity (0
      = off)
  today_marker:
    enabled:           false
    color:         "#CC2222"
    width:              1.5  # px
    style:            "solid" # solid | dashed
    label:            "Today"
    label_visible:     true
```

Listing 26: Theme schema: axis block

0.6.2.4 Track Styling

```yaml
track:
  header_width:         160  # px; fixed left column for track label text
  row_height:            48  # px; single-lane track height (no sub-lanes)
  sub_lane_height:       24  # px; height added per additional overlap lane
  max_sub_lanes:          8  # hard cap; excess activities go to last lane
  row_gap:                8  # px; vertical gap between consecutive tracks
  header_font_size:      11  # pt
  header_font_weight:   600
  header_color:    "#111111"
  header_background:   null  # null = transparent
  header_align:    "right"   # right | left | center
  separator_color: "#CCCCCC"
  separator_width:       0.5
  group_bracket_width:    4  # px; left-edge bracket for grouped tracks
  group_label_size:      10  # pt; group label above bracket
```

Listing 27: Theme schema: track block

0.6.2.5 Activity Styling

```yaml
activity:
  bar_height:            24  # px; height within a single lane
  bar_radius:             2  # px; corner radius
  bar_top_margin:        12  # px; from lane top to bar top (centres bar in lane)
  min_width:              4  # px; minimum rendered bar width (see edge cases)
  tbd_extension_px:    null  # null = 2 * mean tick spacing in pixels
  approx_fade_px:         8  # px; gradient fade width for approximate-date edges
  label_inside_min_width: 40 # px; bar must be >= this to place label inside
  label_font_size:       10  # pt
  label_color_inside: "#FFFFFF"
  label_color_outside: "#111111"
  label_truncate_chars:  32  # max chars before ellipsis truncation
  progress_bar_height:    4  # px; filled strip at bar bottom for progress value
  progress_bar_alpha:    0.35
  progress_label_visible: false
```

Listing 28: Theme schema: activity block

0.6.2.6 Milestone Styling

```yaml
milestone:
  diamond_size:          14  # px; half-diagonal of the diamond polygon
  diamond_stroke_width: 1.5  # px
```

```
  cross_track_label_scale: 1.2  # label font size multiplier for
     cross-track
  label_offset_x:          8  # px; gap from diamond right vertex to
     label left
  label_font_size:         9  # pt
  label_max_width:        80  # px; truncate label if wider
  label_stack_offset:     14  # px; y-shift per collision pass (Phase
     6)
  stack_offset_y:         20  # px; y-offset per stacking slot (Phase
     4)
```

Listing 29: Theme schema: milestone block

0.6.2.7 Annotation and Legend Styling

```
annotation:
  font_size:              9   # pt
  background_color: "#FFFFF0"
  background_alpha:       0.9
  border_color:    "#CCCC88"
  border_width:           0.5
  padding:                6   # px; all sides
  connector_style: "elbow"   # elbow | straight | none

legend:
  position:          "bottom" # bottom | right | top | none
  item_height:           14   # px
  item_gap:               8   # px
  font_size:              9   # pt
  swatch_size:           10   # px square
  column_spacing:        16   # px between columns
```

Listing 30: Theme schema: annotation and legend blocks

0.6.2.8 Status-to-Style Map

Every theme must define a complete `status_map` covering all seven IR status values. The map is a lookup table: `status` → { fill, stroke, opacity, pattern }.

Status	Fill role	Opacity	Pattern	Visual intent
`planned`	Accent-light	1.0	`solid`	Default; not yet started
`in-progress`	Accent-dark	1.0	`solid`	Active; full saturation
`done`	Grey-mid	0.85	`solid`	Muted; completed
`at-risk`	Amber	1.0	`solid`	Warning; attention needed
`blocked`	Red	1.0	`solid`	Danger; cannot proceed
`cancelled`	Grey-light	0.60	`diagonal-hatch`	De-emphasized; struck through
`tentative`	Grey-light	0.70	`dashed-border`	Uncertain; may not happen

Table 23: Status-to-style map structure (colour roles; each theme supplies hex values)

Pattern vocabulary. Patterns are defined as named SVG `<pattern>` elements in the theme's `<defs>` block. All themes must use the same pattern geometry so that cross-theme pattern rendering is consistent:

- `solid` — no pattern; fill and stroke colours only.
- `diagonal-hatch` — 45° lines at 4 px spacing, 0.5 px stroke width, stroke colour = `status.stroke`, fill = `status.fill` at 30% opacity.
- `dashed-border` — solid fill at `status.fill`; border rendered as a dashed stroke (4 px on, 4 px off) at `status.stroke`.

Status-map completeness. A theme with a missing status entry is malformed. The renderer must detect and report this at theme load time, not at render time.

0.6.2.9 Category-to-Style Map

Categories are arbitrary strings defined by IR authors. Themes provide an optional `category_map` that overrides colour for activities carrying a matching `category` value:

```
category_map:
  infrastructure: { fill: "#2D6A8F", stroke: "#1A4A6A" }
  security:       { fill: "#8B0000", stroke: "#5A0000" }
  ux:             { fill: "#4A7C59", stroke: "#2A5C39" }
```

Listing 31: Category map example

Category styling overrides *only* the `fill` and `stroke` of the status entry. Opacity and pattern are always drawn from the status map. This preserves the semantic visual signal of status (done = muted, cancelled = hatched) while allowing brand-aligned colour coding per initiative type.

If an activity's category is absent from `category_map`, status-only styling applies without error or warning.

0.6.3 Built-in Themes

Five built-in themes address the primary use cases identified in David's research. Each is a complete, self-contained style specification; none requires IR changes or additional IR fields.

0.6.3.1 Consulting Theme

Visual identity. McKinsey/BCG-style executive roadmap. Black, white, and a single accent colour (default navy `#1F497D`). Heavy typography, generous whitespace, no decorative chrome. Optimised for A4/Letter printing and A3 poster output. Visual language matches consulting deliverables that non-technical executives expect [24].

- **Font:** Helvetica Neue (Arial fallback); weights 400/600/700. Embedded for metric determinism.

- **Canvas:** 1200 px wide; 48 px top/bottom margins; track header width 180 px.
- **Axis:** No gridlines. Tick marks only. Quarter labels in bold uppercase (e.g., **Q1 2026**). Today marker disabled by default.
- **Tracks:** Wide headers (180 px). Thick separator (1 px, `#333333`). No row background colour alternation.
- **Activities:** Square corners (`bar_radius: 0`). Two status colours: navy for planned/in-progress, mid-grey for done. Cancelled activities are rendered at 0% opacity with diagonal hatching at 20% opacity (structural presence without visual weight). No progress bar by default.
- **Milestones:** Solid black diamonds, 14 px half-diagonal. Label right, bold, 9 pt.
- **Legend:** Suppressed (`position: none`). Status is communicated by colour; a legend would clutter the clean aesthetic.

Use case: Programme timelines, transformation roadmaps, board presentations. The deliberate sparseness focuses attention on the strategic narrative rather than operational detail.

0.6.3.2 Executive Theme

Visual identity. Corporate boardroom aesthetic. Navy/slate palette on white, with subtle alternating quarter column shading and a gentle border radius on bars. Status is communicated by both colour and a small status icon at the left of each bar. Designed for 16:9 slide dimensions.

- **Font:** Georgia or Garamond (serif headings); Helvetica Neue (body). Both embedded.
- **Canvas:** 1600 px wide (16:9 target); 40 px margins; header width 160 px.
- **Axis:** Light grey gridlines (0.5 px). Month abbreviation labels. Alternating quarter shading (`section_band_alpha: 0.04`). Today marker enabled (red dashed line, "Today" label).
- **Tracks:** Medium headers (150 px). Alternating row background: white / `#F8F8F8`. Thin separator (0.5 px, `#DDDDDD`).
- **Activities:** Rounded corners (`bar_radius: 4`). Full status palette: navy (planned), royal blue (in-progress), silver (done), amber (at-risk), coral (blocked), grey hatched (cancelled), grey dashed-border (tentative). Status icon (8 pt glyph) at bar left edge; icon colour is white on dark fills.
- **Milestones:** Navy diamonds with a white centre dot (4 px radius inset fill). Label right, 9 pt.
- **Legend:** Right-aligned, full status legend.

Use case: Quarterly business reviews, investor roadmaps, product strategy presentations. Polished without being sparse; the status palette lets executives absorb delivery health at a glance.

0.6.3.3 Product Theme

Visual identity. Developer-friendly; information-dense. Per-track colour coding from the `color` hint field. Category colours take visual precedence over status colours; status is communicated by a small icon rather than fill. Progress bars always visible when set.

- **Font:** Inter or Roboto (system-friendly sans-serif), 10 pt base. Compact sizing for high information density.
- **Canvas:** 1400 px wide; tighter margins (32 px top/bottom); header 140 px.
- **Axis:** Dashed monthly gridlines. Today marker always visible and prominent. Week numbers as secondary axis labels when `axis_unit = week`.
- **Tracks:** Color-coded header backgrounds when `track.color` hint is set. Compact rows (36 px). Sub-lanes shown without track-height expansion cue — the density is expected.
- **Activities:** Slightly rounded bars (`bar_radius: 3`). Category colours override status fill; status glyph icon at bar left edge. Progress bar always rendered (even if thin) when `progress` is set.
- **Milestones:** Smaller diamonds (10 px half-diagonal). Label *below* diamond, 8 pt. Stacks horizontally when space permits.
- **Legend:** Bottom; full category + status legend in two rows.

Use case: Product roadmaps for engineering and product-management audiences. Dense information; appropriate for teams who need per-track status at a glance.

0.6.3.4 Release Theme

Visual identity. Engineering release calendar. Traffic-light status palette (green/amber/red). Monochrome headers. Today marker is the most prominent element. Optimised for CI/CD dashboard display and dense sprint/release schedules.

- **Font:** JetBrains Mono or Courier New (monospace). Consistent character widths simplify label truncation computations.
- **Canvas:** 1200 px wide; tight margins (24 px top/bottom); header 120 px.
- **Axis:** Solid weekly gridlines (1 px, `#CCCCCC`). Today marker: bold red solid line (2 px), "TODAY" label in uppercase red above axis.
- **Tracks:** No separator lines; alternating row backgrounds (`#FFFFFF` / `#F4F4F4`) imply boundaries. Minimal header (dark text on light bg).
- **Activities:** No border radius (`bar_radius: 0`). Traffic-light fill: `#2E7D32` (done), `#1565C0` (in-progress), `#757575` (planned), `#F57F17` (at-risk), `#C62828` (blocked); diagonal hatching for cancelled; dashed border for tentative. No status icon; colour is the sole status signal.
- **Milestones:** Upward-pointing triangle (shape: `triangle-up`), 12 px height, filled with status colour. This is a theme-level shape override; the IR still uses the Milestone entity type.
- **Legend:** Top-right; compact icon + one-word label per status.

Use case: Software release schedules, deployment windows, sprint calendars. Engineers over executives; clarity of status over aesthetic refinement.

0.6.3.5 Minimal Theme

Visual identity. Maximum whitespace, minimum decoration. All bars are a single dark grey regardless of status; status is communicated by pattern alone (hatching for cancelled, dashed border for tentative). Print-optimised; renders correctly in greyscale. Designed to blend into reports and LaTeX documents without visual dominance.

- **Font:** Times New Roman or Latin Modern (LaTeX-compatible serif). 10 pt base.
- **Canvas:** 900 px wide; narrow margins (24 px top/bottom); header 140 px.
- **Axis:** No gridlines. No tick marks. Period labels only (year or quarter), in 9 pt regular weight. No today marker.
- **Tracks:** No header background. Single thin separator (0.5 px, `#AAAAAA`). No row alternation.
- **Activities:** No border radius. All bars `#333333` fill, regardless of status. Pattern differentiates cancelled (diagonal-hatch) and tentative (dashed-border). No progress bar. No icons. Done activities rendered at 65% opacity.
- **Milestones:** Open diamonds (stroke only, no fill), 1 px stroke. Label right, 9 pt regular.
- **Legend:** None (`position: none`).

Use case: Academic papers, internal reports, LaTeX-embedded timelines, any context where the diagram must be subordinate to surrounding text.

0.6.4 Cross-Theme Visual Comparison

To illustrate data-independence, consider a single IR with three tracks (Platform, Mobile, Data), six activities across statuses planned / in-progress / done / at-risk / cancelled / tentative, and two milestones (Beta, GA), spanning 2026-Q1 through 2026-Q4.

Geometric invariant. All five renderings share *identical* geometry: the same bar widths, milestone x-positions, track heights, and label coordinate values. Only style differs. This invariant can be verified by asserting coordinate equality across the five outputs before applying theme styling.

0.6.5 Theme Inheritance and Extension

Themes support single-level inheritance via the `extends` key:

```
theme_id:     "acme-corp"
display_name: "ACME Corporate"
extends:      "consulting"

canvas:
```

Property	Consulting	Executive	Product	Release	Minimal
Background	White	White	White	White	White
Axis labels	Bold Q labels	Small-caps months	Week numbers	Week dates	Year only
Gridlines	None	Light quarterly	Dashed monthly	Solid weekly	None
Track header	180 px bold	150 px serif	140 px colored	120 px mono	140 px serif
Bar corners	Square	Rounded 4 px	Rounded 3 px	Square	Square
Status signal	Colour only	Colour + icon	Icon + colour	Colour only	Pattern only
Done bar	Grey	Silver muted	Grey + icon	Green	Dark 65% opacity
Cancelled bar	0% opacity	Grey hatch	Grey hatch	Grey hatch	Hatch
Progress bar	Hidden	Shown	Always shown	Hidden	Hidden
Milestones	Filled black	Navy + dot	Small, below	Triangle	Open diamond
Today marker	Off	Red dashed	Red solid	Bold red	None
Legend	None	Right, full	Bottom, full	Top-right	None

Table 24: Visual comparison: same IR rendered under five built-in themes

```
  background_color: "#F5F5F0"   # warm paper white

typography:
  font_family: "Futura, Century Gothic, sans-serif"
  font_files:
    regular: "fonts/Futura-Book.woff2"
    bold:    "fonts/Futura-Bold.woff2"

status_map:
  planned:
    fill:   "#005A8E"   # ACME primary brand blue
    stroke: "#003A5E"
  in-progress:
    fill:   "#003A5E"
    stroke: "#001A2E"
```

Listing 32: Custom theme extending Consulting

Inheritance rules:

1. Properties not specified in the child are taken from the parent exactly.
2. `status_map` and `category_map` are merged entry-by-entry: a child overrides only the entries it declares; unspecified entries inherit from parent.
3. Introducing a new font family requires supplying the corresponding embedded font files. A theme that names a font without providing files is malformed.
4. Inheritance depth is exactly one level. Chains (`A` extends `B` extends `C`) are not supported; each theme must be resolvable from its parent alone.
5. A theme may `extend: "default"` explicitly to inherit from the baseline default theme without inheriting any of the five named built-in themes.

0.6.6 Theme-Engine Contract (Formal)

A conforming theme engine must satisfy all of the following:

1. **Accept all valid IR.** A theme engine must not reject a well-formed IR document for content-related reasons. An unrecognised `status` value emits a warning and falls back to `planned` styling.
2. **Require no additional IR fields.** The theme engine reads only the fields defined in Section 0.4. It may read hint fields (`activity.color`, `activity.icon`) as advisory inputs but must not fail if they are absent.
3. **Full status-map coverage.** The theme's `status_map` must contain entries for all seven IR status values. A missing entry is a theme authoring error detectable at theme-load time.
4. **Geometry immutability.** The theme engine applies visual treatment only; it may not alter any coordinate computed by the rendering phases in Section 0.5.4. Style does not affect geometry.
5. **Hint fields are advisory.** The `activity.color` and `activity.icon` hint fields may be silently ignored by the theme engine. If honoured, they supplement status styling (colour override only); they do not replace the pattern or opacity components of the status-map entry.
6. **Unknown categories fall back silently.** If an activity's `category` is absent from the theme's `category_map`, status-only styling applies without error or warning. New categories never require theme changes; they are transparent to the theme engine until explicitly mapped.

0.7 Output Targets

The rendering pipeline produces a single deterministic geometry from an IR document and theme. That geometry is then serialised into one or more output formats. The central architectural choice is to treat **SVG as the primary geometry format**: all other formats are derived from the SVG representation. This design means the determinism guarantee established in Section 0.5.1 is inherited by every format: if the SVG output is deterministic, any format produced from it by a deterministic transformation is also deterministic.

0.7.1 SVG — Primary Geometry Format

Format: Scalable Vector Graphics 1.1 (W3C Recommendation).

Benefits.

- **Deterministic.** SVG coordinates are integers or fixed-decimal values derived from the layout algorithm. Given fixed embedded font metrics, output is byte-stable across runs, platforms, and renderer versions.

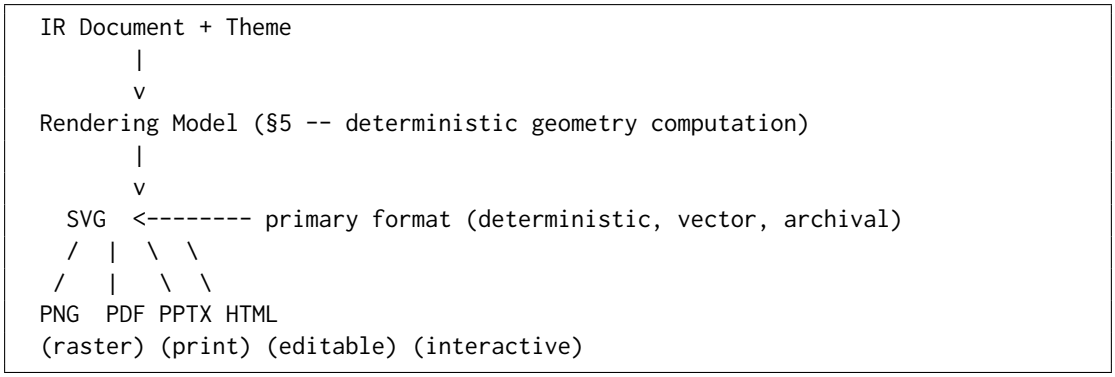


Figure 2: Output pipeline: SVG as primary geometry; all formats derived

- **Scalable.** Vector format; renders crisply at any resolution, from screen preview to A0 conference poster.
- **Inspectable.** XML text format; human-readable, version-controllable with meaningful diffs, and inspectable in browser developer tools.
- **Embeddable.** Can be inlined in HTML, referenced in Markdown (`![]()`), embedded in LaTeX via `\includegraphics`, or opened directly in Inkscape, Figma, and Affinity Designer.
- **Accessible.** Live text elements remain as SVG `<text>` nodes, enabling copy-paste, search, and screen-reader access.

Limitations.

- Text rendering varies across SVG viewers when system fonts are used. Mitigation: embed font files or convert text to paths for archival exports.
- Not natively editable as shapes in PowerPoint or Keynote without conversion.
- Complex diagrams with many activities produce proportionally large SVG files (no rasterization compression).

Implementation strategy.

1. Execute all six layout phases to produce an abstract scene graph (ordered list of typed drawing primitives: `Rect`, `Diamond`, `Line`, `Text`, `Path`, `Group`).
2. Serialise the scene graph to SVG 1.1 with the following invariants:
   - All coordinates rounded to 2 decimal places (round-half-up).
   - All colour values expressed as 6-digit hex (`#RRGGBB`); no named colours, no `rgb()` syntax.
   - All dimension values as integers for rectangles, lines, and polygons; 2-decimal for text positions.
   - Font references via `<style>` or `@font-face` in `<defs>`; no inline `style=` font declarations.
   - Pattern definitions (`<pattern>`) in `<defs>`, referenced by ID (`url(#tc-pattern-diagonal-hatch)`).

- Diamond vertices computed directly; no `transform="rotate(...)"` on individual elements.
- Element IDs use the stable prefix `tc-` followed by the entity `id` from the IR (e.g., `tc-api-redesign`).
- Elements serialised in painting order: sections, gridlines, activity bars, milestones, annotations, labels.

3. Two SVG outputs from identical inputs must be byte-identical: same element order, same attribute order within each element, same `<defs>` structure.

Text-as-paths option. For archival, print, and PPTX export, an optional `-text-as-paths` flag converts all `<text>` elements to `<path>` elements using pre-computed glyph outlines from embedded font files. This eliminates font-dependent text rendering differences at the cost of SVG searchability and file size. It is not the default.

0.7.2 PNG — Raster Output

Format: Portable Network Graphics (ISO/IEC 15948).

Benefits.

- Universal compatibility: pasteable into Google Docs, Microsoft Office, Notion, Slack messages, GitHub issue comments, and presentation tools without conversion.
- Lossless at the selected rendering resolution.
- Predictable, inspectable output; readily embedded in email.

Limitations.

- Not scalable; resolution is fixed at export time. Upscaling produces pixelation.
- Text is rasterised; not searchable or selectable.
- File size grows quadratically with DPI; 300 DPI exports of large canvases can be several megabytes.

Implementation strategy.

1. Rasterise the primary SVG output at a caller-specified DPI. Default: 150 DPI for web/screen use; 300 DPI for print-quality output.
2. Use a headless SVG rasteriser with embedded font support and no system-font dependency. Recommended: `resvg` (Rust, permissive licence) or `librsvg` (C, LGPL). Both accept WOFF2 font files directly.
3. Anti-aliasing: MSAA at 4× (fixed). Sub-pixel AA disabled (produces different results across LCD orientations). Font rendering uses pre-computed glyph outlines (not system hinting).

4. PNG compression: deflate level 6. Level 9 is not used (marginal size savings; produces different byte sequences on different zlib versions, breaking byte-identity guarantee).
5. The rasteriser is treated as a deterministic function: same SVG + same DPI + same rasteriser version → identical PNG bytes.

0.7.3 PDF — Archival and Print

Format: PDF 1.7 (ISO 32000-1); PDF/A-1b recommended for archival.

Benefits.

- Industry-standard archival and print format universally accepted by email, cloud storage, and document workflows.
- Vector quality at any print size (A4 through A0 and beyond).
- Font embedding guarantees identical rendering across all compliant PDF viewers.
- PDF/A profile ensures long-term readability without external dependencies.

Limitations.

- Not editable as shapes in PowerPoint or Keynote; read-only in presentation tools.
- Some SVG features (advanced filter effects, certain gradient types) require PDF equivalents that may not render identically across all PDF viewers.
- Larger file size than PNG for complex diagrams at equivalent visual fidelity.

Implementation strategy.

1. Convert the primary SVG to PDF using a programmatic library. Recommended: `svg2pdf` (Rust, Apache 2.0) or `cairosvg` (Python, MIT). **Not** LibreOffice, headless Chrome, or Puppeteer (too heavyweight; introduces rendering non-determinism from browser layout engine).
2. Embed all fonts from the theme's `font_files` entries, converted to Type 1/CFF for PDF. Subsetting to used glyphs is permitted.
3. Set PDF metadata fields deterministically:
   - `/Title` ← `metadata.title`
   - `/Author` ← `metadata.author` (if set)
   - `/CreationDate` ← `metadata.created` (NOT system timestamp; if `created` absent, omit the field)
4. Generate the PDF document ID as a SHA-256 hash of the SVG content bytes, truncated to 16 bytes and formatted as a PDF hex string. This ensures byte-stable output without relying on random UUIDs.

0.7.4 PPTX — Editable Presentation

Format: Office Open XML Presentation (ISO/IEC 29500). **Library:** `python-pptx` (MIT licence) [22].

The fidelity–editability trade-off. PPTX is the most architecturally complex output target because it offers a fundamental design choice between two strategies, each with distinct properties:

Strategy	Fidelity	Editability	Implementation
Image embedding	Perfect: SVG or EMF embedded as an opaque image element	None: users cannot move, resize, or recolour individual elements	Low: one API call per slide
Native shapes	High: geometry reconstructed as native MSO shape objects	Full: bars, diamonds, and labels are editable PowerPoint shapes	High: requires full geometry-to-EMU conversion and shape API
Hybrid	High for standard elements; exact for decorations	Partial: bars/labels editable; gradient patterns as image overlay	Medium: native shapes with fallback image layer

Table 25: PPTX output strategies and trade-offs

Think-cell comparison. Think-cell [24] achieves the highest-quality editable output: every timeline element is a native PowerPoint shape at the XML level, with full fidelity in font, colour, and position regardless of slide dimensions. Its approach is PowerPoint-locked and requires a proprietary COM add-in, precluding text-format authoring or agent generation paths. Our target is to approach think-cell's editability for the standard element vocabulary (bars, diamonds, text boxes) while retaining full pipeline determinism and text-format IR.

Recommended strategy: native shapes with image fallback.

1. **Slide dimensions.** Set slide width and height to the theme's canvas width and computed height. Coordinate conversion: 1 logical pixel = 914.4 EMU (English Metric Units, the PPTX coordinate unit at 96 DPI logical resolution). Apply round-half-up rounding; this conversion is deterministic.
2. **Canvas background.** A full-slide rectangle fill in `theme.canvas.background_color`.
3. **Track headers.** `MSO_AUTO_SHAPE_TYPE.RECTANGLE` with `TextFrame`. Font from `theme.typography`. No fill (transparent) or light fill per theme.
4. **Activity bars.** `ROUNDED_RECTANGLE` with corner radius proportional to `theme.activity.bar_radius`. Fill colour from the resolved status/category style. For diagonal-hatch pattern: a second semi-transparent rectangle with a hatch pattern fill layer overlaid.
5. **Milestone diamonds.** `MSO_AUTO_SHAPE_TYPE.DIAMOND` shape, filled with the resolved status colour.
6. **Labels.** `TextFrame` inside each shape. Font size, weight, and colour from theme. For labels placed outside bars: free-floating `TextBox` shapes positioned at the coordinates computed in Phase 3/6.

7. **Axis header.** Rectangle shape with tick-label `TextBox` elements. Gridlines as thin Line shapes.
8. **Today marker (if enabled).** A vertical Line shape spanning the full track height, styled per `theme.axis.today_marker`.
9. **Gradient and complex pattern fallback.** Approximate-date gradient fades and annotation connectors that cannot be faithfully represented as native MSO shapes are rasterised to a PNG layer at 150 DPI and embedded as a transparent image shape above the native shapes. The native shapes remain editable beneath the overlay.

Limitations.

- Diagonal-hatch patterns require an MSO hatch fill, which is supported in python-pptx 1.x via the `pptx.oxml` XML interface; it is not a first-class `python-pptx` API.
- Gradient fills at activity bar edges (approximate-date treatment) fall back to the image overlay strategy.
- The slide must be set to a fixed pixel dimension; PPTX files opened on displays with different DPI settings will be scaled by the viewer but remain geometrically faithful.

0.7.5 HTML — Interactive Preview

Format: HTML5 with inline SVG.

Benefits.

- **Zero-install preview.** Any browser can display the file. Ideal for the VS Code extension live-preview pane and for sharing timelines via URL.
- **Interactive.** JavaScript hover handlers can display `description` and `metadata` fields from the IR as tooltips, without altering geometry.
- **Accessible.** SVG `<text>` nodes remain live text; labels are searchable and selectable. `<title>` elements on shapes are readable by assistive technology.
- **MCP-agent integration.** The MCP server (Section 0.9) can return an HTML URL as the rendering result, allowing agents to preview timelines in a browser tab without file-system access.

Limitations.

- The interactive state (hover, click, tooltip visibility) is not deterministic and is not part of the rendering contract. The underlying SVG geometry is identical to the standalone SVG output; only the interactive layer varies.
- Not suitable for archival or print without stripping the JavaScript layer.
- Browser-to-browser rendering differences in SVG gradients and filter effects may produce minor visual variations; these are HTML rendering environment differences, not Timeline Compiler defects.

Implementation strategy.

1. Take the primary SVG output (from §0.7.1) as-is.
2. Wrap in an HTML5 document shell: `<!DOCTYPE html>`, `<meta charset="utf-8">`, `<title>` from `metadata.title`, minimal CSS for page background and centre alignment.
3. Inject the SVG inline (not as `<img src="...svg">`) to allow JavaScript access to SVG DOM nodes.
4. Attach a lightweight JavaScript event listener (no framework dependency) that reads `data-description` and `data-id` attributes from SVG elements (set during SVG serialisation from activity/milestone `description` and `id` fields) and displays a tooltip `<div>` on hover.
5. The JS listener is the only non-deterministic component; it has no effect on SVG geometry. A `-no-js` flag produces geometry-identical HTML with the JavaScript block omitted.

0.7.6 Output Priority Recommendation

The following priority ordering is recommended for implementation, based on the value delivered per implementation unit and the dependency structure of the pipeline:

1. **SVG** — *Build first, always.* SVG is the primary geometry format and the foundation of every other output. Without a correct, deterministic SVG output, no other format is trustworthy. It also serves as the primary debugging surface during development: every layout defect is visible in SVG before any rasterisation or conversion step can obscure it.
2. **PNG** — *Immediate universal value.* PNG requires only a rasteriser (one well-tested library call) and unlocks the most common real-world use case: pasting a timeline into a document, email, or Slack message. The effort is minimal once SVG is correct.
3. **PDF** — *Print and archival; consulting use case.* The primary differentiator vs. Mermaid [11] is presentation quality; PDF is the delivery format for consulting deliverables. The SVG-to-PDF conversion path is well-understood and deterministic. Priority 3 because it requires font subsetting and PDF metadata handling, but the engineering effort is low relative to PPTX.
4. **PPTX** — *Highest value, highest complexity.* PPTX native shapes are the closest equivalent to think-cell [24] editability—the quality benchmark identified in David's research. The implementation is the most complex (EMU coordinate conversion, MSO shape API, hatch pattern XML). Priority 4 because it requires a stable SVG geometry first, and its correctness depends on all earlier priorities being solid.
5. **HTML** — *Developer experience and agent integration.* HTML is nearly free once SVG is correct (wrap in an HTML shell, attach JS). It is essential for the VS Code extension live preview and the MCP agent integration path described in Section 0.9. Priority 5 because its primary consumers (the extension and the MCP server) are themselves lower-priority deliverables in the distribution architecture.

0.7.7 Format Comparison Summary

Property	SVG	PNG	PDF	PPTX	HTML
Scalable	Yes	No	Yes	Yes	Yes
Byte-deterministic	Yes	Yes	Yes	Yes	Yes (geometry)
Editable shapes	Via Inkscape	No	No	Yes (native)	No
Text searchable	Yes	No	Yes	Yes	Yes
Print-ready	Via viewer	At 300 DPI	Yes	Via PPT	No
Universal compat.	High	Very high	Very high	Office-only	Browser-only
Interactive	No	No	No	No	Yes
Impl. complexity	Foundation	Low	Low–medium	High	Low
Build priority	1	2	3	4	5

Table 26: Output format properties and recommended build priority

Determinism across formats. SVG, PNG, and PDF satisfy the full byte-determinism contract defined in Section 0.5.1. PPTX satisfies geometric determinism (same EMU coordinates across runs) but PPTX files contain internal XML IDs (`r:id`) that may vary across `python-pptx` versions; renderers should stabilise these by deriving them from entity `id` values rather than using auto-generated values. HTML satisfies SVG geometry determinism; its interactive JavaScript layer is explicitly excluded from the determinism contract.

0.8 Distribution Strategy

This section evaluates packaging and distribution models for Timeline Compiler, recommending an architecture that serves human authors, AI agents, and embedding use cases.

0.8.1 Distribution Requirements

Timeline Compiler must be accessible to multiple audiences:

CLI Users
— Developers, DevOps engineers, and technical users who invoke rendering from terminal or scripts.

Application Embedders
— Tools that integrate Timeline Compiler as a library (documentation generators, planning tools, dashboards).

AI Agents
— LLM-based agents that need to render timelines as a tool capability.

Non-Technical Users
— Product managers and executives who want live preview while authoring (via IDE integration).

CI/CD Pipelines
— Automated rendering in build processes.

Key requirements:

- Zero-friction installation for common platforms
- Embeddable as a library
- Invocable by AI agents (MCP server or similar)
- Deterministic output regardless of invocation method
- Self-contained — minimal or no external dependencies at runtime

0.8.2 Packaging Options

0.8.2.1 Core Library

Concept: A core rendering library with programmatic API, distributed as a package in the target ecosystem's package manager.

Options:

`@timeline-compiler/core` (npm)
Node.js/TypeScript library. Pros: large ecosystem, easy embedding in JS/TS tools, npm distribution is well-understood. Cons: Node runtime dependency, potentially slow for large renders, binary output (PDF, PNG) may require native dependencies.

Go Module
Single-binary compilation, embeddable via Go's module system. Pros: no runtime dependency, fast, easy cross-compilation. Cons: embedding in non-Go tools requires FFI or subprocess.

Rust Crate
Similar to Go — single binary, high performance. Smaller ecosystem adoption than Go for CLI tools. WASM compilation possible.

Python Package (pip)
Broad adoption in data/ML communities. Cons: runtime dependency, packaging complexity, slower execution.

Recommendation: Implementation language is not decided in this spec, but the core library should expose:

- A stable programmatic API (`render(ir, theme, options) -> output`)
- Schema validation functions
- Theme loading and composition

0.8.2.2 Command-Line Interface

Concept: A CLI tool for rendering from terminal.

```
# Basic rendering
timeline render roadmap.yaml -o roadmap.pdf

# With theme
timeline render roadmap.yaml --theme corporate-blue -o roadmap.svg

# Validate only
timeline validate roadmap.yaml

# Watch mode (optional)
timeline watch roadmap.yaml -o roadmap.svg
```

Listing 33: CLI usage examples

Distribution:

npm global install
`npm install -g @timeline-compiler/cli` — Easy for Node.js users, but requires Node runtime.

Standalone Binary
Distributed via GitHub Releases, Homebrew, apt, etc. No runtime dependency. Cross-platform (macOS, Linux, Windows).

Docker Image
`docker run timeline-compiler render ...` — Isolated environment, good for CI/CD.

npx
`npx @timeline-compiler/cli render ...` — Zero-install execution for one-off use.

Recommendation: Provide both:

- npm package for Node.js ecosystems
- Standalone binaries for users without Node.js

0.8.2.3 VS Code Extension

Concept: IDE integration providing:

- Syntax highlighting for `.timeline.yaml` files
- Live preview panel (re-renders on save or keystroke)
- Export commands (PDF, PNG, SVG, PPTX)
- Schema validation and error highlighting
- Theme selection

Value: Dramatically improves authoring experience for non-CLI users. Live preview is critical for iterative refinement.

Implementation: The extension embeds the core library (via WASM, bundled Node module, or subprocess call to CLI).

Distribution: VS Code Marketplace.

Recommendation: High-priority for adoption. Authors should see their timeline update as they type.

0.8.2.4 MCP Server (Model Context Protocol)

Concept: A server exposing Timeline Compiler as a tool for AI agents.

```
tools:
  - name: render_timeline
    description: "Renders a timeline IR to visual output"
    inputSchema:
      type: object
      properties:
        ir:
          type: object
          description: "Timeline IR conforming to schema"
        theme:
          type: string
          description: "Theme name or inline theme object"
        format:
          type: string
          enum: [svg, png, pdf]
    returns:
      type: string
      description: "Base64-encoded output or file path"
```

Listing 34: MCP tool definition (conceptual)

Value: Agents can render timelines as a native capability. The agent generates IR, invokes the tool, receives visual output.

Deployment:

- Local MCP server (runs alongside the agent environment)
- Hosted MCP endpoint (cloud-deployed rendering service)

Recommendation: Essential for the agent use case. MCP server should be a first-class distribution target, not an afterthought.

0.8.2.5 NPM Package for Embedding

Concept: The core library packaged for use in JavaScript/TypeScript applications.

```
import { render, loadTheme, validate } from
   '@timeline-compiler/core';

const ir = loadYAML('roadmap.yaml');
const errors = validate(ir);
if (errors.length > 0) throw new ValidationError(errors);
```

```
6
7  const theme = await loadTheme('corporate-blue');
8  const svg = await render(ir, theme, { format: 'svg' });
9
10 res.setHeader('Content-Type', 'image/svg+xml');
11 res.send(svg);
```

Listing 35: Embedding in a Node.js application

Use Cases:

- Documentation sites that render timelines dynamically
- Internal tools with timeline visualization
- SaaS products embedding Timeline Compiler

Recommendation: Core distribution path for the JavaScript ecosystem.

0.8.2.6 Standalone Go Binary

Concept: If the implementation uses Go, the CLI is a single statically-linked binary.

Pros:

- No runtime dependencies
- Fast startup
- Easy cross-compilation (macOS, Linux, Windows, ARM)
- Simple distribution (download and run)

Cons:

- Embedding in non-Go applications requires subprocess or FFI
- Community contribution may be lower than TypeScript (smaller Go population among target users)

Recommendation: Attractive for CLI distribution. If TypeScript is the core language, a Go wrapper or WASM compilation may achieve similar benefits.

0.8.3 Implementation Language Trade-offs

The distribution strategy depends on implementation language. Key considerations:

Recommendation: This spec does not mandate implementation language. However, the distribution architecture should support:

1. A core library embeddable in JavaScript/TypeScript contexts (npm)
2. A standalone CLI that does not require a runtime install
3. An MCP server for agent integration

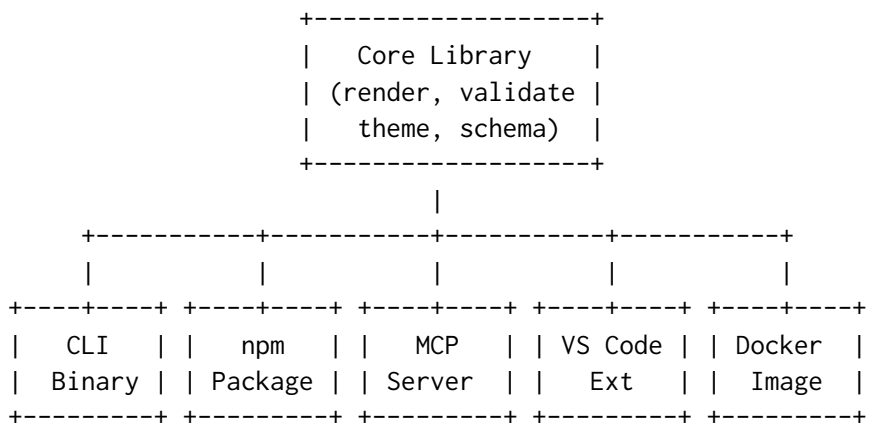
Table 27: Language Trade-offs for Distribution

Factor	TypeScript/Node	Go
CLI standalone binary	Requires pkg/nexe or similar	Native
npm embedding	Native	Requires WASM or subprocess
WASM (browser/edge)	Via esbuild or similar	Native compilation
MCP server	Native (Node ecosystem)	Native
VS Code extension	Native embedding	Subprocess or WASM
Community contributions	Larger pool (frontend devs)	Smaller but growing
PDF generation	Needs native dep (puppeteer, etc.)	Native libraries available
Startup time	Slower (V8 init)	Fast
Binary size	Larger (bundled runtime)	Smaller

4. A VS Code extension with live preview

A TypeScript core with WASM or native compilation for CLI may satisfy all requirements. Alternatively, a Go core with a thin Node wrapper for npm distribution is viable.

0.8.4 Recommended Architecture



Core Library: Single implementation of IR parsing, validation, theme loading, layout, and rendering. All distribution targets wrap this core.

CLI Binary: Wraps core library. Distributed via:

- GitHub Releases (tarballs for each platform)
- Homebrew (`brew install timeline-compiler`)
- npm global install (if Node.js-based)
- curl one-liner for quick install

npm Package: The core library published to npm for JavaScript/TypeScript embedding.

MCP Server: Wraps core library, exposes `render_timeline` and `validate_timeline` tools. Distributed as:

- npm package (for local MCP server)
- Docker image (for hosted deployment)

VS Code Extension: Embeds core library (via bundled WASM or Node module). Published to VS Code Marketplace.

Docker Image: Contains CLI binary. For CI/CD pipelines and isolated execution.

0.8.5 Versioning and Compatibility

Semantic Versioning: All packages follow semver. Breaking changes to IR schema or theme format require major version bump.

Schema Versioning: The IR declares its schema version (`version: "1.0"`). The compiler reports errors if the IR version is unsupported.

Theme Versioning: Themes declare compatibility with IR/compiler versions.

CLI Version Reporting: `timeline -version` reports compiler version, supported IR versions, and supported output formats.

0.8.6 Distribution Priorities

For MVP, distribution priority order:

1. **CLI Binary** — Core functionality, scriptable, CI/CD compatible
2. **npm Package** — Embedding in JS/TS tools
3. **VS Code Extension** — Authoring experience
4. **MCP Server** — Agent integration
5. **Docker Image** — Isolated execution

Post-MVP:

- Homebrew formula
- apt/yum packages
- GitHub Action for rendering in workflows
- Hosted rendering API (optional commercial offering)

0.9 Agent Integration

AI agents are first-class users of Timeline Compiler. They are not an afterthought wired to a human-facing CLI; they are a primary consumer of the IR, the primary beneficiary of a published JSON Schema, and the primary audience for the MCP server surface. This section designs the full agent integration path: from raw inputs through ingestion to a valid IR, through validation and repair, and through round-trip editing without loss of provenance.

The fundamental unit of agent work is the *ingestion task*: given a **data source** (work items, prose, structured export) and a **natural-language prompt**, produce a valid Timeline IR. The prompt is a first-class input—it determines what is selected, how it is framed, which entities are promoted to milestones, and which are dropped entirely. An agent that ignores the prompt and imports everything it can find is not doing its job.

0.9.1 The Ingestion Contract

The ingestion contract defines what the ingestion path is permitted to do with the inputs it receives. Table 28 captures the four categories of ingestion decision. Violations of this contract—particularly silent data loss or silent inference— are the most common source of incorrect or misleading timelines.

Category	Definition	Examples
Assumed	Facts taken from the prompt or schema without source evidence	Title derived from prompt instruction; version `"1.0"`
Inferred	Derived from source data by a deterministic rule	`axis_unit` from date span; `time_range` from min/max activity dates; track from `AreaPath`
Defaulted	Omitted from IR, relying on documented schema default	`status: planned` omitted when all items are future; `locale` omitted
Rejected	Source data discarded, reason logged	Bugs dropped when prompt says "features and milestones only"; items outside prompted time window; duplicate or zero-duration events with no label

Table 28: Ingestion decision categories

0.9.1.1 The Prompt as First-Class Input

The prompt directs every non-trivial ingestion decision. Table 29 lists what the prompt controls and what the ingestion layer must extract from it before touching the source data.

0.9.1.2 What Ingestion May Not Do

- **Must not compute dates.** If a source item has no date, it must use `tbd`, not invent one. Timeline Grammar does not compute schedules (see §0.3).

Prompt Signal	Ingestion Effect
Time horizon (*"Q1–Q3 2026"*)	Sets or constrains `time_range`; items outside window are rejected
Entity filter (*"features and milestones only"*)	Restricts work item types included; bugs, tasks dropped
Track grouping (*"group by team"*)	Maps source field (e.g., `AreaPath`) to tracks
Emphasis (*"highlight at-risk items"*)	Promotes `status` field; may add annotation
Framing (*"executive summary"*)	Reduces label verbosity; promotes high-level items
Title / subtitle	Populates `metadata.title` / `metadata.subtitle`
Milestone promotion (*"mark GA as a milestone"*)	Converts matching activity to milestone entity

Table 29: Prompt signals and their ingestion effects

- **Must not collapse ambiguous status silently.** An ADO work item in state "On Hold" that has no clear IR equivalent must log a rejection warning, not silently map to `planned`.
- **Must not import the whole backlog.** The prompt defines the editorial scope. Importing everything that exists and hoping the renderer trims it is a contract violation.
- **Must not generate non-stable IDs.** IDs derived from sequential numbers (`act-1`, `act-2`) are permitted only as a last resort. IDs should be derived from the item's title slug for git-friendliness (see §0.9.7).

0.9.2 Ingestion Workflows

The following subsections describe four concrete ingestion flows, each with a source excerpt, a prompt, the resulting IR YAML, and an explicit account of what the prompt selected, inferred, and rejected.

0.9.2.1 Azure DevOps Work Items

Source format. The ADO Work Items REST API [13] returns JSON objects. The fields relevant to timeline ingestion are listed in Table 30.

ADO Field	IR Target	Notes
`System.Title`	`activity.label` / `milestone.label`	Verbatim or prompt-shortened
`System.State`	`activity.status`	Via Table 31
`System.WorkItemType`	`activity.category` / entity type	Feature → activity; Milestone –
`System.IterationPath`	`span` or `start/end`	E.g. `Platform\2026\Q2` → `span`
`System.AreaPath`	`track`	Leaf segment becomes track lab
`Microsoft.VSTS.Scheduling.TargetDate`	`milestone.date`	Used when promoting to milesto
`System.Tags`	`activity.tags`	Passed through
`System.Id`	`metadata.ado_id`	Provenance (not rendered)

Table 30: ADO work item field mapping

Source excerpt.

```
# ADO API response (workItems array, fields selected)
- id: 1024
  fields:
```

ADO State	IR Status
New	`planned`
Active	`in-progress`
Resolved	`done`
Closed	`done`
Removed	`cancelled`
On Hold	`blocked`
(other)	`planned` (warn: unmapped state)

Table 31: ADO state to IR status mapping

```
    System.Title: "API Gateway Migration"
    System.State: "Active"
    System.WorkItemType: "Feature"
    System.IterationPath: "Platform\\2026\\Q2"
    System.AreaPath: "Platform Team"
- id: 1025
  fields:
    System.Title: "Mobile App 3.0 GA"
    System.State: "New"
    System.WorkItemType: "Milestone"
    System.AreaPath: "Mobile Team"
    Microsoft.VSTS.Scheduling.TargetDate: "2026-09-15"
- id: 1031
  fields:
    System.Title: "Fix login timeout bug"
    System.State: "Active"
    System.WorkItemType: "Bug"
    System.AreaPath: "Platform Team"
```

Listing 36: ADO work item JSON (abridged)

Prompt. *"Show the Q1–Q3 2026 Platform and Mobile roadmap. Include features and milestones only. Group by team. Drop bugs and tasks."*

Resulting IR.

```
version: "1.0"
metadata:
  title: Platform & Mobile Roadmap
  subtitle: Q1-Q3 2026
  time_range:
    start: 2026-Q1
    end: 2026-Q3
  axis_unit: quarter   # inferred: 3-quarter window -> quarter
     granularity

tracks:
  - id: platform
    label: Platform Team
  - id: mobile
    label: Mobile Team

activities:
  - id: api-gateway-migration
    label: API Gateway Migration
    track: platform
```

```
    span: 2026-Q2        # inferred from IterationPath
       "Platform\2026\Q2"
    status: in-progress   # mapped from ADO state "Active"
    category: feature
    metadata:
      source: ado
      ado_id: 1024
      ado_iteration: "Platform\\2026\\Q2"

milestones:
  - id: mobile-app-3-ga
    label: Mobile App 3.0 GA
    date: 2026-09-15    # from TargetDate field
    track: mobile
    status: planned      # mapped from ADO state "New"
    metadata:
      source: ado
      ado_id: 1025
```

Listing 37: IR produced from ADO work items

What the prompt did.

- **Selected:** Items 1024 (Feature) and 1025 (Milestone). Item 1031 (Bug) was rejected because the prompt said "features and milestones only".
- **Framed:** `time_range` set to Q1–Q3 2026 from prompt instruction. Item 1025's target date (2026-09-15) falls within Q3, so it is included.
- **Inferred:** Tracks derived from `AreaPath` leaf segments. `axis_unit` inferred as `quarter` from the three-quarter window.
- **Dropped:** Bug work item 1031 (not in filter). Had the time range excluded any item, it would be dropped with a logged rejection.

0.9.2.2 Natural Language Prose

When the source is free-form text—a planning document, a briefing note, an email thread—the agent must extract temporal structure from prose. This is the most ambiguity-prone ingestion path; the contract requires explicit handling of every point of uncertainty.

Source text.

```
We are planning to migrate the data warehouse in the first half of
    2026.
The new customer portal is targeted for Q3. Both initiatives are
    owned
by the Engineering org. An executive review is scheduled for 15 July
    2026.
The legacy decommission date is TBD, pending the warehouse cutover.
```

Listing 38: Natural language source

Prompt. *"Show this as a two-track roadmap: Infrastructure and Product. Mark the executive review as a milestone. Include the decommission even though its date is not fixed."*

Resulting IR.

```yaml
version: "1.0"
metadata:
  title: 2026 Engineering Roadmap
  time_range:
    start: 2026-H1
    end: 2026-Q3
  axis_unit: quarter  # inferred: H1+Q3 window, quarterly detail

tracks:
  - id: infrastructure
    label: Infrastructure
  - id: product
    label: Product

activities:
  - id: data-warehouse-migration
    label: Data Warehouse Migration
    track: infrastructure
    span: 2026-H1
    status: planned

  - id: customer-portal
    label: New Customer Portal
    track: product
    span: 2026-Q3
    status: planned

  - id: legacy-decommission
    label: Legacy Decommission
    track: infrastructure
    start: tbd          # explicit: date not yet determined
    status: planned

milestones:
  - id: executive-review
    label: Executive Review
    date: 2026-07-15
    status: planned
```

Listing 39: IR produced from natural language prose

What the prompt did.

- **Structured:** Track assignments (Infrastructure vs. Product) came from the prompt. The source text only named "Engineering org".
- **Promoted:** The executive review was elevated to a milestone (not an activity) by explicit prompt instruction.
- **Included with uncertainty:** Legacy decommission included despite no fixed date; `start: tbd` preserves the fact on the timeline without inventing a date.
- **Inferred:** `time_range` derived from the earliest (`2026-H1` start) and latest (`2026-07-15`) concrete dates. `axis_unit` set to `quarter` as the coarsest granularity that fits the items.

Ambiguity resolution. "First half of 2026" is unambiguously `2026-H1` (ISO half, per the date model). "Q3" without a year is inferred as `2026-Q3` from context (all other

dates are 2026). If the year were ambiguous, the agent must surface a warning rather than guess.

0.9.2.3 GitHub Projects

GitHub Projects (ProjectV2) [5, 6] supports custom field types that carry roadmap-relevant data. The ingestion mapping is summarised in Table 32.

GitHub ProjectV2 Field	IR Target	Notes
`title` (built-in)	`activity.label` / `milestone.label`	Verbatim or shortened
`status` (select field)	`activity.status`	Label mapped to IR enum
`DATE` custom field	`start` / `milestone.date`	Named field determines role
`ITERATION` custom field	`span`	Iteration title maps to quarter
`labels`	`activity.category` (first) / `activity.tags`	First label to category
`milestone` (linked)	`milestones` entry	Promoted to IR milestone
`repository`	`track`	When prompt says "group by repo"
`assignees`	`metadata.assignees`	Not rendered; preserved for provenan
`number` (issue #)	`metadata.github_issue`	Provenance link

Table 32: GitHub ProjectV2 field mapping

The ITERATION field type is the most useful for quarterly roadmaps: iteration titles like "2026 Q2" map directly to `span: 2026-Q2`, making the ingestion path from GitHub Projects to a quarterly roadmap nearly lossless.

Gap note. GitHub Projects does not have a first-class concept analogous to "work item type" (Feature vs. Bug vs. Task). Filtering by type requires label or custom field conventions, which vary per team. The prompt must specify the filter criterion explicitly.

0.9.2.4 Mermaid Timeline Syntax

Mermaid's `timeline` diagram [9, 11] is the most common structured timeline format already embedded in team documentation. Its syntax maps cleanly to IR sections and activities.

Source.

```
timeline
    title Platform Evolution 2026
    section H1 2026
        API Gateway : Q1 : Migration complete
        Auth Refresh : Q2 : New OAuth2 provider
    section H2 2026
        Mobile 3.0 GA : Q3
        Legacy Cutover : Q4 : Final decommission
```

Listing 40: Mermaid timeline source

Resulting IR.

```
version: "1.0"
metadata:
  title: Platform Evolution 2026
  time_range:
```

```
    start: 2026-Q1
    end: 2026-Q4
  axis_unit: quarter

tracks:
  - id: platform
    label: Platform      # default single-track when Mermaid has no
        swimlanes

sections:
  - id: h1-2026
    label: H1 2026
    time_range:
      start: 2026-Q1
      end: 2026-Q2

  - id: h2-2026
    label: H2 2026
    time_range:
      start: 2026-Q3
      end: 2026-Q4

activities:
  - id: api-gateway
    label: API Gateway
    track: platform
    span: 2026-Q1
    description: "Migration complete"
    status: planned

  - id: auth-refresh
    label: Auth Refresh
    track: platform
    span: 2026-Q2
    description: "New OAuth2 provider"
    status: planned

  - id: mobile-3-ga
    label: Mobile 3.0 GA
    track: platform
    span: 2026-Q3
    status: planned

  - id: legacy-cutover
    label: Legacy Cutover
    track: platform
    span: 2026-Q4
    description: "Final decommission"
    status: planned
```

Listing 41: IR produced from Mermaid timeline syntax

Mapping notes. Mermaid `timeline` does not have a swimlane concept; all events are rendered in a single horizontal flow grouped by section. When promoting to IR, a default single track is created unless the prompt specifies a multi-track structure. Mermaid's colon-separated description field maps to `description`. Because Mermaid `gantt` [10] uses a different syntax (task dependencies, progress), Mermaid gantt ingestion requires

a separate mapping pass; that mapping is out of scope for the initial IR but is an extension point (see §0.3).

0.9.3 Reliable IR Generation by Agents

LLM-based agents generate IR reliably when three conditions hold: (1) the target schema is machine-readable and published, (2) the generation call is constrained to valid output shapes, and (3) the IR's structural choices minimise the surface area for generation errors.

0.9.3.1 JSON Schema Publication

The IR schema must be published as a JSON Schema document [17]. This is a binding requirement established by David's research constraints (Constraint 7 in §0.10): without a machine-readable schema, agent generation is unreliable. The schema is a first-class deliverable of this project, versioned alongside the spec.

The top-level JSON Schema structure mirrors the IR document structure directly:

```
# $schema: https://json-schema.org/draft/2020-12/schema
# $id: https://timeline-compiler.dev/schema/v1/timeline.json
title: Timeline IR
type: object
required: [version, metadata, tracks, activities]
properties:
  version:
    type: string
    description: "Specification version, e.g. \"1.0\""

  metadata:
    type: object
    required: [title, time_range]
    properties:
      title:       { type: string }
      subtitle:    { type: string }
      time_range:
        type: object
        required: [start]
        properties:
          start: { "$ref": "#/$defs/Date" }
          end:   { "$ref": "#/$defs/Date" }
      axis_unit:   { "$ref": "#/$defs/AxisUnit" }
      theme:       { type: string }
      locale:      { type: string }

  tracks:
    type: array
    minItems: 1
    items: { "$ref": "#/$defs/Track" }

  activities:
    type: array
    items: { "$ref": "#/$defs/Activity" }

```

```
  milestones:
    type: array
    items: { "$ref": "#/$defs/Milestone" }

  # groups, annotations, sections, legend all optional arrays

$defs:
  ID:
    type: string
    pattern: "^[a-z][a-z0-9-]*$"

  Date:
    type: string
    description: "ISO date, quarter (2026-Q2), half (2026-H1), relative (+3m),
                  symbolic (now), or uncertain token (tbd, ongoing, unknown,
                  or tilde-prefix ~2026-Q2)"

  AxisUnit:
    type: string
    enum: [day, week, month, quarter, half, year]

  Status:
    type: string
    enum: [planned, in-progress, done, at-risk, blocked, cancelled, tentative]

  Track:
    type: object
    required: [id, label]
    properties:
      id:    { "$ref": "#/$defs/ID" }
      label: { type: string }
      color: { type: string }
      order: { type: integer, minimum: 0 }

  Activity:
    type: object
    required: [id, label, track]
    oneOf:
      - required: [start]    # start + optional end
      - required: [span]     # span shorthand
    properties:
      id:       { "$ref": "#/$defs/ID" }
      label:    { type: string }
      track:    { type: string }   # bare string ref to Track.id
      start:    { "$ref": "#/$defs/Date" }
      end:      { "$ref": "#/$defs/Date" }
      span:     { "$ref": "#/$defs/Date" }
      status:   { "$ref": "#/$defs/Status", default: planned }
      progress: { type: number, minimum: 0, maximum: 1 }
      category: { type: string }
      group:    { type: string }
      description: { type: string }
      metadata: { type: object }

  Milestone:
```

```
    type: object
    required: [id, label, date]
    properties:
      id:     { "$ref": "#/$defs/ID" }
      label:  { type: string }
      date:   { "$ref": "#/$defs/Date" }
      track:  { type: string }
      status: { "$ref": "#/$defs/Status", default: planned }
      category: { type: string }
      icon:     { type: string }
      description: { type: string }
      metadata: { type: object }
```

Listing 42: Abbreviated JSON Schema for the IR (YAML representation)

The schema is published at a versioned URL. Minor version bumps to the IR may add optional fields; the schema handles this via `additionalProperties: true` for `metadata` maps and by allowing unknown top-level fields on minor revisions.

New cite key needed. The JSON Schema specification itself does not have a cite key in `references.bib`. A key `json-schema2020` pointing to the IETF JSON Schema draft (2020-12) should be added by David.

0.9.3.2 Structured Output Constraints

Platforms that support constrained generation [17] should use the published JSON Schema as the output schema for the IR generation call. This eliminates entire classes of generation errors:

- Required fields can never be missing when the schema enforces `required`.
- ID format is enforced by the `pattern` constraint on the `ID` definition, preventing spaces, uppercase, or other invalid characters.
- Status values are constrained to the enum; the model cannot hallucinate "completed" or "active".
- References are just bare strings—the model emits `track: "platform"`, not a nested object, which is trivially correct.

When structured output is not available, the agent prompt must include the JSON Schema inline and instruct the model to validate its own output before returning it.

0.9.3.3 IR Design Choices That Help Agents

Several deliberate IR design choices reduce the error rate for LLM-generated documents:

Optional fields default to safe values.
`status` defaults to `planned`; `progress` defaults to `null`. An agent that omits these fields produces a valid document. Agents only need to emit fields they have evidence for.

`span` shorthand reduces reasoning load.
Instead of requiring an agent to compute both a `start` and `end` date for a quarter-length activity, it can emit `span: 2026-Q2` and let the compiler expand it. This avoids off-by-one boundary errors (is Q2 end June 30 or July 1?).

References are bare strings.
`track: platform` is far easier for an LLM to emit correctly than a nested reference object. The schema context (field name) determines the target type; no wrapper object is needed.

IDs are kebab-case slugs.
Agents derive IDs from labels: "API Gateway Migration" → `api-gateway-migration`. This is a simple and reliable transformation. UUIDs would be safe but opaque; sequential numbers would collide with re-generation.

No dependency scheduling.
Since the IR is a Timeline Grammar (not a Task Scheduling Grammar), there is no dependency resolution. Each activity is independently valid. Agents do not need to compute a topological order or check resource constraints.

0.9.4 Validation Pipeline

Validation is layered. Each layer is a gate: failure at an earlier layer produces errors that block later layers. This design prevents cryptic failures (e.g., a reference-resolution error caused by a silently malformed ID).

Layer	Name	What it checks	Failure mode
1	Syntactic parse	Valid YAML or JSON; no parse errors	Hard error: stop
2	Schema conformance	Required fields present; types match; enum values valid; ID regex; `oneOf` for start/span	Hard error: list all violations
3	Well-formedness	Referential integrity; ID uniqueness; date ordering; progress bounds; group acyclicity	Hard error: list all violations
4	Render-readiness	See §0.5 (Barbara)	Error or warning depending on severity
5	Semantic advisory	Degenerate documents; empty `activities` and `milestones`; items outside `time_range`	Warning only

Table 33: Validation pipeline layers

Layer 4 dependency. Render-readiness validation—checking that the IR can produce a legible visual output—is defined and owned by Barbara in §0.5. This includes checks such as overlapping activities on the same track that cannot be visually separated, labels that exceed track width at the chosen axis unit, and milestone dates that fall outside the document's `time_range`. The validation pipeline calls Barbara's render-readiness checks as Layer 4; this section does not duplicate them.

0.9.4.1 Errors vs. Warnings

Error
Prevents rendering. The document is invalid and the renderer must refuse to proceed. Errors come from Layers 1–3 and from Layer 4 severity "fatal". The validator must return *all* errors in a single pass, not just the first.

Warning
The document is valid but the output may be suboptimal. Warnings come from Layer 4 (advisory) and Layer 5. Renderers proceed and may annotate the output (e.g., a clipped activity) or log the warning and continue.

0.9.5 Error Handling and Agent Repair Cycle

0.9.5.1 Error Message Format

Every validation error is **path-anchored**: it identifies the exact location in the document, the nature of the violation, and a suggested fix. This design enables an agent to mechanically apply repairs without re-reading the whole document.

The error message format is:

```
ERROR  <path>: <description>
       Expected: <constraint>
       Found:    <actual value>
       Fix:      <suggested correction>
```

Listing 43: Validation error message format

Concrete error examples:

```
# Missing required field
ERROR  activities[0].id: required field 'id' is missing
       Expected: string matching ^[a-z][a-z0-9-]*$
       Found:    (absent)
       Fix:      add id: api-gateway-migration

# Invalid ID format
ERROR  activities[1].id: "API Migration" does not match
    ^[a-z][a-z0-9-]*$
       Expected: kebab-case slug
       Found:    "API Migration"
       Fix:      id: api-migration

# Unresolved track reference
ERROR  activities[2].track: "eng" references a non-existent track
       Expected: one of [platform, mobile, data]
       Found:    "eng"
       Fix:      track: platform  (or add a track with id: eng)

# Invalid status value
ERROR  activities[3].status: "active" is not a valid Status
       Expected: one of [planned, in-progress, done, at-risk,
                         blocked, cancelled, tentative]
       Found:    "active"
```

```
       Fix:      status: in-progress

# Date ordering violation
ERROR  activities[4]: end precedes start
       Expected: end >= start
       Found:    start: 2026-Q3, end: 2026-Q1
       Fix:      swap dates, or use span: 2026-Q3 if this is
                 a single-quarter activity

# Progress out of range
ERROR  activities[5].progress: 1.4 is outside [0, 1]
       Expected: number in [0, 1]
       Found:    1.4
       Fix:      progress: 1.0

# Duplicate ID
ERROR  milestones[1].id: "platform-launch" is already used by
   activities[2].id
       Expected: globally unique identifier
       Found:    duplicate
       Fix:      id: platform-launch-ms  (or another unique slug)

# Missing date source (neither start nor span present)
ERROR  activities[6]: no temporal anchor defined
       Expected: exactly one of: (start [+ optional end]) or span
       Found:    neither start nor span present
       Fix:      add  span: 2026-Q2  or  start: 2026-04-01
```

Listing 44: Example validation errors with suggested fixes

0.9.5.2 Agent Repair Cycle

The validation-error-repair loop is the core of reliable agent IR generation. The cycle is:

1. **Generate.** Agent generates an IR document, optionally using structured output constraints (§0.9.3.2).
2. **Validate.** The IR is submitted to the `validate_timeline` MCP tool (§0.9.6.1). All layers are run; all errors collected.
3. **Report.** Errors are returned to the agent as a structured list with path, description, and suggested fix.
4. **Repair.** The agent applies targeted fixes. Because errors are path-anchored and include suggested fixes, repairs are surgical: the agent patches `activities[2].track` rather than regenerating the entire document.
5. **Re-validate.** The patched document is re-submitted. This loop repeats until validation passes with no errors (warnings may remain).
6. **Render.** Once validation passes, the agent calls `render_timeline` to produce output.

In practice, structured output constraints eliminate most Layer 2 errors on the first generation pass. Layers 3 (referential integrity, date ordering) are the most common source of residual errors for agents generating multi-track documents.

0.9.6 MCP Server

The MCP server [3] exposes Timeline Compiler as a tool-callable service for AI agents. It is a first-class distribution target, not an afterthought (see the distribution architecture in §0.8). The server hosts four tools.

0.9.6.1 Tool Contracts

validate_timeline Runs the full validation pipeline (Layers 1–5) against a provided IR document. Returns all errors and warnings in structured form.

```
tool: validate_timeline
description: >
  Validate a Timeline IR document. Runs syntactic, schema, well-formedness,
  render-readiness, and semantic advisory checks. Returns structured errors
  and warnings with path anchors and suggested fixes.

input:
  ir:
    type: object | string   # IR as parsed object or raw YAML/JSON string
    required: true
  stop_at_layer:
    type: integer           # optional; 1-5; default: 5 (run all layers)
    required: false

output:
  valid:     boolean        # true iff zero errors (warnings do not block)
  errors:
    - path:    string       # e.g. "activities[2].track"
      code:    string       # machine-readable error code, e.g. UNRESOLVED_REF
      message: string       # human-readable description
      fix:     string       # suggested correction
  warnings:
    - path:    string
      code:    string
      message: string
  layers_run: integer       # how many layers were executed before stopping
```

Listing 45: `validate_timeline` tool contract

render_timeline Renders a valid Timeline IR to a visual output format. Returns the rendered output as a base64-encoded string or a file path (for local MCP deployments).

```
tool: render_timeline
description: >
  Render a Timeline IR document to SVG, PNG, PDF, or PPTX. The IR must
  be valid (validate_timeline returns valid: true) before rendering.
```

```
  Rendering is deterministic: identical IR + theme + format always
  produces bit-identical output.

input:
  ir:
    type: object | string   # IR as parsed object or raw YAML/JSON
       string
    required: true
  format:
    type: string
    enum: [svg, png, pdf, pptx]
    default: svg
    required: false
  theme:
    type: string            # theme identifier, e.g. "default",
       "corporate"
    default: "default"
    required: false
  width:
    type: integer           # output width in pixels (PNG/PDF only)
    required: false

output:
  format:  string           # echo of requested format
  data:    string           # base64-encoded output bytes
  mime_type: string         # e.g. "image/svg+xml"
  warnings:                 # render-layer warnings (does not block
     output)
    - message: string
```

Listing 46: `render_timeline` tool contract

`describe_schema` Returns the full JSON Schema for the IR and field-level documentation. Agents call this to bootstrap IR generation or to recover from schema errors.

```
tool: describe_schema
description: >
  Return the JSON Schema for the Timeline IR, plus human-readable
  documentation for each field. Use this to understand what fields
     are
  required, their types, allowed values, and defaults.

input:
  version:
    type: string            # IR version to describe; default: latest
    required: false
  entity:
    type: string            # optional: limit to one entity, e.g.
       "Activity"
    required: false

output:
  version:     string       # IR version described
  schema:      object       # full JSON Schema document
  docs:
    - entity:  string       # entity name
      fields:
```

```
        - name:        string
          type:        string
          required:    boolean
          default:     string | null
          description: string
          example:     string
```

Listing 47: `describe_schema` tool contract

`suggest_time_range` A convenience tool for agents that need to derive a sensible `time_range` and `axis_unit` from a collection of candidate activities before composing the full document. This is especially useful when ingesting data from sources that lack explicit time windows.

```
tool: suggest_time_range
description: >
  Given a list of activity date hints (start, end, span values), suggest
  appropriate time_range and axis_unit values for the IR metadata. Does not
  require a complete IR document.

input:
  dates:
    type: array             # list of date strings, e.g. ["2026-Q1", "2026-09-15"]
    items: { type: string }
    required: true
  prefer_unit:
    type: string            # optional hint: "quarter", "month", etc.
    required: false

output:
  time_range:
    start: string
    end:   string
  axis_unit: string         # recommended AxisUnit
  rationale: string         # explanation of the choice
```

Listing 48: `suggest_time_range` tool contract

0.9.6.2 MCP Server Deployment

The MCP server is deployed in two modes:

Local server
Runs as a subprocess alongside the agent environment. Started via the CLI (`timeline mcp-server -port 3000`) or as an npm/pip package. Suitable for development, CI, and agent runtimes on the same host.

Hosted endpoint
A cloud-deployed instance of the MCP server. Suitable for cloud-native agent runtimes where a local subprocess is impractical.

Both modes expose identical tool contracts. The local server operates on the local filesystem (output file paths are local); the hosted server returns base64-encoded output in all cases.

0.9.7 Round-Trip and Iterative Editing

A Timeline IR document lives in version control alongside the source data it was derived from. Humans and agents edit it directly. Re-syncing against an updated source must not silently overwrite editorial decisions made in the IR. This section designs the mechanisms that make round-trip editing safe.

0.9.7.1 Source Provenance via Metadata

Every entity produced by an ingestion workflow includes a `metadata` block that records the source of record:

```
activities:
  - id: api-gateway-migration
    label: API Gateway Migration
    track: platform
    span: 2026-Q2
    status: in-progress
    metadata:
      source: ado
      ado_id: 1024
      ado_revision: 42           # ADO ETag for change detection
      ado_area: "Platform Team"
      ado_iteration: "Platform\\2026\\Q2"
      ingested_at: 2026-06-09
```

Listing 49: Provenance metadata block (ADO example)

```
activities:
  - id: customer-portal-redesign
    label: Customer Portal Redesign
    track: product
    span: 2026-Q3
    status: planned
    metadata:
      source: github
      github_project_id: PVT_kwDOA12345
      github_issue: 214
      github_url: "https://github.com/acme/portal/issues/214"
      ingested_at: 2026-06-09
```

Listing 50: Provenance metadata block (GitHub Projects example)

The `source` key in `metadata` is a reserved provenance marker. The ingester writes it; the renderer ignores it. Re-sync logic uses it to correlate IR entities back to their source records.

0.9.7.2 Re-Sync Strategy

When source data is updated (e.g., a work item's state changes from Active to Resolved), the re-sync operation must:

1. **Match by source ID.** Use `metadata.ado_id` or `metadata.github_issue` as the stable foreign key, not the IR `id`. The IR `id` is a human-edited slug that may have been renamed.
2. **Update only source-mapped fields.** Fields that the ingestion workflow maps from the source (e.g., `status`, `span` derived from IterationPath) are overwritten. Fields the human may have edited in the IR (`label`, `category`, `description`, `color`, `progress`) are *not* overwritten.
3. **Preserve the IR `id`.** The kebab-case slug must not be regenerated on re-sync. Once set, it is stable.
4. **Log what changed.** The re-sync operation must produce a structured change log (entity id, field, old value, new value) before writing, enabling human review.
5. **Reject destructive deletions.** If a source item is deleted or removed from scope, the re-sync must flag it as a warning rather than silently removing the IR entity. A human or agent must confirm removal.

0.9.7.3 Stable IDs and Git-Friendly YAML

The combination of stable kebab-case IDs and line-oriented YAML serialisation produces IR documents that diff well under version control:

```
  activities:
    - id: api-gateway-migration
      label: API Gateway Migration
      track: platform
      span: 2026-Q2
-     status: in-progress
-     progress: 0.4
+     status: done
+     progress: 1.0
      metadata:
        source: ado
        ado_id: 1024
```

Listing 51: Git diff of an IR update (status change + progress update)

Key properties of the YAML serialisation that preserve git-friendliness:

- Fields within each entity are emitted in a **canonical order** (required fields first, optional fields in schema definition order). Reordering fields does not change semantics but does generate spurious diffs; canonical order prevents this.
- **No auto-generated IDs or timestamps** in non-`metadata` fields. The IR ID is derived from content and frozen; `ingested_at` in `metadata` only changes when the entity is re-ingested from source.
- **Consistent quoting conventions.** Strings that are safe as bare YAML scalars are not quoted; strings with special characters use double quotes. Serialisers must apply this consistently to avoid quote-flip diffs.

- **Lists maintain insertion order**, not alphabetical. New entities appended to the end of a list produce a single-line diff at the bottom.

0.9.7.4 Human Edits and Incremental Refinement

A human author may open the IR YAML in any editor, add annotations, adjust labels, add a milestone the ingester missed, or change a `status` value to reflect a decision made after ingestion. These edits are valid IR operations and must survive re-sync (per the strategy above).

An agent may similarly refine an IR: for example, an editorial agent might re-read the document, notice that five activities in one track are unlabelled, and add descriptions. Because the agent is editing a YAML file with stable IDs and a well-defined schema, this is a safe, auditable operation. The MCP `validate_timeline` tool can be called after any edit to confirm the document remains valid.

The editing workflow is:

```
1. ingest(source, prompt)   -> roadmap.yaml          # initial
   ingestion
2. validate(roadmap.yaml)   -> valid, 0 errors        # confirm valid
3. edit(roadmap.yaml)       -> roadmap.yaml (modified) # human/agent
   edit
4. validate(roadmap.yaml)   -> valid, 0 errors        # re-confirm
5. render(roadmap.yaml, svg) -> roadmap.svg            #
   deterministic output
6. (commit roadmap.yaml + roadmap.svg to git)
```

Listing 52: Iterative refinement loop (human or agent)

Step 6 commits both the IR source and the rendered output. Because rendering is deterministic, the SVG in the repository is always reproducible from the YAML. The YAML is the source of truth; the SVG is a derived artefact that can be regenerated on demand.

0.9.8 IR Gaps and Open Questions

During the design of this section, three gaps or ambiguities in the IR contract were identified. These are flagged here for Leslie (reviewer) and Mark to reconcile; they have *not* been silently resolved in this section's examples.

1. **`today` field missing from metadata.** Leslie's scope decisions specify that the `now` symbolic date must be evaluated from an explicit `today` field in the IR (not from the system clock), in order to preserve determinism. However, the `metadata` table in Mark's IR contract does not include a `today` field. Until this is resolved, the annotation type `today-marker` with `date: now` is non-deterministic. Suggested resolution: add `today: date?` to the `metadata` schema; if absent, render-time clock is used and a warning is emitted.
2. **Track sort field: `index` vs. `order`.** Mark's binding contract (well-formedness invariant 14) refers to a `track.index` field with the rule "track index order values unique and non-negative". The `04-ir.tex` specification defines the same field as

`order` (type `int?`). The canonical field name should be resolved to one term; this section uses `order` to match the spec, but the contract should be updated to match.

3. **`span` vs. `start`+`end` mutual exclusion.** The IR allows either `span` (shorthand) or `start` plus optional `end`, but does not specify the behaviour when both are present (e.g., `span: 2026-Q2` and `start: 2026-05-01` coexist in the same activity). Should this be a schema error, or should one take precedence? For agent generation, structured output can prevent this from occurring, but the validation specification needs an explicit ruling. Suggested resolution: treat co-presence of `span` and `start`/`end` as a schema error.

0.10 Comparison Analysis

0.10.1 Framing the Comparison

Timeline Compiler is not a project-management tool, a Gantt chart engine, or an interactive web application. It is a *visual-communication compiler*: a system that accepts structured temporal data (or natural-language plans) and deterministically renders presentation-quality timeline artefacts — SVG, PNG, PDF, and PPTX — from a stable, version-controlled, LLM-friendly Intermediate Representation (IR).

The competitive landscape therefore contains two distinct clusters:

- **Diagram-as-code tools** (Mermaid, PlantUML, D2) — source- control friendly, developer-facing, but limited in presentation fidelity and output format breadth.
- **Presentation / project-management tools** (think-cell, PowerPoint, MS Project) — presentation-quality, but proprietary, manual, and hostile to automation and version control.

Timeline Compiler is designed to occupy the empty cell at the intersection: open-source, git-friendly, agent-generation-friendly, *and* presentation-quality. Each tool below is evaluated on the axes most relevant to that position.

0.10.2 Tool-by-Tool Analysis

0.10.2.1 Mermaid (timeline and gantt)

Mermaid [11] is an MIT-licensed JavaScript diagramming library with over 88,000 GitHub stars [26] and native rendering on GitHub, GitLab, Notion, Obsidian, Azure DevOps, and hundreds of other platforms [11]. Its `timeline` diagram type [9] graduated from beta in 2023. The syntax groups events under named sections on a horizontal time axis:

```
timeline
  title 2025 Platform Roadmap
  section Infrastructure
    2025 Q1 : Database migration
```

```
    2025 Q2 : CDN rollout
  section Product
    2025 Q2 : v2 public beta
    2025 Q4 : GA launch : milestone
```

Mermaid also provides a `gantt` diagram type [10] with sections, task durations, dependencies, and a today-marker.

What Mermaid does well. Git-friendly plain text; effortless integration into Markdown-driven workflows; zero-install web rendering; large community; MIT license; AI/LLM tools already generate valid Mermaid syntax.

Where Mermaid falls short for our goal.

- *Presentation quality is limited.* Output is SVG/PNG rendered by a JavaScript engine. The `timeline` type lacks per-row swimlane formatting, precise font control, brand themes, or layout tuning suitable for executive slides.
- *No PPTX or PDF output.* Export is browser-dependent; no native headless PPTX generation.
- *The gantt type defaults to project-management idioms.* It renders task bars with dependency arrows and progress indicators — appropriate for engineering sprints, not for McKinsey-style roadmaps where visual communication is the goal.
- *Non-deterministic layout at scale.* Label placement and bar sizing change with viewport width; large roadmaps (> 50 events) degrade in readability.
- *No first-class theme layer.* Theming is CSS injection, not a typed theme schema.

0.10.2.2 PlantUML

PlantUML [18] is a GPL/commercial text-to-diagram tool built on GraphViz. Its `@startgantt` [19] mode is experimental as of 2024 and materially less capable than Mermaid's gantt: no section grouping, no resource assignment, no today-marker, and limited coloring. Its timeline diagram type is likewise minimal.

PlantUML outputs PNG, SVG, ASCII art, and PDF, and integrates with Confluence, Jenkins, and many JetBrains plugins. The server-side rendering architecture (a Java process) is reliable for CI pipelines.

Where it falls short. Gantt/timeline support is a secondary feature. Syntax is verbose and UML-centric — not natural for non-technical stakeholders. Visual output has a distinctly technical aesthetic; unsuitable for polished executive deliverables. The GPL license creates friction for proprietary embedding.

0.10.2.3 vis-timeline

vis-timeline [25] (MIT) is an interactive JavaScript timeline widget. Items and groups are defined as JavaScript objects; the library renders them in the browser as scrollable, zoomable, drag-and-drop timelines.

Where it falls short. It is entirely browser-interactive: there is no declarative text format (source data is a JavaScript data structure), no static SVG/PDF export, and no presentation layer. It is the right tool for dashboard UIs, not for printed roadmaps or slide decks. Version-controlling the rendered output is not meaningful because the output is dynamic HTML.

0.10.2.4 Frappe Gantt

Frappe Gantt [4] (MIT) is a lightweight SVG-based Gantt library used in ERPNext. Tasks are provided as JavaScript objects with `start`, `end`, `progress`, and `dependencies` fields. It renders to SVG in the browser.

Where it falls short. Like vis-timeline, input is programmatic JavaScript, not a text-serialisable data format. There is no declarative IR, no file-format input (YAML/JSON from a file), no PPTX/PDF output, and no theme system. The presentation quality of the output is good for a project-tracking dashboard but lacks the typography and layout control needed for executive presentations.

0.10.2.5 Microsoft Project

MS Project [14] is the industry-standard project-management application. It exports to a custom XML schema (`Project.xsd`) [12] that is non-deterministic between saves: GUIDs and timestamps change on each export, making diffs meaningless without post-processing. Output formats are proprietary `.mpp` binary and the unstable XML. No git-friendly text format exists natively.

Where it falls short. Proprietary, expensive ($~30/user/month), entirely interactive, and requires manual visual design for presentation quality. Resource allocation and critical-path scheduling are first-class concerns — exactly the project-management semantics Timeline Compiler intentionally excludes (see §0.10.4). The binary format is hostile to LLM generation.

0.10.2.6 think-cell

think-cell [24] is a PowerPoint add-in widely used by management consultancies (McKinsey, BCG, Bain). It produces presentation-quality Gantt/timeline charts [23] with quarter/month/week granularity, responsibility columns, and smart label placement. Excel data-linking enables live updates. Pricing is approximately $27.30/user/month [24].

Where it falls short. Proprietary, expensive, and tightly coupled to PowerPoint. There is no text-format input (no YAML/JSON), no CLI, no version-control workflow, and no programmatic (agent) generation path. Every change requires a human with PowerPoint. Git-diffing a `.pptx` file is not meaningful. The tool is excellent at its intended task (consulting slide production) but solves a different problem from Timeline Compiler.

0.10.2.7 PowerPoint Timeline (native SmartArt)

PowerPoint includes SmartArt-based timeline shapes [15].

Where it falls short. Fully manual. No source format, no determinism, no agent path, no version control. Visual quality is constrained by SmartArt templates; professional roadmaps almost always require manual box placement, which is time-consuming and fragile when dates change.

0.10.3 Comparison Table

Table 34 scores each tool on seven axes relevant to Timeline Compiler's goals. Ratings are qualitative (H = High / M = Medium / L = Low) based on the analysis above and are not precise benchmarks.

Table 34: Comparison of timeline and roadmap tools on axes relevant to Timeline Compiler. H = High, M = Medium, L = Low. "Agent-gen" = suitability for automated generation by an LLM/agent. "Determinism" = stable, reproducible output from the same input. "SCM" = source-control management friendliness.

Tool	Pres. quality	Determinism	SCM	Agent-gen	Open source	PPTX out	SVG/PDF out
Mermaid timeline [9]	M	M	H	H	H	L	M
Mermaid gantt [10]	L	M	H	H	H	L	M
PlantUML [18]	L	H	H	M	H	L	H
vis-timeline [25]	M	L	L	L	H	L	L
Frappe Gantt [4]	M	L	L	L	H	L	L
MS Project [14]	M	L	L	L	L	L	M
think-cell [24]	H	M	L	L	L	H	L
PowerPoint [15]	M	L	L	L	L	H	L
Timeline Compiler (target)	**H**	**H**	**H**	**H**	**H**	**H**	**H**

The table shows that no existing tool simultaneously achieves high scores across all seven axes. think-cell scores highest on presentation quality but is closed-source with no agent path. Mermaid scores highest on openness and SCM friendliness but is limited in presentation fidelity and output format breadth. Timeline Compiler's design targets every axis at High — a combination that is, at the time of writing, unoccupied.

0.10.4 Where Timeline Compiler Intentionally Differs

Three design choices explicitly diverge from the dominant approaches in this landscape.

1. Visual-communication grammar, not task-scheduling grammar. Mermaid's `gantt`, MS Project, and Frappe Gantt all model a project schedule: tasks have durations, dependencies, progress percentages, and critical paths. Timeline Compiler's IR models a *narrative* about time: events, spans, milestones, and swimlane groups, with an explicit visual-communication intent. Resource allocation, dependency graphs, and critical-path analysis are intentionally out of scope. The Grammar of Graphics [27] and Vega-Lite's declarative approach [21] inform this separation of "what to show" from "how to render it".

2. Deterministic rendering as a first-class property. Mermaid's layout is viewport-dependent; MS Project XML changes on every save; PowerPoint files are binary blobs. Timeline Compiler requires that identical IR input produces bit-identical output. This property is essential for CI/CD pipelines, git-based review, and reliable LLM round-trips.

3. Agent generation as a primary use case. Existing tools treat human-in-the-loop as the default workflow: a user opens an application, adjusts bars, and exports. Timeline Compiler is designed to be driven by an AI agent with no human in the loop: the agent generates a valid IR (JSON or YAML conforming to the Timeline IR schema), the compiler renders it, and the artefact lands in a pull request. This requires a small, well-typed, documented IR that LLMs can target reliably — more analogous to Vega-Lite's JSON grammar [21] than to Mermaid's informal text syntax or MS Project's XML schema.

0.10.5 Real-Data Crawl: Practical Patterns Observed

During the research phase, several real-world examples of the artefacts Timeline Compiler targets were surveyed. The following recurring patterns constrain the IR and rendering model.

1. **Quarter granularity dominates executive roadmaps.** McKinsey/BCG-style roadmaps consistently use Q1–Q4 as the primary time unit, with sub-quarter resolution reserved for milestones. The IR must support a `granularity` hint (year, quarter, month, week) so the renderer scales the time axis appropriately.
2. **Swimlane + milestone is the universal visual idiom.** Horizontal swimlanes (one per product, workstream, or business unit) with diamond or dot milestones at key dates is the dominant structural pattern across consulting firms, product teams, and programme offices. The IR needs first-class `lane` and `milestone` elements.
3. **Spans and points coexist.** Real roadmaps mix spans ("Migration: Q2–Q4 2025") and point events ("GA Launch: 15 Sep 2025"). Both must be first-class IR entities.
4. **Azure DevOps and GitHub Projects are dominant data sources.** ADO work items [13] carry iteration path (`System.IterationPath`) for sprint/quarter mapping. GitHub Projects [6] use `DATE` and `ITERATION` custom fields for roadmap bars. An ingestion layer must map these fields to IR span/milestone events.
5. **Mermaid syntax is the lowest-common-denominator input.** Mermaid is already widely embedded in documentation. An ingester that accepts Mermaid `timeline` and `gantt` blocks lowers the migration barrier significantly.
6. **Manual PowerPoint is the current state-of-the-art for presentation quality.** The gap between what Mermaid can produce and what think-cell produces is large. Meeting (or approaching) think-cell output quality is the primary rendering challenge.

0.11 MVP Design

This section defines the Minimum Viable Product for Timeline Compiler, identifying must-have features, phased delivery, risks, and complexity hotspots.

0.11.1 MVP Philosophy

The MVP must answer one question definitively: *Can Timeline Compiler produce presentation-quality timelines from structured input, with deterministic output and theme-driven styling?*

Everything else — advanced features, edge cases, integrations — can follow once this core loop is validated. The MVP is not a prototype; it is a shippable product with limited scope.

0.11.2 Feature Prioritization

0.11.2.1 Must Have (P0)

These features are required for MVP launch:

IR Parser and Validator
Parse YAML input conforming to the Timeline IR schema. Validate against JSON Schema. Report clear, actionable errors.

Core IR Elements
Support for:

- `timeline` root with `title` and `range`
- `tracks` (swimlanes)
- `activities` with labels, date ranges, status, and progress
- `milestones` with dates and labels

Deterministic Layout Engine
Compute positions for all elements deterministically. Handle overlapping activities within tracks (stacking or compression).

SVG Output
Primary output format. Clean, well-structured SVG suitable for web embedding and further processing.

PDF Output
Print-quality vector output for documents and decks.

Basic Theme Support
At least one built-in theme (`default`). Theme defines colors, typography, spacing, and shapes.

CLI
Command-line interface with:

- `timeline render <input> -o <output>` — render to file

- `timeline validate <input>` — validate IR without rendering
- `-theme <name>` — select theme
- `-format <svg|pdf>` — output format (or infer from extension)

Schema Documentation
Published JSON Schema for the IR, with human-readable documentation.

0.11.2.2 Should Have (P1)

These features significantly improve usability and should be included if feasible:

PNG Output
Rasterized output for contexts requiring bitmap images.

Multiple Built-in Themes
At least 3 themes: `default`, `minimal`, `corporate`. Demonstrates theme system versatility.

Sections
Temporal divisions (e.g., quarters, phases) that span all tracks with visual demarcation.

Status Visualization
Color or icon mapping for status values (`planned`, `in-progress`, `complete`, `at-risk`).

Progress Bars
Visual representation of `progress` (0.0–1.0) within activity bars.

Today Marker
Vertical line at a specified "today" date.

npm Package
Core library published to npm for embedding.

Custom Theme Loading
Load themes from user-provided YAML/JSON files.

0.11.2.3 Nice to Have (P2)

These features are desirable but not blocking for MVP:

PPTX Output
PowerPoint-compatible output. Complex due to PPTX format internals.

VS Code Extension
Live preview, syntax highlighting. High value but significant development effort.

Groups
Nested groupings within tracks for hierarchical organization.

Annotations
Callouts and emphasis markers for specific dates or activities.

Dependency Arrows
Visual arrows between activities (visual only, no scheduling).

Legend Generation
Auto-generated legends for status colors or track semantics.

Watch Mode
CLI watches input file and re-renders on change.

MCP Server
Agent integration. Important for long-term vision but not required for initial launch.

0.11.2.4 Future (P3)

These features are out of scope for MVP but part of the long-term vision:

Theme Marketplace
Community-contributed themes.

Multi-file IR Composition
`$ref` or include mechanisms for large timelines.

External Image Embedding
Logos, icons from external files.

Interactive SVG
Clickable regions, tooltips (breaks determinism constraints — careful design needed).

Hosted Rendering API
Cloud endpoint for rendering without local install.

Adapter Ecosystem
Tools that convert Jira, ADO, GitHub Projects to Timeline IR.

Localization
Date formats, RTL support, translated labels.

0.11.3 MVP Scope Summary

Table 35: MVP Feature Summary

Priority	Label	Features
P0	Must Have	IR parser, validation, tracks, activities, milestones, layout engine, SVG, PDF, CLI, 1 theme, schema docs
P1	Should Have	PNG, 3 themes, sections, status colors, progress bars, today marker, npm package, custom themes
P2	Nice to Have	PPTX, VS Code extension, groups, annotations, dependency arrows, legends, watch mode, MCP server
P3	Future	Theme marketplace, multi-file IR, images, interactive SVG, hosted API, adapters, localization

0.11.4 Phased Roadmap

0.11.4.1 Phase 1: Core Engine (MVP Launch)

Duration: 6–8 weeks

Deliverables:

- IR schema (JSON Schema) finalized and documented
- Parser and validator
- Layout engine (deterministic)
- SVG renderer
- PDF renderer (via SVG conversion or direct)
- CLI with `render` and `validate` commands
- Default theme
- Public repository, MIT license, README, contributing guide

Exit Criteria:

- A 3-track, 10-activity roadmap renders to SVG and PDF
- Output is byte-identical across runs
- CLI works on macOS, Linux, Windows

0.11.4.2 Phase 2: Polish and Usability

Duration: 4–6 weeks (overlapping with Phase 1 completion)

Deliverables:

- PNG output
- 2 additional themes (`minimal`, `corporate`)
- Sections support
- Status and progress visualization
- Today marker
- npm package published
- Custom theme loading from file

Exit Criteria:

- Users can author themes without modifying core code
- npm package installable and functional

0.11.4.3 Phase 3: Ecosystem

Duration: 8–12 weeks

Deliverables:

- VS Code extension (syntax highlighting, live preview, export)
- MCP server (render tool for agents)
- PPTX output

- Watch mode in CLI
- Groups and annotations in IR

Exit Criteria:

- AI agent can generate IR and invoke rendering
- VS Code extension in marketplace

0.11.4.4 Phase 4: Scale and Community

Duration: Ongoing

Deliverables:

- Theme contribution workflow
- Adapter examples (Jira, GitHub, ADO)
- Documentation site
- Advanced IR features (dependency arrows, legends)
- Performance optimization for large timelines

0.11.5 Risks and Mitigations

0.11.5.1 Technical Risks

PDF Generation Complexity
PDF output requires either converting SVG (quality/fidelity concerns) or direct PDF writing (complex). *Mitigation:* Use proven libraries; accept minor fidelity differences in MVP; iterate.

Font Determinism
Different systems have different fonts. Varying fonts break determinism. *Mitigation:* Require explicit font specification in themes; bundle fallback fonts; document font requirements.

Layout Edge Cases
Overlapping activities, long labels, dense timelines may produce poor layouts. *Mitigation:* Start with simple layout rules; document limitations; accept that MVP handles "typical" cases, not all.

Cross-Platform Rendering Differences
Floating-point differences across platforms may affect layout. *Mitigation:* Use integer math or fixed-point where possible; test on multiple platforms early.

0.11.5.2 Adoption Risks

Learning Curve
Users must learn the IR format. *Mitigation:* Excellent examples, quick-start guide, VS Code extension with auto-complete.

Theme Quality
If built-in themes are unappealing, users won't adopt. *Mitigation:* Invest in theme design; consult with visual designers; study exemplary slides.

Missing Feature X
Users may expect Gantt-like features (dependencies, resources) that are out of scope. *Mitigation:* Clear documentation of scope boundaries; position as "communication tool, not PM tool."

0.11.5.3 Complexity Hotspots

PPTX Output
The PPTX format (Office Open XML) is complex. Producing valid, editable PPTX files is non-trivial. *Mitigation:* Defer to Phase 3; use established libraries; accept limited editability initially.

Theme System
Balancing flexibility with simplicity is hard. Too simple: users can't customize. Too complex: themes become a programming language. *Mitigation:* Start minimal; extend based on feedback; keep theme schema documented.

Layout Engine
Deterministic layout that handles edge cases (long text, overlap, many tracks) is complex. *Mitigation:* Define clear layout rules; document limitations; iterate.

0.11.6 Smallest Viable Version

If resources are constrained, the absolute smallest viable version is:

- IR schema for `tracks`, `activities`, `milestones`
- YAML parser and validator
- SVG output only
- CLI with `render` command only
- Single hardcoded theme
- No npm package (CLI only)

This version proves the core thesis: structured input to presentation-quality output, deterministically. Everything else is enhancement.

0.11.7 Success Metrics

Adoption
GitHub stars, npm downloads, VS Code extension installs.

Quality
Issue count, bug severity, user-reported rendering defects.

Determinism
CI tests that verify byte-identical output across runs and platforms.

Agent Usage
Number of agents using MCP server (post-Phase 3).

Community Contribution
Theme contributions, adapter contributions, PRs.

0.12 Open-Source Strategy

0.12.1 Is This a Viable Open-Source Project?

The short answer is yes, but with conditions. The diagram-as-code category has demonstrated commercial and community viability at scale: Mermaid [11] has over 88,000 GitHub stars, raised $7.5M in seed funding in 2024 [8], and has been natively integrated into GitHub, GitLab, Notion, Obsidian, and Azure DevOps. The lesson from Mermaid is that a plain-text format with zero-install rendering, embedded in documentation ecosystems, can achieve massive adoption.

Timeline Compiler's gap analysis (Section 0.10) identifies a cell in the tool landscape that is genuinely unoccupied: an *open-source*, *presentation-quality*, *agent-generation-friendly* timeline compiler. This gap is both an opportunity and a risk; the following sections analyse both.

0.12.2 Closest Existing Projects and the Gap They Leave

- **Mermaid** (§0.10) leaves the presentation-quality gap. Its `timeline` and `gantt` types are documentation tools, not slide-production tools. No PPTX target. No theme engine. Layout is viewport-dependent, not deterministic.
- **TimelineJS** [7] (Knight Lab, GPL) is the best-known open-source timeline tool, but it is a web-storytelling widget (iframes, Google Sheets), not a document/slide compiler. No SVG/PPTX/PDF output; no CLI; no agent path.
- **Frappe Gantt** [4] and **vis-timeline** [25] are interactive browser widgets. No declarative text format; no static artefact output.
- **think-cell** [24] produces the closest visual quality but is closed-source, expensive, and PowerPoint-locked. It is the benchmark for what Timeline Compiler output should look like, not a substitute for it.
- **Plotly.js** [20] can produce high-quality timeline charts, but requires programming and has no declarative timeline grammar or agent-generation story.

The gap is real: the market has diagram-as-code (low quality, wrong idiom) or presentation tools (high quality, closed, manual). Nothing sits in between.

0.12.3 Potential Adopters

Four adopter personas have been identified based on real-data patterns observed during the research phase:

Developer-Relations and Technical Writers. Dev-rel teams already live in Mermaid and Markdown. They need to produce roadmaps and architecture-evolution timelines for conference talks, blog posts, and product docs. The barrier to adoption is near-zero if Timeline Compiler has a CLI, a Markdown code-fence syntax, and SVG output. This group provides early adopters and community feedback.

Product Managers and Programme Leads. PMs need quarterly roadmaps and programme timelines for steering committees and executive reviews. They currently use PowerPoint or think-cell manually. A compiler that takes a YAML/CSV definition and emits a PPTX with think-cell-quality layout would replace hours of work per roadmap cycle. This group provides the strongest word-of-mouth growth vector.

Management Consultants (Freelance / Boutique). Large firms use think-cell. Freelancers and boutique consultancies cannot justify $27.30/user/month per project. An open-source tool with a polished PPTX target has a credible value proposition here.

AI Agent Toolchains. The emergent user is not a human at all. LLM-based planning agents (GitHub Copilot, OpenAI Assistants, Anthropic Claude with tools) generate project plans, architecture proposals, and transformation roadmaps. Today there is no standard open-source *target format* for those plans. Timeline Compiler's IR, if published as a JSON Schema and exposed as an MCP tool [3], becomes the target format AI agents compile to. This persona drives long-tail adoption as agent toolchains proliferate.

0.12.4 Ecosystem Fit

0.12.4.1 Mermaid / Markdown / CI Ecosystem

Timeline Compiler should position itself as the "next step" after Mermaid: use Mermaid in your README; use Timeline Compiler when you need a slide. Concretely:

- A Mermaid-syntax ingester (accepting `timeline` and `gantt` blocks) provides a zero-friction migration path.
- A Markdown code-fence processor (similar to Mermaid's own fence renderer) enables embedding timelines in MkDocs, VitePress, and Quarto [11].
- A GitHub Actions step for CI-driven rendering integrates into the workflow teams already use for diagram-as-code.
- SVG output embeds directly in Slidev [2] and Obsidian [16] — both are Mermaid-native environments.

0.12.4.2 MCP / Agent Ecosystem

The Model Context Protocol [3] is the emerging standard for LLM tool invocation. An MCP server that exposes Timeline Compiler as a tool would allow any MCP-compatible agent (GitHub Copilot, Claude, etc.) to:

1. Accept a natural-language plan from the user.
2. Generate a valid Timeline IR (JSON/YAML, constrained by the published JSON Schema).
3. Invoke the Timeline Compiler MCP tool.
4. Receive back an SVG, PNG, or PPTX and embed it in the agent's response.

OpenAI's Structured Outputs feature [17] (JSON Schema-constrained generation) enables agents to reliably produce valid IR without hallucinating schema fields. The design implication is that the Timeline IR schema *must* be published as a JSON Schema document alongside the compiler.

0.12.4.3 PPTX Ecosystem

python-pptx [22] is the leading open-source Python library for programmatic PPTX generation. It is MIT-licensed, well-maintained, and can produce slides that open natively in PowerPoint and Keynote. Building the PPTX output target on python-pptx avoids a dependency on LibreOffice or headless Chrome, making the compiler pip-installable with minimal system requirements.

0.12.5 Extension Opportunities

The compiler-IR architecture creates natural extension points:

Ingesters.
New data sources can be added without touching the renderer: CSV/Excel, Jira API, Linear API, Notion database, ADO work items [13], GitHub Projects [6], markdown plans, Mermaid blocks. Ingesters are the highest-traffic contribution path for community members.

Themes.
A typed theme schema (fonts, colour palettes, swimlane heights, milestone shapes) allows community contribution of brand themes without touching rendering code. McKinsey-style, BCG-style, GitHub-style, and corporate-brand themes are natural community contributions.

Output Targets.
Additional renderers (HTML interactivity, Keynote, Google Slides API, LaTeX beamer) can be added without changing the IR or ingesters. The Vega-Lite model [21] demonstrates that a clean IR/renderer separation enables this pattern reliably.

Grammar Extensions.
The IR can be versioned and extended (new event types, annotations, dependency arrows) without breaking existing themes or renderers, following established IR versioning practice from compiler design [1].

0.12.6 Strongest Differentiators

Three differentiators are assessed as strongest, in decreasing order of defensibility:

1. **The only open-source, agent-generation-friendly timeline compiler with PPTX output.** No existing open-source tool combines a stable text-format IR, deterministic rendering, and a PPTX output target. This is the clearest gap in the landscape.
2. **Visual-communication grammar vs. task-scheduling grammar.** Positioning the tool as a "timeline grammar for human communication" rather than a "project management tool" is a genuine conceptual distinction. It attracts users who are currently underserved by Mermaid (insufficient quality) and repelled by MS Project (wrong abstraction).
3. **AI-agent-first design.** Publishing a JSON Schema for the IR and an MCP tool interface makes Timeline Compiler the natural compiler target for any LLM that generates plans or roadmaps. As agent-driven workflows proliferate [3], this positions Timeline Compiler at the centre of a growing distribution channel with no human adoption friction.

0.12.7 Community Contribution Model

The Mermaid model [8] is instructive: open-source core (MIT), commercial SaaS layer for hosted rendering, team features, and enterprise integrations. Timeline Compiler should follow a similar open-core pattern:

- **Core compiler** (MIT): IR schema, renderer, CLI, standard output targets, standard ingesters. All of this is open-source. Community contributions to ingesters and themes are accepted here.
- **Theme marketplace** (community-driven): A GitHub-hosted registry of community themes, analogous to the npm registry for packages or the VS Code extension marketplace. Themes are independent packages; they do not require merging into the core.
- **Hosted service** (future, optional): Web UI for non-technical users, CI integration hooks, and enterprise SSO. This funds maintenance without restricting the open-source core.

The contribution surface most likely to attract community contributions (in descending order of probability) is: ingesters, then themes, then additional output targets, then IR schema extensions. Core renderer changes require higher trust and should require maintainer review.

0.12.8 Adoption Risk Assessment

No adoption analysis is honest without a candid risk section.

Risk 1: The presentation-quality bar is high. think-cell is the quality benchmark. Matching it with an open-source, text-driven renderer is a significant engineering challenge. If the initial PPTX output is visually inferior to think-cell, the "presentation quality" differentiator collapses, and Timeline Compiler is merely a less capable Mermaid. *Mitigation*: Define a strict visual quality bar in the design spec and validate against real executive roadmaps before launch.

Risk 2: The IR may be too narrow or too wide. An IR that is too narrow (only covering the simplest roadmaps) will be inadequate for real-world use. An IR that is too wide (attempting to model every possible timeline idiom) will be complex to understand, hard for agents to generate reliably, and fragile to maintain. *Mitigation*: Scope the IR to the five canonical timeline types identified in the scope section, and use real data crawls (ADO exports, GitHub Projects exports) to validate coverage before freezing the schema.

Risk 3: Mermaid is a gravity well. Mermaid has 88,000 stars, is embedded everywhere, and is improving. If Mermaid's `timeline` type adds PPTX export and better theming, the core differentiator weakens. *Mitigation*: The visual-communication grammar thesis and the agent-generation design are harder to retrofit into Mermaid than a PPTX renderer. The IR schema and MCP interface are the durable moats.

Risk 4: Agent toolchains may standardise on Mermaid instead. LLMs already reliably generate Mermaid syntax. If the agent ecosystem converges on Mermaid + post-processing as the standard agent-to-timeline pipeline, the agent-first positioning loses force. *Mitigation*: Publish the JSON Schema early; build the MCP tool before the ecosystem settles. Structured Outputs [17] are demonstrably more reliable than free-text Mermaid generation for complex schemas.

Risk 5: PPTX format instability. The OOXML PPTX specification is controlled by Microsoft. python-pptx [22] may lag new PowerPoint features. *Mitigation*: Treat PPTX as an output layer, not the IR. SVG and PDF are canonical; PPTX is a convenience export. Degrade gracefully when PPTX features are unavailable.

0.13 Timeline Grammar vs. Task Scheduling Grammar

This section presents the central thesis of Timeline Compiler: the correct abstraction for this problem is a **Timeline Grammar** (optimized for visual communication) rather than a **Task Scheduling Grammar** (optimized for project management). This distinction is not merely terminological — it fundamentally shapes the IR design, scope boundaries, and long-term extensibility.

0.13.1 The Thesis

> *Gantt-thinking is the wrong default. Timeline Compiler should not model tasks, dependencies, and schedules. It should model visual narratives: tracks, phases, milestones, emphasis, and annotations. The IR is a grammar for visual communication, not a grammar for work decomposition.*

0.13.2 Two Grammars Compared

0.13.2.1 Task Scheduling Grammar

A Task Scheduling Grammar models *work to be done*. Its primitives:

Tasks
Discrete units of work with effort estimates, assignees, and dependencies.

Dependencies
Relationships between tasks (finish-to-start, start-to-start, etc.) that constrain scheduling.

Resources
People, teams, or assets assigned to tasks, subject to capacity constraints.

Dates
Computed from dependencies and resource availability, not directly authored.

Critical Path
The longest chain of dependent tasks; determines project duration.

Milestones
Markers for task completion or deliverable readiness, often derived from task dependencies.

This grammar powers MS Project, Primavera, Jira Advanced Roadmaps, and enterprise scheduling tools. Its purpose is *operational*: compute feasible schedules, optimize resource allocation, identify bottlenecks.

A Gantt chart is the canonical visualization of a task scheduling grammar. It shows tasks as bars, dependencies as arrows, resources as assignments, and the critical path highlighted. The visual is a *byproduct* of the scheduling model.

0.13.2.2 Timeline Grammar

A Timeline Grammar models *communication about time*. Its primitives:

Tracks
Visual lanes that organize content by theme, workstream, or category — chosen for *narrative clarity*, not operational structure.

Activities
Time-bounded visual elements representing initiatives, phases, or efforts. Dates are directly authored, not computed. Activities have no inherent dependencies.

Milestones
Emphasized points in time representing decisions, deliverables, or checkpoints. Their meaning is communicative, not operational.

Sections
Temporal divisions (quarters, phases, years) that provide visual structure and context.

Annotations
Callouts, emphasis, and labels that direct attention.

Status and Progress
Visual indicators that communicate state to an audience, not operational data for a scheduler.

This grammar powers McKinsey roadmaps, BCG transformation timelines, board-deck quarterly plans, and executive communication artifacts. Its purpose is *rhetorical*: convey a plan, tell a story about time, align stakeholders.

0.13.3 Why Gantt-Thinking Is Wrong Here

If Timeline Compiler adopted a Task Scheduling Grammar, it would inherit these problems:

0.13.3.1 Computational Complexity

Task scheduling is NP-hard in the general case (resource-constrained project scheduling). A scheduling grammar implies:

- Dependency resolution algorithms
- Constraint satisfaction
- Resource leveling
- What-if scenario computation

None of this is relevant to producing a PDF for a board presentation. It is massive accidental complexity.

0.13.3.2 Semantic Overload

A scheduling grammar interprets user input operationally. If users declare:

```
activities:
  - id: design
    depends_on: [requirements]
```

A scheduling grammar must:

- Validate that `requirements` exists
- Compute `design`'s start date from `requirements`' end date
- Propagate changes transitively
- Detect cycles

A timeline grammar simply draws an arrow from `requirements` to `design`. The dependency is a *visual annotation*, not a scheduling constraint.

0.13.3.3 Schema Bloat

Scheduling grammars require:

- Resource definitions (people, teams, capacities)
- Effort estimates (person-days, story points)
- Calendars (working days, holidays)
- Dependency types (FS, SS, FF, SF with lags)
- Constraint types (must-start-on, no-earlier-than)

None of this serves the communication use case. It bloats the IR, confuses authors, and distracts from the actual goal.

0.13.3.4 Agent Hostility

LLMs can easily generate timeline grammars. "Show Platform work from January to March, with a milestone in February" maps directly to:

```
tracks:
  - label: "Platform"
    activities:
      - label: "Core Development"
        range: { start: 2026-01, end: 2026-03 }
milestones:
  - label: "Alpha Release"
    date: 2026-02-15
```

Agents struggle with scheduling grammars because:

- Dependencies must be globally consistent (cycles are invalid)
- Dates must satisfy constraint propagation
- Resource assignments must respect capacities

These invariants are hard to maintain without iterative refinement. Timeline grammars are *flat*: each element is independently valid.

0.13.3.5 The Gantt Trap

Gantt charts are so ubiquitous that "timeline" often defaults to "Gantt chart." But Gantt charts are poor communication artifacts:

- Dense with dependency arrows that obscure the narrative
- Emphasize task-level detail over strategic phases
- Require operational context (resources, constraints) that audiences lack
- Visually noisy — optimized for schedulers, not executives

Timeline Compiler explicitly rejects the Gantt default. It produces *timeline visualizations*, not Gantt charts. Gantt-style output may be achievable via themes, but it is not the design center.

0.13.4 Timeline Grammar: Design Implications

Adopting a Timeline Grammar shapes the architecture:

0.13.4.1 IR Simplicity

The IR is a flat description of visual elements:

```
timeline:
  title: "Product Roadmap 2026"
  range: { start: 2026-01, end: 2026-12 }
  tracks:
    - id: platform
      label: "Core Platform"
      activities:
        - label: "API v2"
          range: { start: 2026-01, end: 2026-04 }
        - label: "Scale Testing"
          range: { start: 2026-05, end: 2026-07 }
  milestones:
    - label: "GA Launch"
      date: 2026-08-01
```

No dependencies to resolve. No resources to level. No constraints to satisfy. The IR *is* the plan, directly.

0.13.4.2 No Computation

The compiler performs no computation on plan semantics:

- Dates are taken as given
- Progress is taken as given
- Status is taken as given

The compiler's job is *layout and rendering*, not interpretation.

0.13.4.3 Visual-First Extensions

Future extensions follow the timeline grammar:

- **Annotations** — callouts, emphasis, arrows (visual, not operational)
- **Sections** — phase labels, quarter markers (narrative structure)
- **Legends** — explaining visual encodings (communication aid)
- **Dependency arrows** — optional visual hints, not scheduling constraints

Extensions that imply computation (resource leveling, critical path) are rejected by the grammar's philosophy.

0.13.4.4 Determinism by Default

Timeline grammars are inherently deterministic. There is no computation that could introduce variability. The IR fully specifies the content; the theme fully specifies the style; the renderer maps one to the other.

Scheduling grammars resist determinism because scheduling is sensitive to algorithm implementation, floating-point accumulation, and tie-breaking heuristics.

0.13.5 Addressing the Counter-Arguments

0.13.5.1 "But I need dependency arrows"

Response: Timeline Compiler can render dependency arrows as visual annotations. The IR may include:

```
activities:
  - id: alpha
    label: "Alpha Phase"
    range: { start: 2026-01, end: 2026-03 }
  - id: beta
    label: "Beta Phase"
    range: { start: 2026-04, end: 2026-06 }
    depends_on: [alpha]  # Visual arrow, not scheduling constraint
```

The `depends_on` field means "draw an arrow from alpha to beta." It does not mean "compute beta's start from alpha's end." The dates are author-provided. The arrow is visual.

0.13.5.2 "What if my dates are inconsistent?"

Response: The validator may warn if beta starts before alpha ends (given a declared dependency). But this is a *warning*, not an error. The compiler renders what the author specified. Fixing inconsistencies is the author's job, or an upstream tool's job.

This matches the communication use case. A roadmap slide may intentionally show overlapping phases. The compiler should not second-guess the author.

0.13.5.3 "Project management tools already have timeline views"

Response: True, and those views are outputs of a scheduling grammar. They:

- Look like their source tool (Jira exports look like Jira)
- Include scheduling clutter (dependencies, resources)
- Require the source tool to generate

Timeline Compiler is a *standalone* rendering target. It accepts a simple IR that any tool (or agent) can generate. It produces presentation-quality output that looks like a designed slide, not a tool export.

0.13.5.4 "Won't users expect scheduling features?"

Response: Clear positioning is essential. Timeline Compiler is a *communication compiler*, not a scheduling engine. Users who need scheduling should use MS Project, Jira, or Linear — and then export to Timeline IR for rendering.

This is the LaTeX model: LaTeX is a typesetting system, not a word processor. It does one thing excellently. Timeline Compiler is a timeline renderer, not a project manager.

0.13.6 Optimization Goals Alignment

The brief specifies five optimization goals. Timeline Grammar aligns with all five:

Presentation Quality
Timeline Grammar focuses on visual communication, not operational detail. Outputs are clean, narrative-driven, and boardroom-ready.

Agent Generation
Timeline Grammar is flat and simple. Agents generate valid IR without constraint satisfaction. The schema fits in a context window.

Deterministic Rendering
No scheduling computation means no computational variability. Same IR, same output, always.

Long-Term Extensibility
The grammar is a foundation for visual extensions (annotations, animations, interactive elements) without importing scheduling complexity.

Simplicity
The IR is small. The compiler is focused. There is no hidden complexity in dependency resolution or resource leveling.

0.13.7 Conclusion

Timeline Compiler adopts a **Timeline Grammar**: a model for visual communication about time, not operational scheduling. This is the correct abstraction because:

1. It produces presentation-quality visuals without scheduling complexity.
2. It is agent-friendly: flat, simple, constraint-free.
3. It is deterministic by construction.
4. It is extensible along visual dimensions without inheriting scheduling scope creep.
5. It is simple: the IR describes what to render, not what to compute.

The Gantt chart is not the design center. The McKinsey slide is. Timeline Compiler is a compiler from structured communication intent to visual artifact — and that is all it should be.

Bibliography

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, 2 edition, 2006. ISBN 978-0-321-48681-3. The "Dragon Book". Chapter 5 covers intermediate representations. Informs the argument that a stable, typed IR is preferable to direct rendering from raw input.

[2] Anthony Fu and contributors. Slidev: Presentation slides for developers. https://sli.dev/, 2024. MIT-licensed Markdown-first slide tool with Mermaid integration. Relevant as a downstream consumer of Timeline Compiler SVG output.

[3] Anthropic. Model context protocol — specification. https://spec.modelcontextprotocol.io/, 2024. Open protocol for LLM tool/resource access. Defines how agents invoke external tools and consume structured outputs. Timeline Compiler can expose MCP tools for plan-to-timeline generation.

[4] Frappe Technologies. Frappe gantt: A modern, configurable gantt library for the web. https://github.com/frappe/gantt, 2024. MIT license. SVG-based interactive Gantt; used in ERPNext. No declarative text-format input; requires JavaScript task objects. No static file export built-in.

[5] GitHub, Inc. ProjectV2 — GitHub GraphQL API reference. https://docs.github.com/en/graphql/reference/objects#projectv2, 2024.

[6] GitHub, Inc. About GitHub projects — GitHub documentation. https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects, 2024. ProjectV2 GraphQL API fields include `DATE`, `ITERATION`, `SINGLE_SELECT` custom fields for roadmap bars. Roadmap view became GA in March 2023.

[7] Knight Lab, Northwestern University. TimelineJS: Beautifully crafted timelines that are easy, and intuitive to use. https://timeline.knightlab.com/, 2024. Open source (GPL). Input via Google Sheets or JSON. Interactive web output; no native SVG/PDF/PPTX export. Storytelling-oriented, not presentation-oriented.

[8] Mermaid Chart, Inc. Mermaid chart: AI-powered diagramming for teams. https://www.mermaidchart.com/, 2024. Commercial SaaS layer on top of open-source Mermaid. $7.5M seed funding (March 2024). Demonstrates the open-core commercial model for diagram tooling.

[9] Mermaid Contributors. Timeline diagram — mermaid documentation. https://mermaid.js.org/syntax/timeline.html, 2024. Official documentation for the `timeline` diagram type, which graduated from beta in 2023. Sections group events; time axis accepts year, quarter, or ISO-8601 strings.

[10] Mermaid Contributors. Gantt diagrams — mermaid documentation. https://mermaid.js.org/syntax/gantt.html, 2024. Documents the `gantt` diagram type with `dateFormat`, `section`, and task syntax. Export quality is SVG/PNG only; no native PPTX or PDF.

[11] Mermaid-js Contributors. Mermaid: Generation of diagrams like flowcharts or sequence diagrams from text in a similar manner as markdown. https://github.com/mermaid-js/mermaid, 2023. MIT license. Over 88,000 GitHub stars as of 2026. Raised $7.5M seed round (Sequoia/M12) in March 2024.

[12] Microsoft Corporation. Microsoft project XML data interchange file format. https://learn.microsoft.com/en-us/office-project/xml-reference/introduction-to-the-microsoft-project-xml-data-interchange-file-format, 2023. Official schema reference for Project XML export format.

[13] Microsoft Corporation. Work items — Get — Azure DevOps REST API reference. https://learn.microsoft.com/en-us/rest/api/azure/devops/wit/work-items/get?view=azure-devops-rest-7.1, 2024. Work items export fields include `System.IterationPath`, `System.State`, `System.Title`, `Microsoft.VSTS.Scheduling.Effort`, etc. Quarter/sprint granularity captured via `IterationPath`.

[14] Microsoft Corporation. Microsoft project: Project management software. https://www.microsoft.com/en-us/microsoft-365/project/project-management-software, 2024. Proprietary. Exports to a custom XML schema (`Project.xsd`) that is non-deterministic between saves (GUIDs, timestamps change). Not git-diffable without post-processing.

[15] Microsoft Corporation. Create a timeline — Microsoft PowerPoint support. https://support.microsoft.com/en-us/office/create-a-timeline-ec28c7c7-7b0a-40da-9b74-9a85f8ab4b97, 2024. Manual SmartArt-based timeline creation in PowerPoint. Not source-controlled; not deterministic; not agent-friendly.

[16] Obsidian. Obsidian: A second brain, for you, forever. https://obsidian.md/, 2024. Popular Markdown knowledge-management tool with native Mermaid rendering. Potential embedding target for Timeline Compiler output.

[17] OpenAI. Structured outputs — OpenAI API documentation. https://platform.openai.com/docs/guides/structured-outputs, 2024. JSON Schema-constrained generation from LLMs. A well-specified Timeline IR with JSON Schema enables agents to generate valid IR directly.

[18] PlantUML Contributors. PlantUML: Open-source tool to draw UML diagrams, using a simple and human readable text description. https://plantuml.com/, 2024. Outputs PNG, SVG, ASCII art, LaTeX, PDF via GraphViz/Ditaa. Gantt support (`@startgantt`) is experimental as of 2024. GPL/LGPL/Commercial licenses.

[19] PlantUML Contributors. Gantt diagram — PlantUML documentation. https://plantuml.com/gantt-diagram, 2024.

[20] Plotly Technologies Inc. Plotly.js: Open source graphing library. https://github.com/plotly/plotly.js, 2024. MIT license. Supports Gantt/timeline charts via horizontal bar charts. High-quality SVG export. Requires programming; no declarative text format.

[21] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-Lite: A grammar of interactive graphics. In *IEEE Transactions on Visualization and Computer Graphics (Proc. InfoVis)*, volume 23, pages 341–350, 2017. doi: 10.1109/TVCG.2016.2599030. High-level declarative JSON grammar for interactive

statistical graphics; BSD-3-Clause open source. Demonstrates how declarative IRs enable deterministic, composable rendering.

[22] Scanny and contributors. python-pptx: Create and update PowerPoint files. https://github.com/scanny/python-pptx, 2024. MIT license. Python library for programmatic PPTX generation. Candidate for the PPTX output target of Timeline Compiler.

[23] SlideScience. Ultimate guide to Gantt charts (timelines) in think-cell. https://slidescience.co/gantt-charts-timelines-think-cell/, 2024.

[24] think-cell Software GmbH. think-cell: Intelligent charting and slide automation software for PowerPoint. https://www.think-cell.com/, 2024. Proprietary PowerPoint add-in. Approx. $27.30/user/month (2026 pricing). Industry standard for McKinsey/BCG-style Gantt and waterfall charts. Not open source; not source-control friendly.

[25] visjs Contributors. vis-timeline: Create fully customizable, interactive timelines and 2d graphs with items and groups. https://github.com/visjs/vis-timeline, 2024. MIT license. Browser-only interactive timeline; no native static SVG/PDF export. Successor to the original vis.js timeline component.

[26] Wikipedia contributors. Mermaid (software) — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Mermaid_(software), 2024.

[27] Leland Wilkinson. *The Grammar of Graphics*. Springer, New York, 2 edition, 2005. ISBN 978-0-387-24544-7. Foundational theory decomposing statistical graphics into data, aesthetics, geometry, statistics, scales, coordinates, and facets. Basis for ggplot2 and Vega-Lite.